
Accurate structure prediction of biomolecular interactions with AlphaFold 3 (Part 1)

— Abramson, J., Adler, J., Dunger, J. et al. —

<https://www.nature.com/articles/s41586-024-07487-w>

From Sequences to Multi-molecular Structures

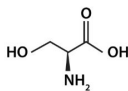
Traditional approaches like X-ray crystallography, Cryo-EM, NMR are **time-consuming, expensive, and sparse**.

AlphaFold 2 revolutionized monomer structure prediction — but biology is more than monomers.

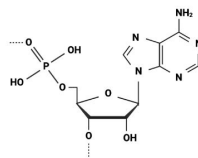
Need for a **unified model** that handles:

- **Multimers**
- **Ligands** (small molecules, ions)
- **RNA and DNA**

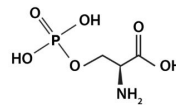
Standard Amino Acid
(serine)



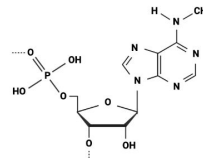
Standard Nucleotide
(adenosine)



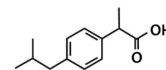
Modified Amino Acid
(phosphoserine)



Modified Nucleotide
(methyladenosine)



Ligand
(ibuprofen)



AlphaFold 3 is not just an extension; it's a **new paradigm**.

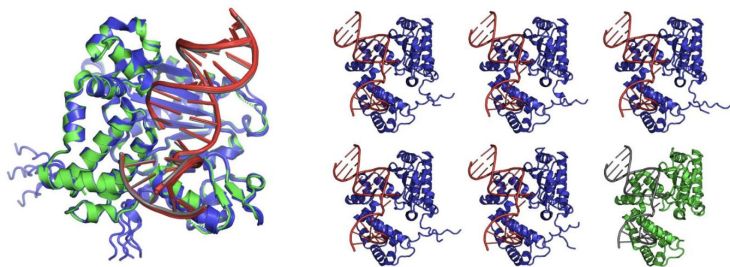
Why Predict Interactions?

- Many cellular processes involve **complex interactions**:

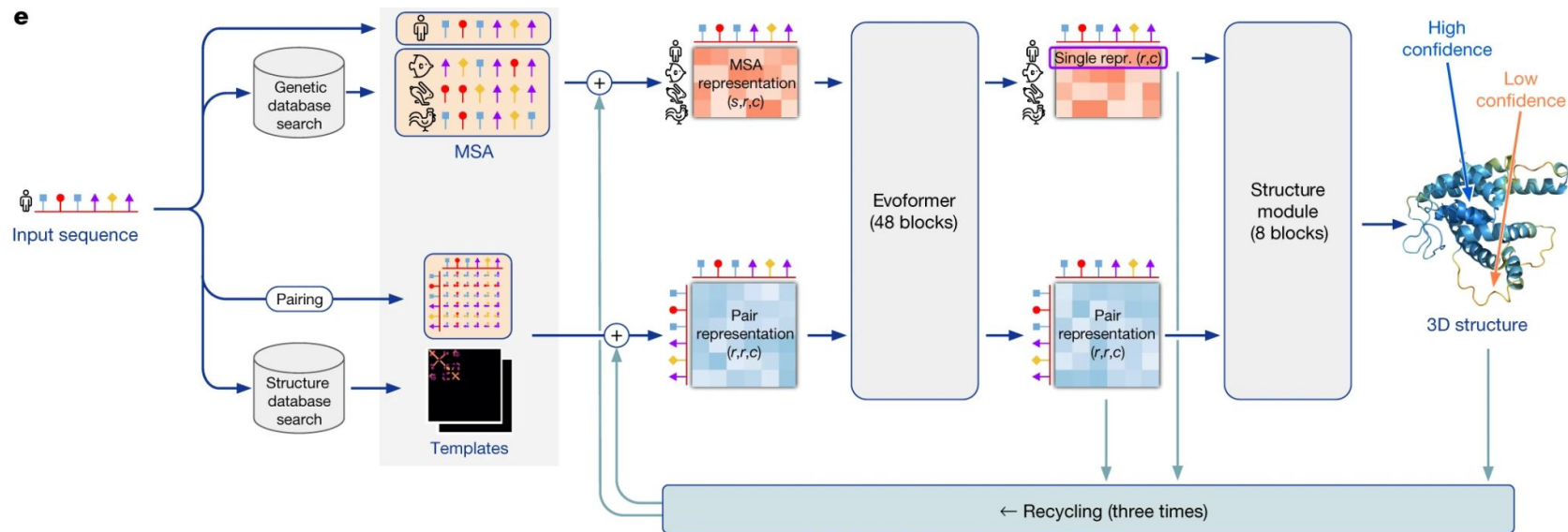
- Protein–protein (multimers)
- Protein–ligand (e.g., drug binding)
- Protein–nucleic acid (e.g., CRISPR-Cas9)

- Structural biology must **generalize across molecules**.
- Predicting *joint structures* requires **flexible representations**.
- Classic models are modular/limited. Need a **single, generalizable model**.

- DNA in complex with globular proteins (5TBA)



From AlphaFold2 to AlphaFold3 – A Paradigm Shift



From AlphaFold2 to AlphaFold3 – A Paradigm Shift

Feature	AlphaFold2 (2021)	AlphaFold-Multimer (2021)	AlphaFold3 (2024)
Input types	Protein sequences	Protein sequences	Proteins, RNA, DNA, ligands
Complex prediction	No	Yes (proteins only)	Yes (all molecule types)
Ligand/RNA/DNA modeling	✗	✗	✓
Underlying architecture	Evoformer + structure module	Extended Evoformer	Pairformer + Diffusion
Training task	Supervised (PDB structures)	Supervised (PDB structures)	Conditional diffusion

AlphaFold 3 vs AlphaFold 2 and AlphaFold-Multimer

1. General Modeling Capabilities

- **AF2 / AF-Multimer:** Focused on protein monomers (AF2) and protein-protein complexes (AF-Multimer).
- **AF3:** Models a broader range of biomolecular complexes, including **ligands, nucleic acids, metals, and modified residues**.

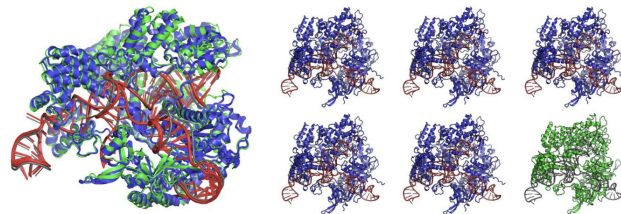
2. Structural Representation

- **AF2:** Uses rigid body frames for amino acids; side chains are parameterized with χ -angles.
- **AF3:** Represents molecules at the **atomic level**, with each atom having **independent global coordinates**—removes rigid constraints and enables modeling of arbitrary molecules.

3. Tokenization

- **AF2:** Residue-centric representations.
- **AF3:** Uses **tokens** for efficient computation:
 - Whole residues/nucleotides for standard biomolecules.
 - Individual heavy atoms for others.

- Conserved RNA-DNA-protein complexes (4O08)



AlphaFold 3 vs AlphaFold 2 and AlphaFold-Multimer

4. Structure Prediction Approach

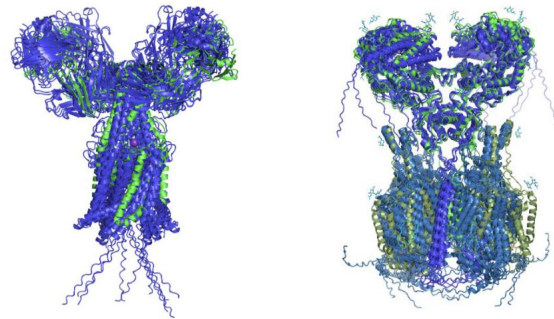
- **AF2:** Uses equivariant spatial transformations and post-relaxation for clash resolution.
- **AF3:** Employs a **generative diffusion model** (adds/removes Gaussian noise) with **minimal spatial inductive bias**. Post-processing/relaxation is rarely needed.

5. Architecture

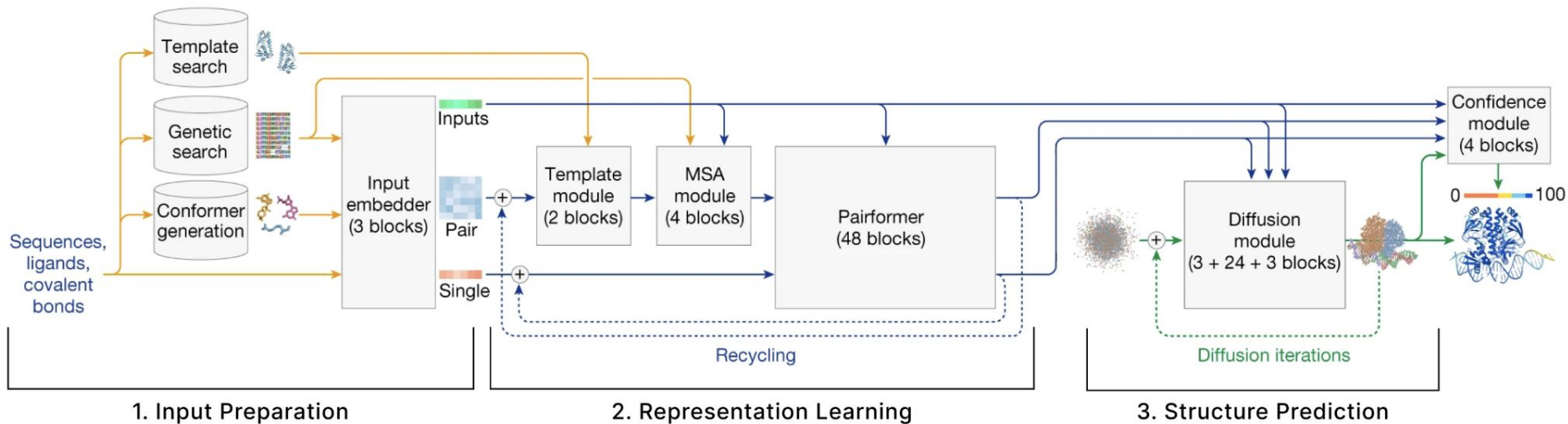
- **AF2:** Uses the **Evoformer** trunk, based on MSA and residue pair representations.
- **AF3:** Replaces Evoformer with **Pairformer**:
 - **No MSA stack** in the core, operates **only on token pairs**.
 - No outer product mean (no single-to-pair flow).

6. Activation and Data

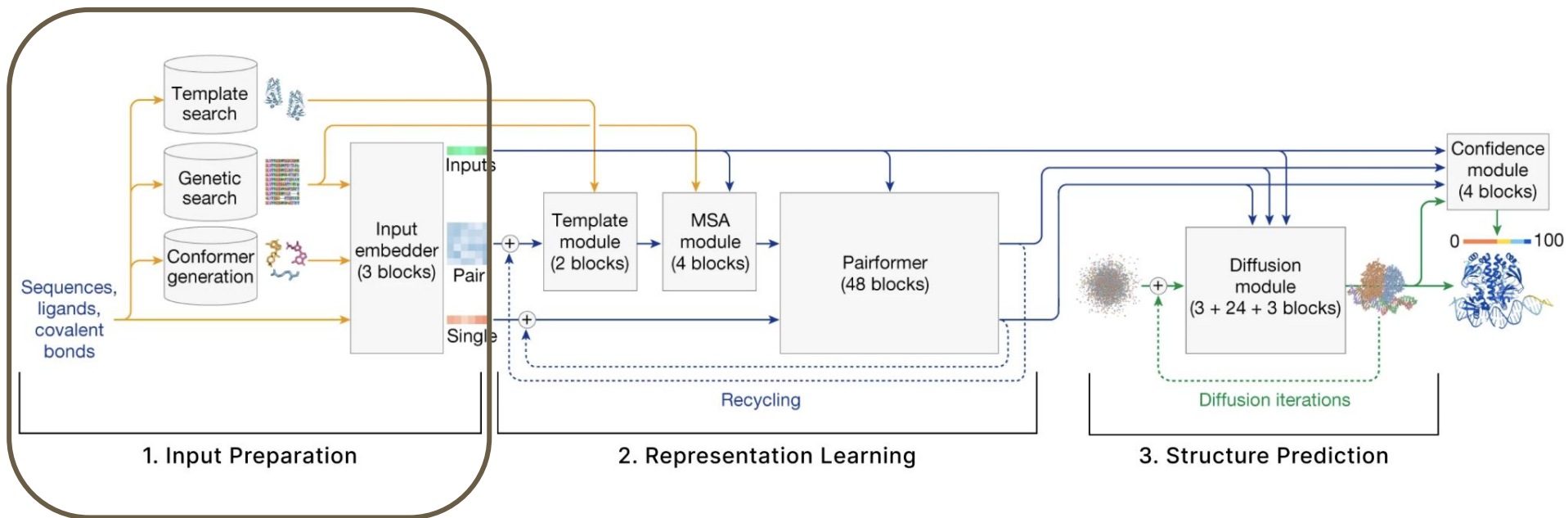
- **AF2:** Uses **ReLU** activations.
- **AF3:** Switches to **SwiGLU** in transition blocks (ReLU still used in atom attention).



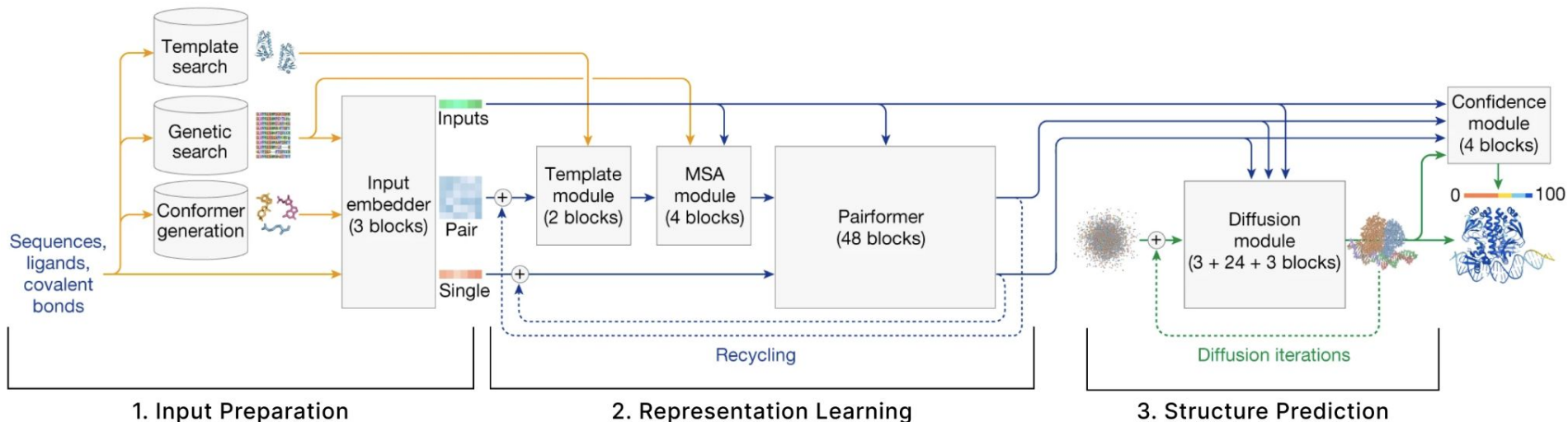
What We'll Cover in Part 1



What We'll Cover in Part 1



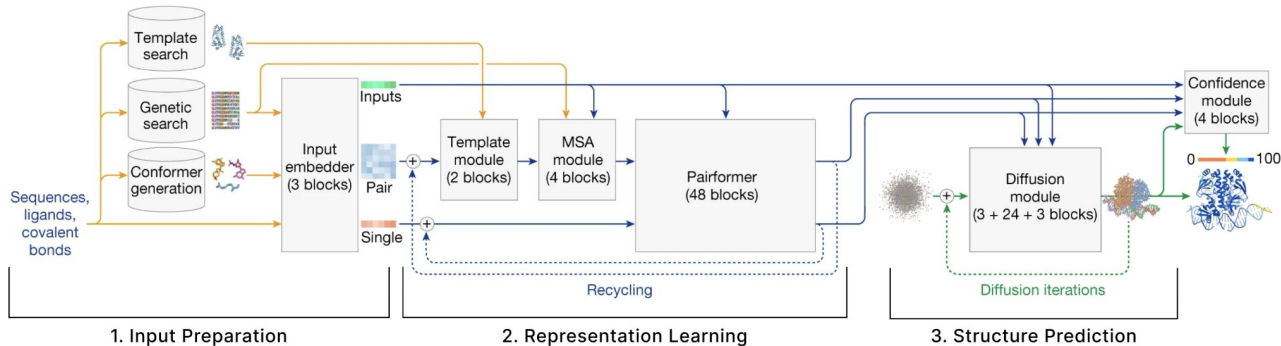
Architecture Overview



Unlike earlier AlphaFold models that only handled standard amino acid sequences, AlphaFold 3 must represent more complex inputs, requiring more advanced tokenization. A "token" can be an amino acid, nucleotide, or atom outside standard residues.

The model has three main parts:

1. **Input Preparation:** Converts user-provided sequences and similar molecules into tensors.
2. **Representation Learning:** Refines the Single and Pair tensors created in section 1 using various attention mechanisms.
3. **Structure Prediction:** Uses refined features and inputs to predict structures via conditional diffusion.



Algorithm 1 Main Inference Loop

def MainInferenceLoop($\{f^*\}$, $N_{\text{cycle}} = 4$, $c_s = 384$, $c_z = 128$) :

```

1:  $\{s_i^{\text{inputs}}\} = \text{InputFeatureEmbedder}(\{f^*\})$ 
2:  $s_i^{\text{init}} = \text{LinearNoBias}(s_i^{\text{inputs}})$ 
3:  $z_{ij}^{\text{init}} = \text{LinearNoBias}(s_i^{\text{inputs}}) + \text{LinearNoBias}(s_j^{\text{inputs}})$ 
4:  $z_{ij}^{\text{init}} += \text{RelativePositionEncoding}(\{f^*\})$ 
5:  $z_{ij}^{\text{init}} += \text{LinearNoBias}(f_{ij}^{\text{token\_bonds}})$ 
6:  $\{z_{ij}\}, \{\hat{s}_i\} = \mathbf{0}, \mathbf{0}$ 
7: for all  $c \in [1, \dots, N_{\text{cycle}}]$  do
8:    $z_{ij} = z_{ij}^{\text{init}} + \text{LinearNoBias}(\text{LayerNorm}(\hat{z}_{ij}))$ 
9:    $\{z_{ij}\} += \text{TemplateEmbedder}(\{f^*\}, \{z_{ij}\})$ 
10:   $\{z_{ij}\} += \text{MsaModule}(\{f_{Si}^{\text{msa}}\}, \{z_{ij}\}, \{s_i^{\text{inputs}}\})$ 
11:   $s_i = s_i^{\text{init}} + \text{LinearNoBias}(\text{LayerNorm}(\hat{s}_i))$ 
12:   $\{s_i\}, \{z_{ij}\} = \text{PairformerStack}(\{s_i\}, \{z_{ij}\})$ 
13:   $\{\hat{s}_i\}, \{\hat{z}_{ij}\} \leftarrow \{s_i\}, \{z_{ij}\}$ 
14: end for
15:  $\{\bar{x}_i^{\text{pred}}\} = \text{SampleDiffusion}(\{f^*\}, \{s_i^{\text{inputs}}\}, \{s_i\}, \{z_{ij}\})$ 
16:  $\{p_i^{\text{plddt}}\}, \{p_i^{\text{pae}}\}, \{p_i^{\text{pde}}\}, \{p_i^{\text{resolved}}\} = \text{ConfidenceHead}(\{s_i^{\text{inputs}}\}, \{s_i\}, \{z_{ij}\}, \{\bar{x}_i^{\text{pred}}\})$ 
17:  $p_{ij}^{\text{distogram}} = \text{DistogramHead}(z_{ij})$ 
18: return  $\{\bar{x}_i^{\text{pred}}\}, \{p_i^{\text{plddt}}\}, \{p_i^{\text{pae}}\}, \{p_i^{\text{pde}}\}, \{p_i^{\text{resolved}}\}, \{p_{ij}^{\text{distogram}}\}$ 

```

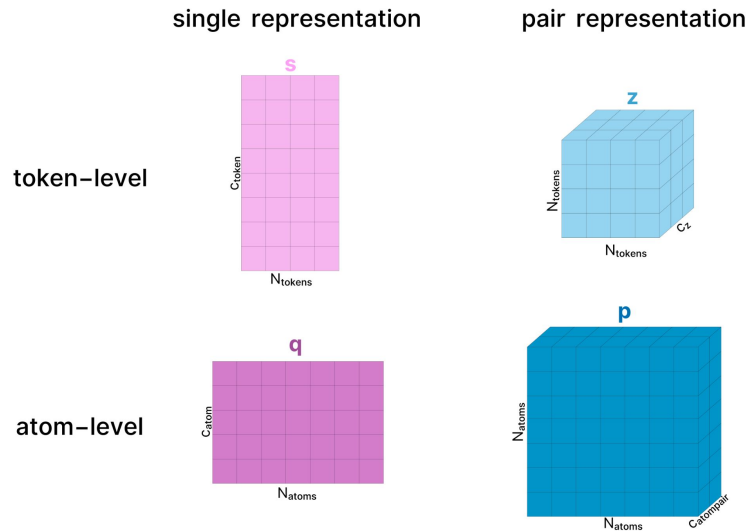
$$s_i^{\text{init}} \in \mathbb{R}^{c_s}$$

$$z_{ij}^{\text{init}} \in \mathbb{R}^{c_z}$$

$$z_{ij} \in \mathbb{R}^{c_z}$$

$$s_i \in \mathbb{R}^{c_s}$$

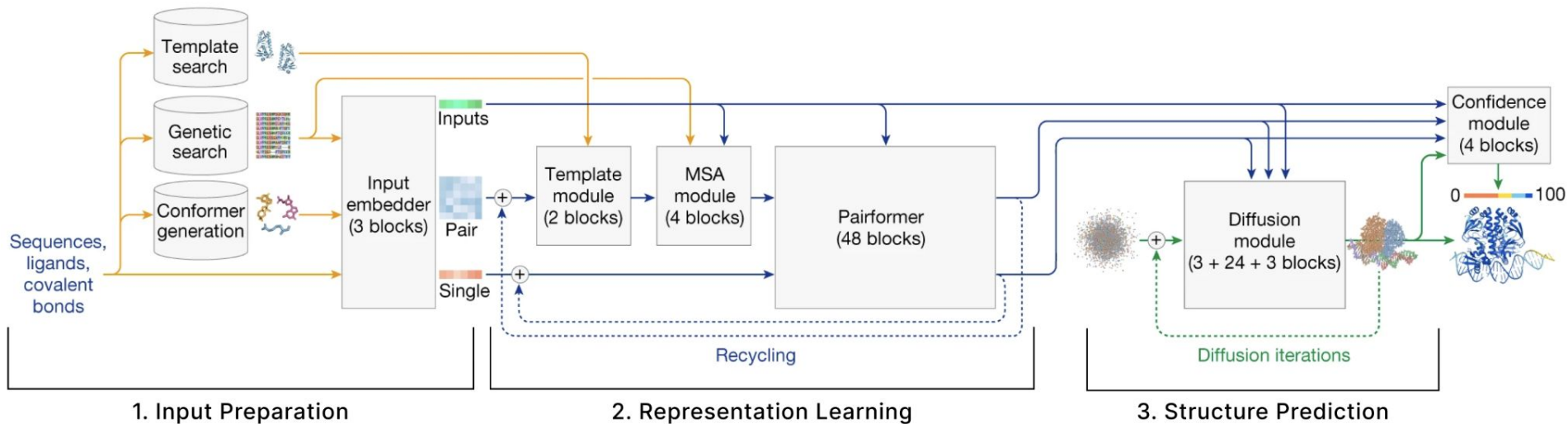
Notes on the variables and diagrams



AlphaFold 3 represents protein complexes using two key tensors: **Single** (per-token/atom features) and **Pair** (relationships between token/atom pairs). These can be at the token or atom level, shown with consistent names and colors.

Diagrams show how activation shapes change—not model weights. Tensor labels match those in the AF3 paper. Names are mostly preserved, but some (e.g., **c** to **q** in atom-level single) indicate updates through processing. LayerNorms are used throughout but mostly omitted for clarity.

1. Input Preparation



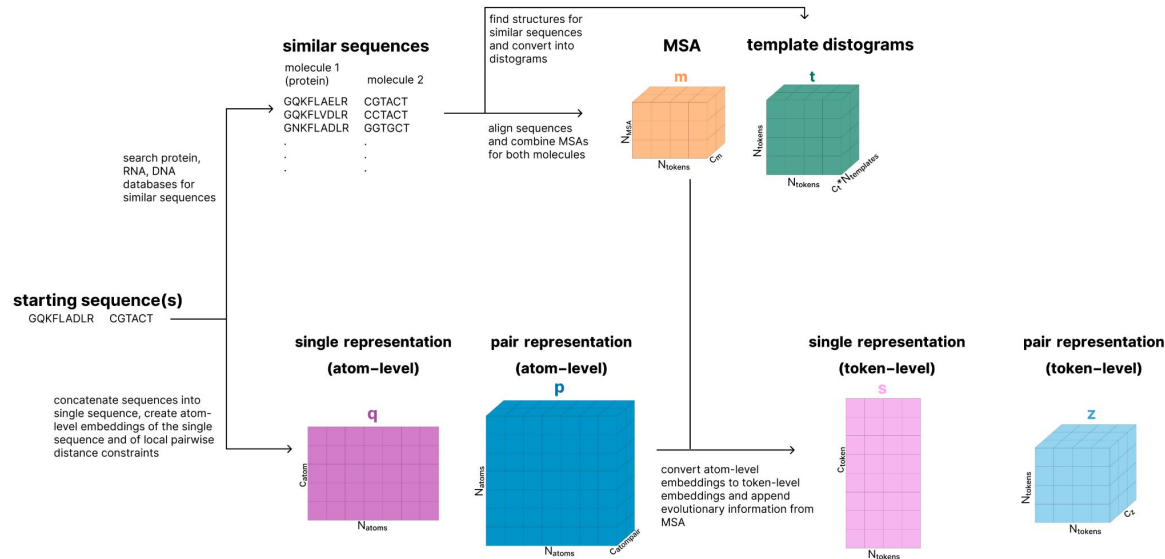
Input Features

Feature & Shape	Description
residue_index $[N_{\text{token}}]$	Residue number in the token's original input chain.
token_index $[N_{\text{token}}]$	Token number. Increases monotonically; does not restart at 1 for new chains.
asym_id $[N_{\text{token}}]$	Unique integer for each distinct chain.
entity_id $[N_{\text{token}}]$	Unique integer for each distinct sequence.
sym_id $[N_{\text{token}}]$	Unique integer within chains of this sequence. E.g. if chains A, B and C share a sequence but D does not, their sym_ids would be [0, 1, 2, 0].
restype $[N_{\text{token}}, 32]$	One-hot encoding of the sequence. 32 possible values: 20 amino acids + unknown, 4 RNA nucleotides + unknown, 4 DNA nucleotides + unknown, and gap. Ligands represented as “unknown amino acid”.
is_protein / rna / dna / ligand $[N_{\text{token}}]$	4 masks indicating the molecule type of a particular token.
ref_pos $[N_{\text{atom}}, 3]$	Atom positions in the reference conformer, with a random rotation and translation applied. Atom positions are given in Å.
ref_mask $[N_{\text{atom}}]$	Mask indicating which atom slots are used in the reference conformer.
ref_element $[N_{\text{atom}}, 128]$	One-hot encoding of the element atomic number for each atom in the reference conformer, up to atomic number 128.
ref_charge $[N_{\text{atom}}]$	Charge for each atom in the reference conformer.

Feature & Shape	Description
ref_atom_name_chars $[N_{\text{atom}}, 4, 64]$	One-hot encoding of the unique atom names in the reference conformer. Each character is encoded as $\text{ord}(c) - 32$, and names are padded to length 4.
ref_space_uid $[N_{\text{atom}}]$	Numerical encoding of the chain id and residue index associated with this reference conformer. Each (chain id, residue index) tuple is assigned an integer on first appearance.
msa $[N_{\text{msa}}, N_{\text{token}}, 32]$	One-hot encoding of the processed MSA, using the same classes as restype.
has_deletion $[N_{\text{msa}}, N_{\text{token}}]$	Binary feature indicating if there is a deletion to the left of each position in the MSA.
deletion_value $[N_{\text{msa}}, N_{\text{token}}]$	Raw deletion counts (the number of deletions to the left of each MSA position) are transformed to $[0, 1]$ using $\frac{2}{\pi} \arctan \frac{d}{3}$.
profile $[N_{\text{token}}, 32]$	Distribution across restypes in the main MSA. Computed before MSA processing (subsection 2.3).
deletion_mean $[N_{\text{token}}]$	Mean number of deletions at each position in the main MSA. Computed before MSA processing (subsection 2.3).
template_restype $[N_{\text{templ}}, N_{\text{token}}]$	One-hot encoding of the template sequence, see restype.
template_pseudo_beta_mask $[N_{\text{templ}}, N_{\text{token}}]$	Mask indicating if the C^β (C^α for glycine) has coordinates for the template at this residue.
template_backbone_frame_mask $[N_{\text{templ}}, N_{\text{token}}]$	Mask indicating if coordinates exist for all atoms required to compute the backbone frame (used in the template_unit_vector feature).
template_distogram $[N_{\text{templ}}, N_{\text{token}}, N_{\text{token}}, 39]$	A one-hot pairwise feature indicating the distance between C^β atoms (C^α for glycine). Pairwise distances are discretized into 38 bins of equal width between 3.25 Å and 50.75 Å; one more bin contains any larger distances.
template_unit_vector $[N_{\text{templ}}, N_{\text{token}}, N_{\text{token}}, 3]$	The unit vector of the displacement of the C^α atom of all residues within the local frame of each residue. Local frames are computed as in [1].
token_bonds $[N_{\text{token}}, N_{\text{token}}]$	A 2D matrix indicating if there is a bond between any atom in token i and token j , restricted to just polymer-ligand and ligand-ligand bonds and bonds less than 2.4 Å during training.

Input Preparation Pipeline

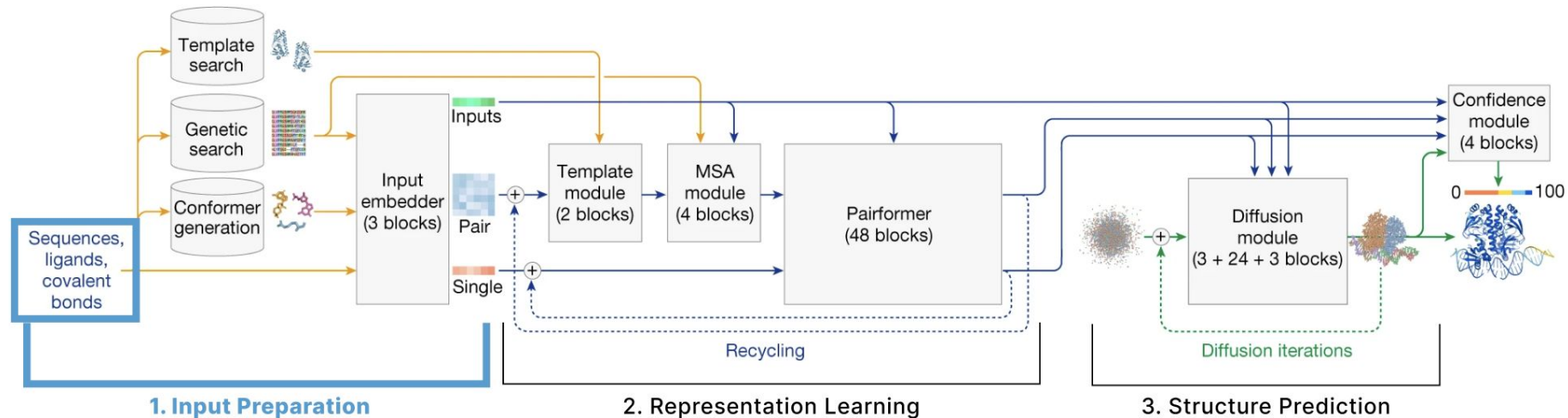
Overview of Input Preparation Pipeline



The user provides a protein sequence and optionally other molecules. This section turns inputs into 6 key tensors for the model: **s** (token-level single), **z** (token-level pair), **q** (atom-level single), **p** (atom-level pair), **m** (MSA), and **t** (template). Steps include:

- **Tokenization:** Molecules are split into tokens (e.g., amino acids, atoms); distinguishes atom-level vs. token-level representations.
- **Retrieval (MSA & Templates):** Uses external databases to find similar sequences (MSA, **m**) and structural templates (**t**) to guide prediction.
- **Create Atom-Level Representations:** Builds **q** and **p** from 3D conformers; encodes atom-level features and inter-atomic relationships.
- **Update Atom-Level Representations:** The Atom Transformer updates **q** using custom attention, LayerNorm, gating, and transition modules.
- **Aggregate Atom- to Token-Level:** Aggregates atom-level **q** and **p** into token-level **s** and **z**, incorporating MSA info and known ligand bonding data.

Tokenization



**Tokenize input
sequences**

Retrieve similar
sequences and
structures to create
MSA and templates

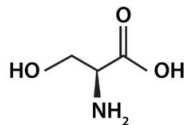
Create atom-level
representation of
sequences

Update atom-level
representation (Atom
Transformer)

Aggregate atom-level
representation to
token-level

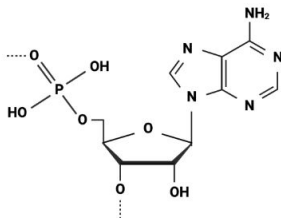
Tokenization

Standard Amino Acid
(serine)



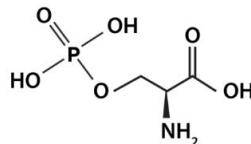
7 atoms*
1 tokens

Standard Nucleotide
(adenosine)



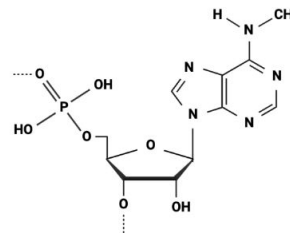
23 atoms*
1 token

Modified Amino Acid
(phosphoserine)



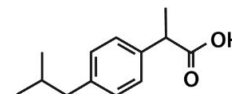
11 atoms*
11 tokens

Modified Nucleotide
(methylenadenosine)



24 atoms*
24 tokens

Ligand
(ibuprofen)



15 atoms*
15 tokens

*atoms=heavy atoms

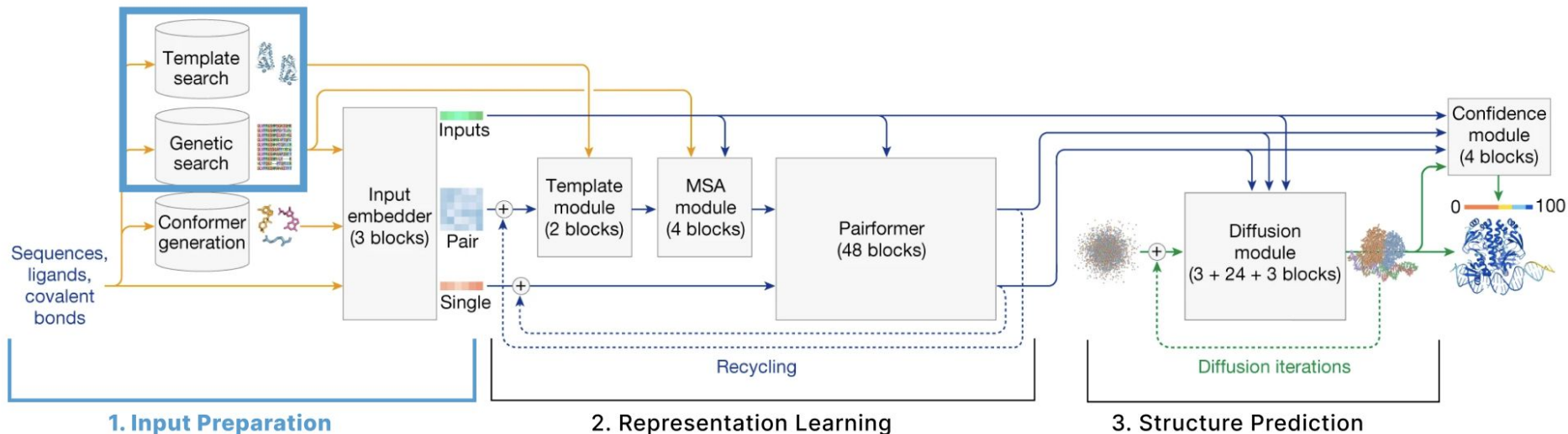
AF2 used one token per amino acid. AF3 extends this:

- **Standard amino acids & nucleotides:** 1 token each
- **Modified residues & other molecules:** 1 token per atom

Thus, tokens may represent multiple atoms (e.g., an amino acid) or a single atom (e.g., a ligand).

- 35 amino acids → 35 tokens (~600 atoms)
- 35-atom ligand → 35 tokens

Retrieval (Create MSA and Templates)



Tokenize input
sequences

**Retrieve similar
sequences and
structures to create
MSA and templates**

Create atom-level
representation of
sequences

Update atom-level
representation (Atom
Transformer)

Aggregate atom-level
representation to
token-level

Retrieval (Create MSA and Templates)

Retrieval as Input

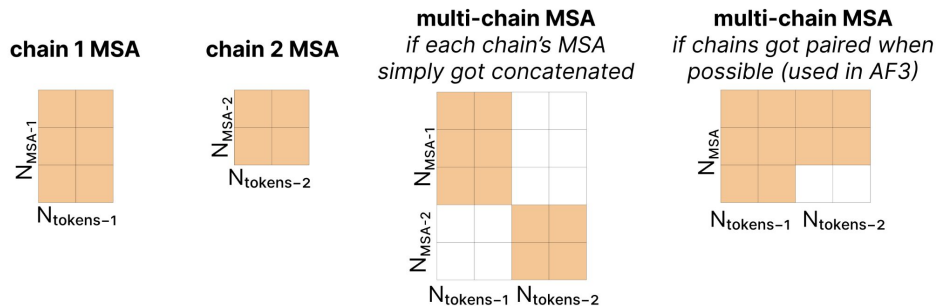
AF3 begins by retrieving similar protein and RNA sequences (MSA) and related structures (templates). Unlike AF-Multimer, RNA sequences are also retrieved.

Why Use MSAs and Templates?

MSAs reveal evolutionary patterns—conserved positions and co-evolving residues—helping infer structure. Known structures (templates) of similar proteins also guide prediction, following the principle of homology modeling.

How Retrieval Works

Protein and RNA chains are searched via HMM-based tools. Hits are aligned into MSAs, with chain pairing used to reduce sparsity. Up to 16384 sequences are included. Templates (up to 4 per chain) are selected from PDB using HMM alignment.

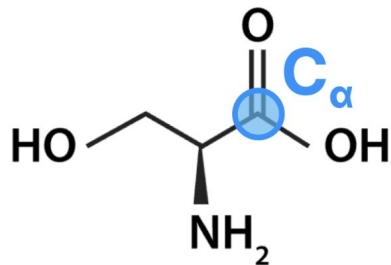


Retrieval (Create MSA and Templates)

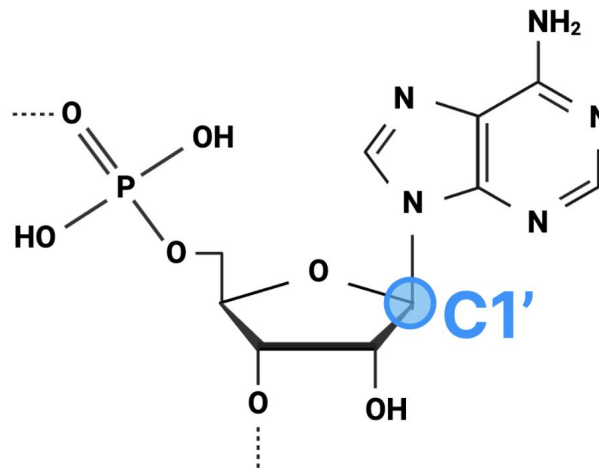
Template Representation

Each template's 3D structure is encoded as a distance matrix between token pairs. Distances are binned into histograms ("distograms"), with added metadata like chain ID and resolution. Only intra-chain distances are used, ignoring inter-chain info.

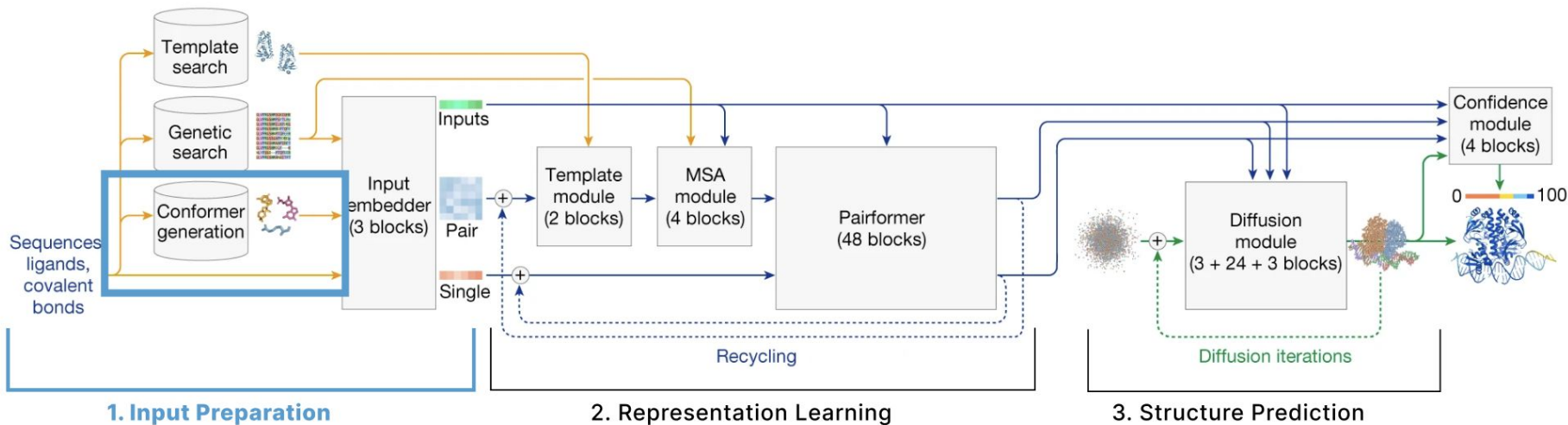
Standard Amino Acid



Standard Nucleotide



Create Atom-Level Representations



Tokenize input
sequences

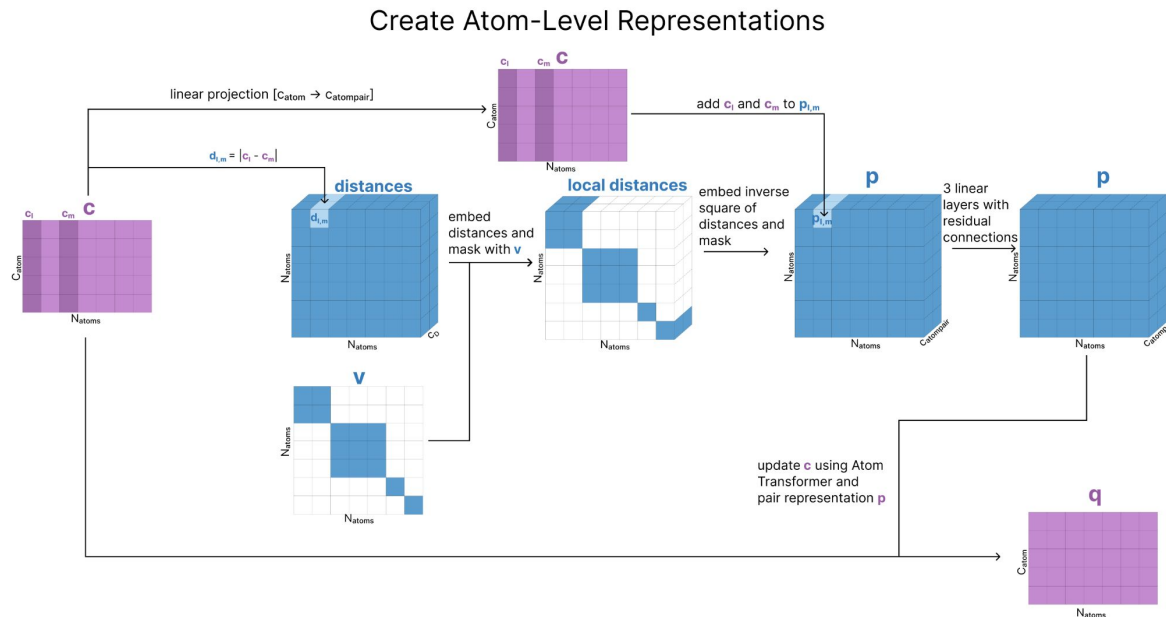
Retrieve similar
sequences and
structures to create
MSA and templates

Create atom-level
representation of
sequences

Update atom-level
representation (Atom
Transformer)

Aggregate atom-level
representation to
token-level

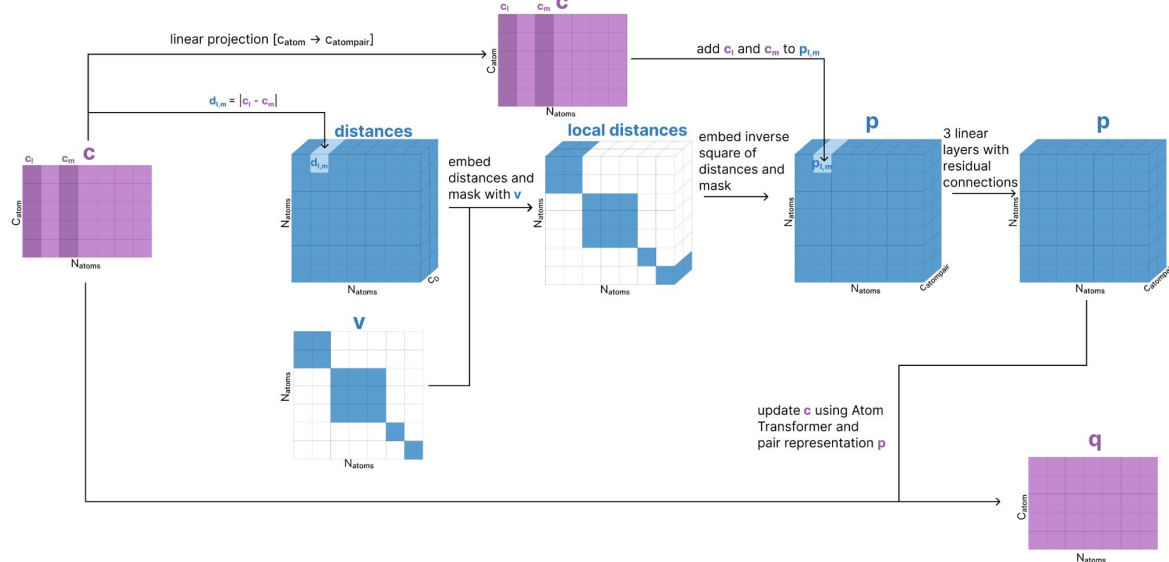
Create Atom-Level Representations



To create the atom-level representation $\mathbf{q}$, we start by generating a “reference conformer” for each amino acid, nucleotide, and ligand—a 3D structure based on known local arrangements. Amino acids use standard conformers, while ligands are generated via RDKit’s ETKDGV3 algorithm. We combine conformer coordinates with atomic features (charge, atomic number) into matrix $\mathbf{c}$, which initializes the atom-pair distance matrix $\mathbf{p}$. Masking ensures distances only reflect known intra-token values. After processing through embeddings and layers, we copy $\mathbf{c}$ to $\mathbf{q}$, which is updated throughout the model.

Create Atom-Level Representations

Create Atom-Level Representations



Algorithm 5 Atom attention encoder

```

def AtomAttentionEncoder( $\{f^*\}, \{r_i\}, \{s_i^{trunk}\}, \{x_{ij}\}, C_{atom}, C_{atompair}, C_{token}$ ):
    # Create the atom single conditioning: Embed per-atom meta data
    1:  $c_i = LinearNoBias(concat(f_i^{ref\_pos}, f_i^{ref\_charge}, f_i^{ref\_mask}, f_i^{ref\_element}, f_i^{ref\_atom\_name\_chars}))$   $c_i \in \mathbb{R}^{C_{atom}}$ 
    2:  $i \in \{1, \dots, N_{atoms}\}$ 

    # Embed offsets between atom reference positions
    3:  $\tilde{d}_{lm} = f_{lm}^{ref\_pos} - f_m^{ref\_pos}$   $\tilde{d}_{lm} \in \mathbb{R}^3$ 
    4:  $v_{lm} = (f_{lm}^{ref\_space\_sid} - f_m^{ref\_space\_sid})$   $v_{lm} \in \mathbb{R}$ 
    5:  $p_{lm} = LinearNoBias(\tilde{d}_{lm}) \cdot v_{lm}$   $p_{lm} \in \mathbb{R}^{C_{atompair}}$ 
    6:  $p_{lm} += LinearNoBias(v_{lm}) \cdot v_{lm}$ 

    # Embed pairwise inverse squared distances, and the valid mask.
    7:  $p_{lm} += LinearNoBias(1 / (1 + \|\tilde{d}_{lm}\|^2)) \cdot v_{lm}$ 
    8:  $p_{lm} += LinearNoBias(v_{lm}) \cdot v_{lm}$ 

    # Initialise the atom single representation as the single conditioning.
    9:  $q_i = c_i$   $q_i \in \mathbb{R}^{C_{atom}}$ 

    # If provided, add trunk embeddings and noisy positions.
    10: if  $\{r_i\} \neq \emptyset$  then
        11: Broadcast the single and pair embedding from the trunk.
        12:  $c_i += LinearNoBias(LayerNorm(s_{tok\_idx(i)}^{trunk}))$ 
        13:  $p_{lm} += LinearNoBias(LayerNorm(x_{tok\_idx(i), tok\_idx(m)}))$ 
        14: Add the noisy positions.
        15:  $q_i += LinearNoBias(r_i)$ 
    16: end if

    # Add the combined single conditioning to the pair representation.
    17:  $p_{lm} += LinearNoBias(reLU(c_i)) + LinearNoBias(reLU(c_m))$ 

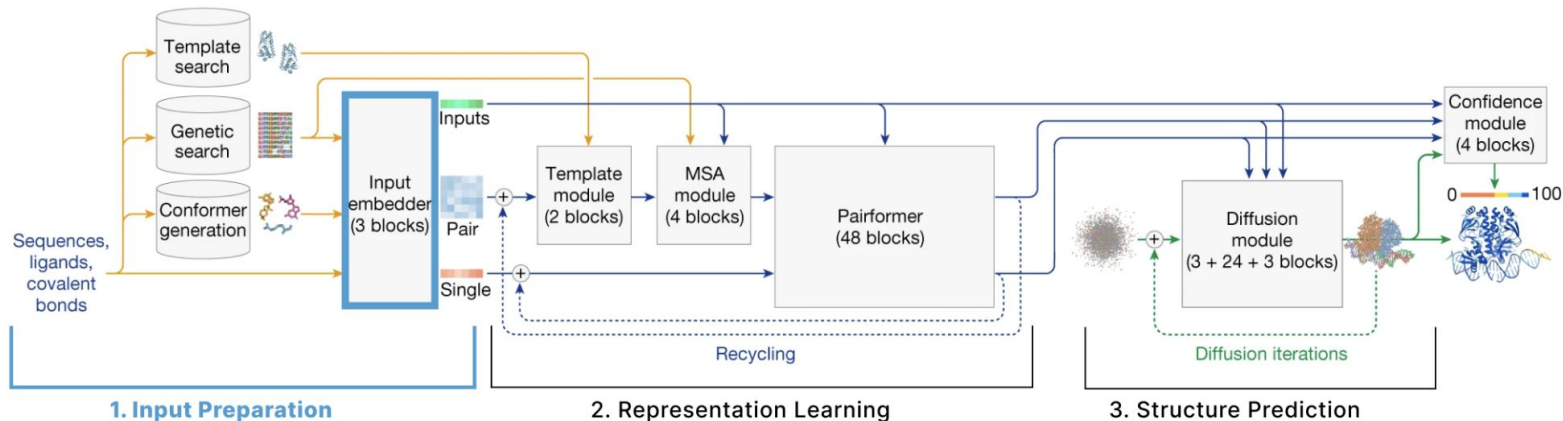
    # Run a small MLP on the pair activations.
    18:  $p_{lm} += LinearNoBias(reLU(LinearNoBias(reLU(LinearNoBias(reLU(p_{lm}))))))$ 

    # Cross attention transformer.
    19:  $\{q_i\} = AtomTransformer(\{q_i\}, \{c_i\}, \{p_{lm}\}, N_{block} = 3, N_{head} = 4)$ 

    # Aggregate per-atom representation to per-token representation
    20:  $a_i = \frac{1}{|I|} \sum_{l \in I, l=1 \dots N_{atoms}} (reLU(LinearNoBias(q_l)))$   $a_i \in \mathbb{R}^{C_{token}}$ 
    21:  $q_i^{skip}, c_i^{skip}, p_{lm}^{skip} = q_i, c_i, p_{lm}$ 
    22: return  $\{a_i\}, \{q_i^{skip}\}, \{c_i^{skip}\}, \{p_{lm}^{skip}\}$ 
    
```

To create the atom-level representation q , we start by generating a “reference conformer” for each amino acid, nucleotide, and ligand—a 3D structure based on known local arrangements. Amino acids use standard conformers, while ligands are generated via RDKit’s ETKDGV3 algorithm. We combine conformer coordinates with atomic features (charge, atomic number) into matrix c , which initializes the atom-pair distance matrix p . Masking ensures distances only reflect known intra-token values. After processing through embeddings and layers, we copy c to q , which is updated throughout the model.

Update Atom-Level Representations (Atom Transformer)



Tokenize input
sequences

Retrieve similar
sequences and
structures to create
MSA and templates

Create atom-level
representation of
sequences

**Update atom-level
representation
(Atom Transformer)**

Aggregate atom-level
representation to
token-level

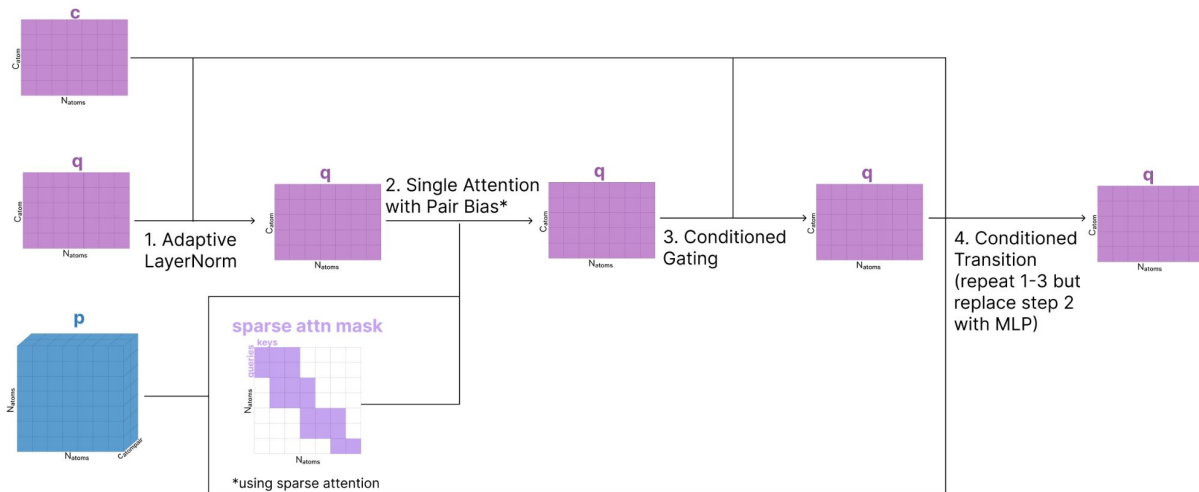
AtomTransformer introduces building blocks used elsewhere

- * Adaptive LayerNorm
- * Attention with Pair Bias
- * Sequence-local atom attention

- * Conditioned Gating
- * Conditioned Transition blocks

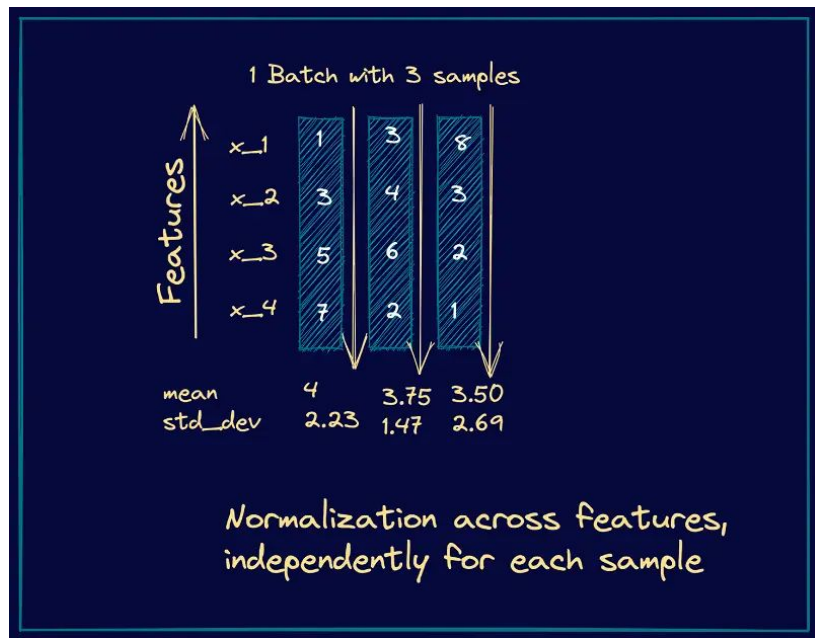
Create Atom-Level Representations

Overview of Atom Transformer

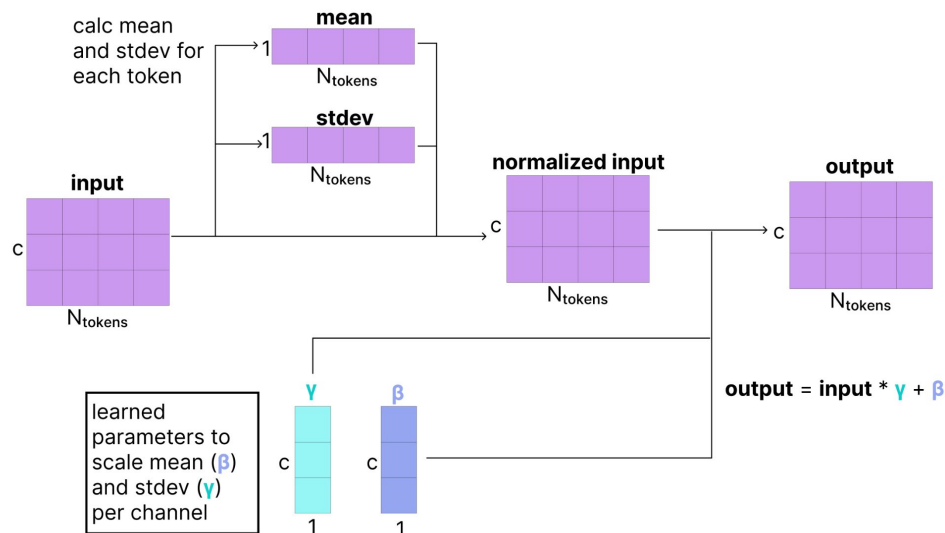


AF3 updates atom-level representations $\mathbf{q}$ and pair representations $\mathbf{p}$ using the Atom Transformer, which integrates original features $\mathbf{c}$ as a residual input.

Atom Transformer - 1. Adaptive LayerNorm (AdaNorm)



Standard Layer Norm



Instead of fixed scaling (gamma) and bias (beta), AdaNorm adaptively predicts these parameters from c to normalize q .

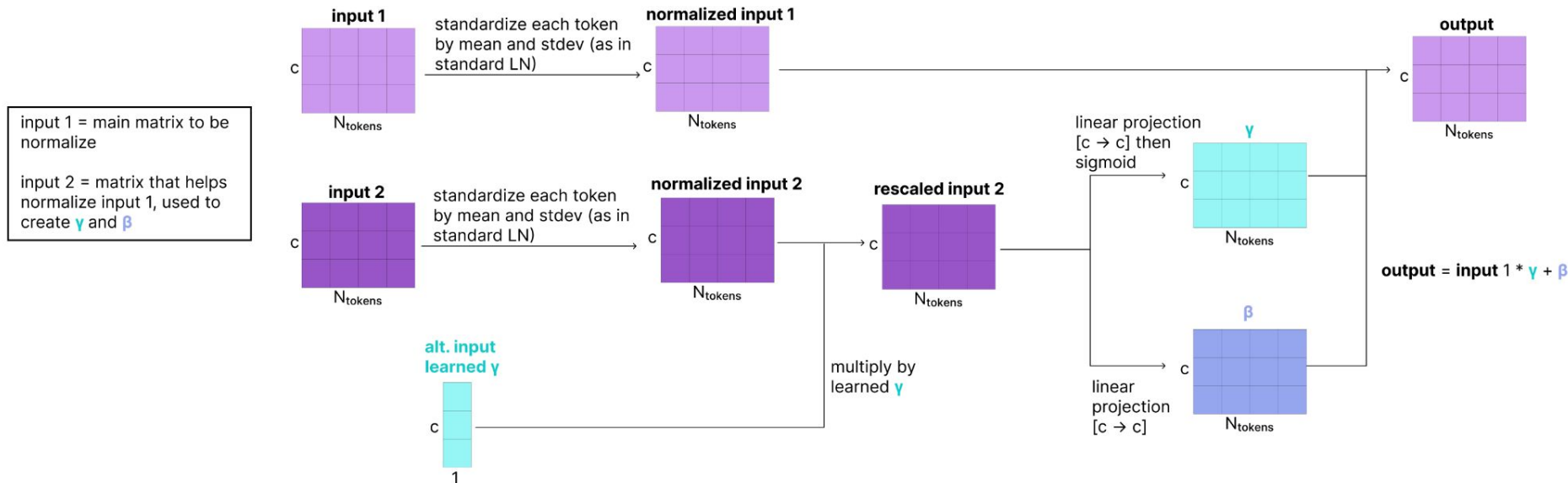
Atom Transformer - 1. Adaptive LayerNorm (AdaNorm)

Algorithm 26 Adaptive LayerNorm

def AdaLN($\mathbf{a}, \mathbf{s}$) :

- 1: $\mathbf{a} \leftarrow \text{LayerNorm}(\mathbf{a}, \text{scale}=\text{False}, \text{offset}=\text{False})$
- 2: $\mathbf{s} \leftarrow \text{LayerNorm}(\mathbf{s}, \text{offset}=\text{False})$
- 3: $\mathbf{a} \leftarrow \text{sigmoid}(\text{Linear}(\mathbf{s})) \odot \mathbf{a} + \text{LinearNoBias}(\mathbf{s})$
- 4: **return** $\mathbf{a}$

Adaptive Layer Norm



Instead of fixed scaling (gamma) and bias (beta), AdaNorm adaptively predicts these parameters from $\mathbf{c}$ to normalize $\mathbf{q}$.

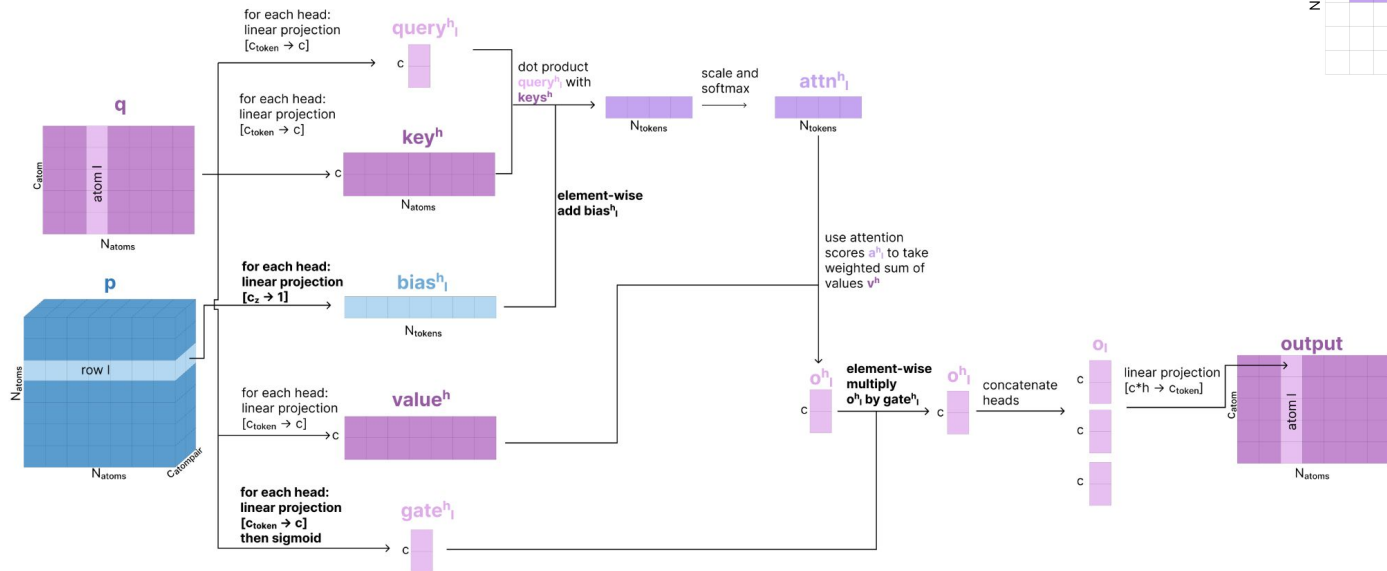
Atom Transformer - 2. Attention with Pair Bias & Gating

Sequence-local atom attention

Self-attention run within rectangular blocks where 32 query atoms can attend to 128 nearby key atoms

Attention with Pair Bias (Atom-Level)

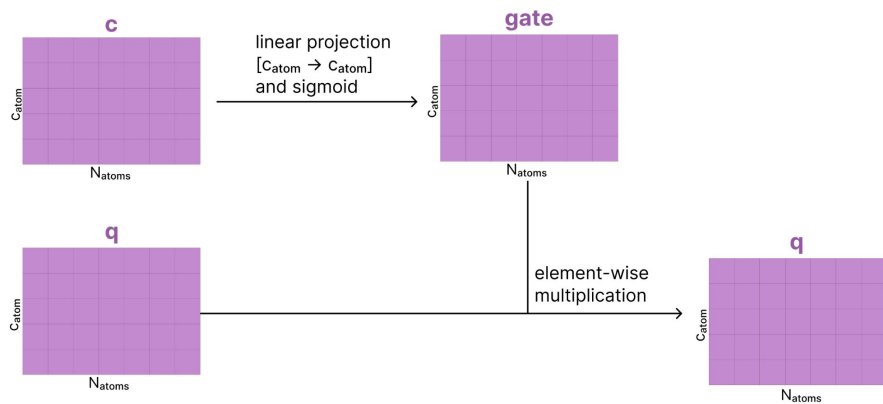
demonstrating the process for updating one atom at a time (atom l)
operations not part of standard multi-head self-attention are **bolded**



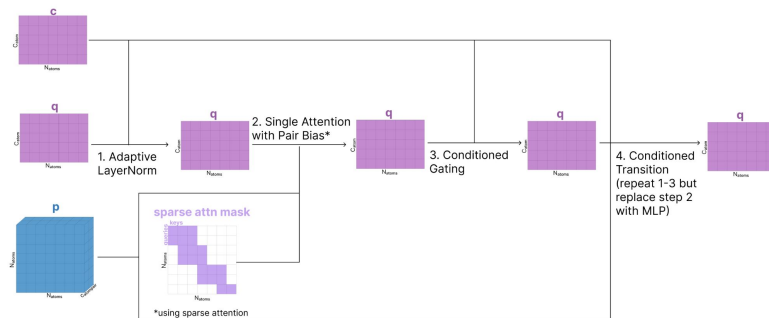
- Attention uses queries, keys, and values from q with added pairwise biases from p .
- A gating sigmoid filters attention output, controlling information flow like LSTM gates.
- Sparse local attention attends over groups of 32 atoms at a time can all attend to 128 other atoms for efficiency.

Atom Transformer - 3. Conditioned Gating

Gate **q** based on **c**



Overview of Atom Transformer

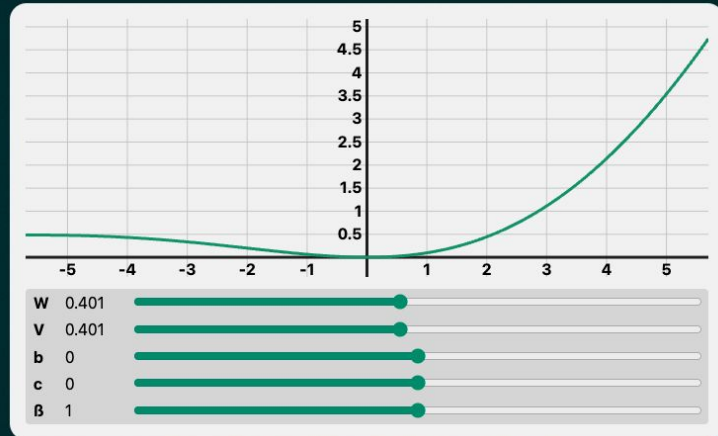


A second gating step is applied, conditioned on **c**, to selectively modulate the representation.

Atom Transformer - 4. Conditioned Transition (MLP with SwiGLU)

SwiGLU

As you could guess, SwiGLU is just a *portmanteau* of Swish and GLU, and as such, it's simply a GLU that is activated using the Swish function instead of a sigmoid, so that $\text{SwiGLU}(x) = (Wx+b)*\text{Swish}(Vx+c)$.

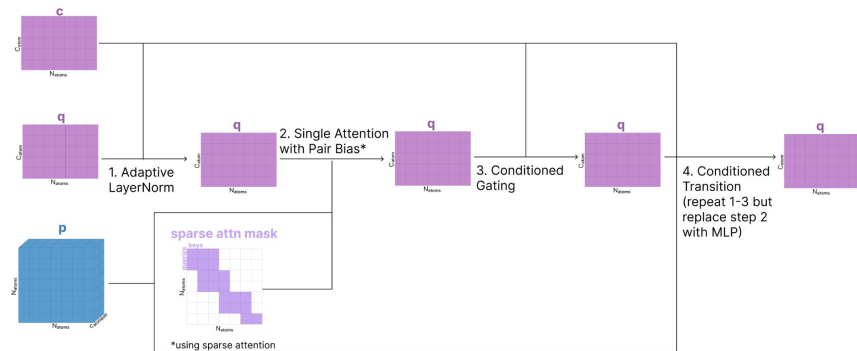


Try setting $W = V = \sqrt{2} \approx 1.4$, and $\beta = 0$.

Why does it work? This is the explanation at the [SwiGLU paper itself](#):

We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

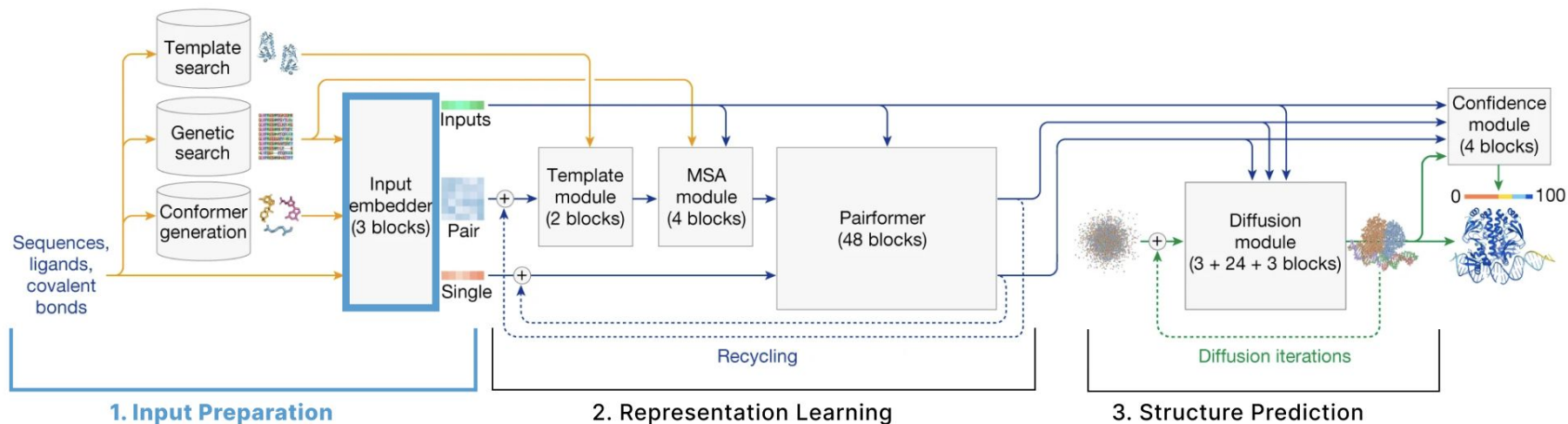
Overview of Atom Transformer



MLP layers are “conditioned” by AdaNorm and gating based on c . AF3 replaces ReLU with SwiGLU for improved non-linearity.

<https://jcarlosroldan.com/post/348/what-is-swiglu>

Aggregate Atom-Level → Token-Level



Tokenize input
sequences

Retrieve similar
sequences and
structures to create
MSA and templates

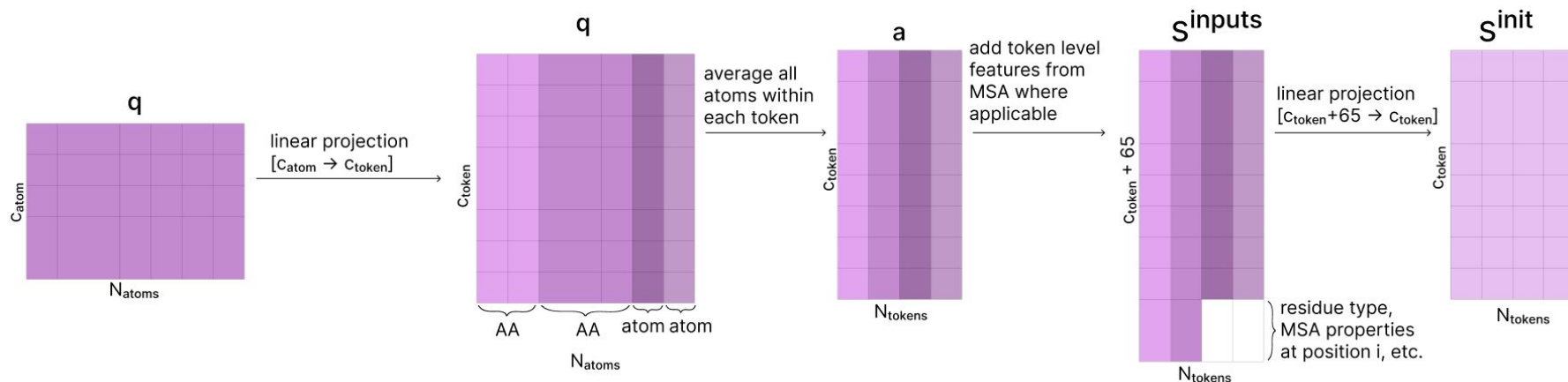
Create atom-level
representation of
sequences

Update atom-level
representation (Atom
Transformer)

**Aggregate atom-
level representation
to token-level**

Aggregate Atom-Level → Token-Level

Create Token-Level Single Sequence Representation

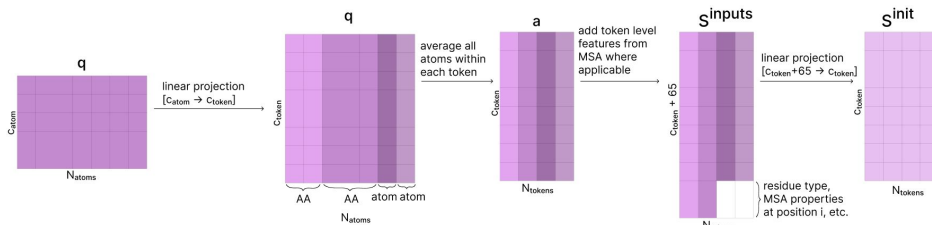


AF3 shifts from atom-level to token-level processing by projecting atom features (q) to higher dimensions, then averaging over atoms belonging to each standard amino acid or nucleotide. These become token-level inputs.

MSA-derived features are concatenated, forming **s_{inputs}** , which is projected to **s_{init}** , the initialized token representation.

Aggregate Atom-Level → Token-Level

Create Token-Level Single Sequence Representation



Cross attention transformer.

15: $\{\mathbf{q}_l\} = \text{AtomTransformer}(\{\mathbf{q}_l\}, \{\mathbf{c}_l\}, \{\mathbf{p}_{lm}\}, N_{\text{block}} = 3, N_{\text{head}} = 4)$

Aggregate per-atom representation to per-token representation

16: $\mathbf{a}_i = \text{mean}_{\substack{l \in \{1, \dots, N_{atoms}\} \\ \text{tok_idx}(l) = i}} (\text{relu}(\text{LinearNoBias}(\mathbf{q}_l)))$

17: $\mathbf{q}_l^{\text{skip}}, \mathbf{c}_l^{\text{skip}}, \mathbf{p}_{lm}^{\text{skip}} = \mathbf{q}_l, \mathbf{c}_l, \mathbf{p}_{lm}$

18: **return** $\{\mathbf{a}_i\}, \{\mathbf{q}_l^{\text{skip}}\}, \{\mathbf{c}_l^{\text{skip}}\}, \{\mathbf{p}_{lm}^{\text{skip}}\}$

def InputFeatureEmbedder($\{\mathbf{f}^*\}$):

Embed per-atom features.

1: $\{\mathbf{a}_i\}, _ _ _ = \text{AtomAttentionEncoder}(\{\mathbf{f}^*\}, \emptyset, \emptyset, \emptyset, c_{atom} = 128, c_{atompair} = 16, c_{token} = 384)$

Concatenate the per-token features.

2: $\mathbf{s}_i = \text{concat}(\mathbf{a}_i, \mathbf{f}_i^{\text{restype}}, \mathbf{f}_i^{\text{profile}}, \mathbf{f}_i^{\text{deletion_mean}})$

3: **return** $\{\mathbf{s}_i\}$

def MainInferenceLoop($\{\mathbf{f}^*\}, N_{\text{cycle}} = 4, c_s = 384, c_z = 128$):

1: $\{\mathbf{s}_i^{\text{inputs}}\} = \text{InputFeatureEmbedder}(\{\mathbf{f}^*\})$

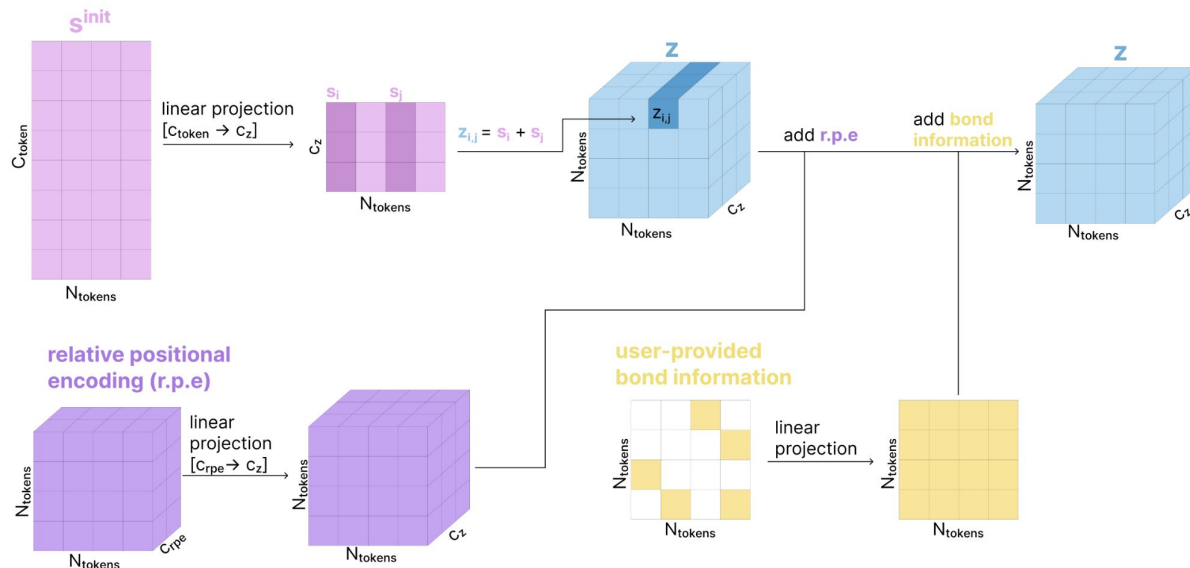
2: $\mathbf{s}_i^{\text{init}} = \text{LinearNoBias}(\mathbf{s}_i^{\text{inputs}})$

AF3 shifts from atom-level to token-level processing by projecting atom features ($\mathbf{q}$) to higher dimensions, then averaging over atoms belonging to each standard amino acid or nucleotide. These become token-level inputs.

MSA-derived features are concatenated, forming $\mathbf{s}_{inputs}$, which is projected to $\mathbf{s}_{init}$, the initialized token representation.

Aggregate Atom-Level → Token-Level

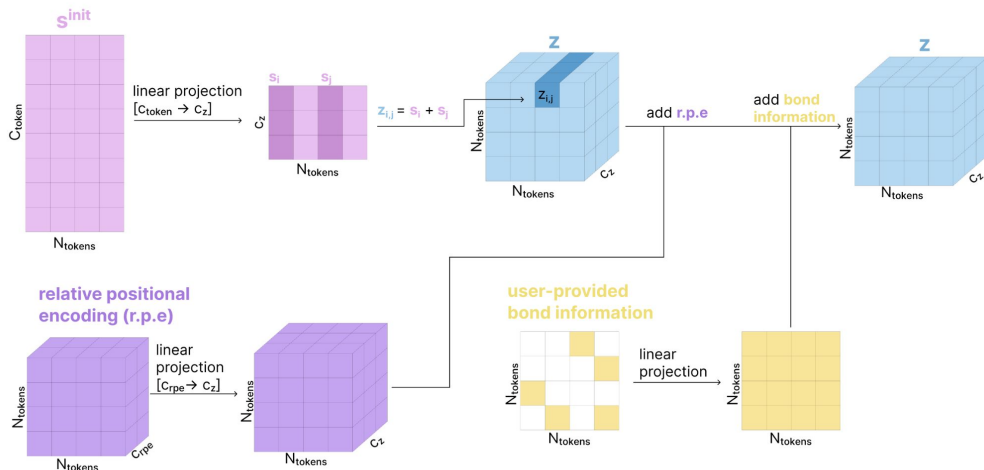
Create Token-Level Pair Representation



Token pair representations $\mathbf{z}_{\text{init}}$ (shape: tokens $\times$ tokens $\times$ 128) are initialized by summing projected token features ($\mathbf{s}_i + \mathbf{s}_j$) with relative positional encodings and optional bond info.

Aggregate Atom-Level → Token-Level

Create Token-Level Pair Representation



def MainInferenceLoop($\{\mathbf{f}^*\}$, $N_{\text{cycle}} = 4$, $c_s = 384$, $c_z = 128$) :

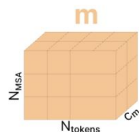
- 1: $\{\mathbf{s}_i^{\text{inputs}}\} = \text{InputFeatureEmbedder}(\{\mathbf{f}^*\})$
- 2: $\mathbf{s}_i^{\text{init}} = \text{LinearNoBias}(\mathbf{s}_i^{\text{inputs}})$
- 3: $\mathbf{z}_{ij}^{\text{init}} = \text{LinearNoBias}(\mathbf{s}_i^{\text{inputs}}) + \text{LinearNoBias}(\mathbf{s}_j^{\text{inputs}})$
- 4: $\mathbf{z}_{ij}^{\text{init}} += \text{RelativePositionEncoding}(\{\mathbf{f}^*\})$
- 5: $\mathbf{z}_{ij}^{\text{init}} += \text{LinearNoBias}(\mathbf{f}_{ij}^{\text{token_bonds}})$

Token pair representations $\mathbf{z}_{\text{init}}$ (shape: tokens $\times$ tokens $\times$ 128) are initialized by summing projected token features ($\mathbf{s}_i + \mathbf{s}_j$) with relative positional encodings and optional bond info.

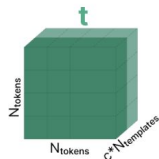
Aggregate Atom-Level → Token-Level

Information about related sequences and their structures

Multiple Sequence Alignment

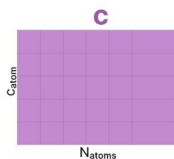


Structure Templates

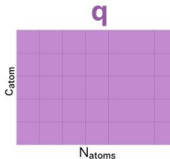


Information about all the atoms ("single")

Original Atom-Level Single Representation

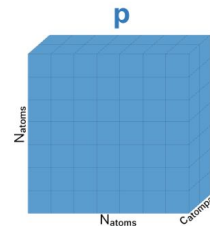


Updated Atom-Level Single Representation



Information about all the pairs of atoms ("pair")

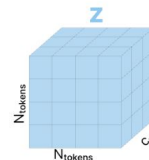
Atom-Level Pair Representation



Token-Level Single Representation

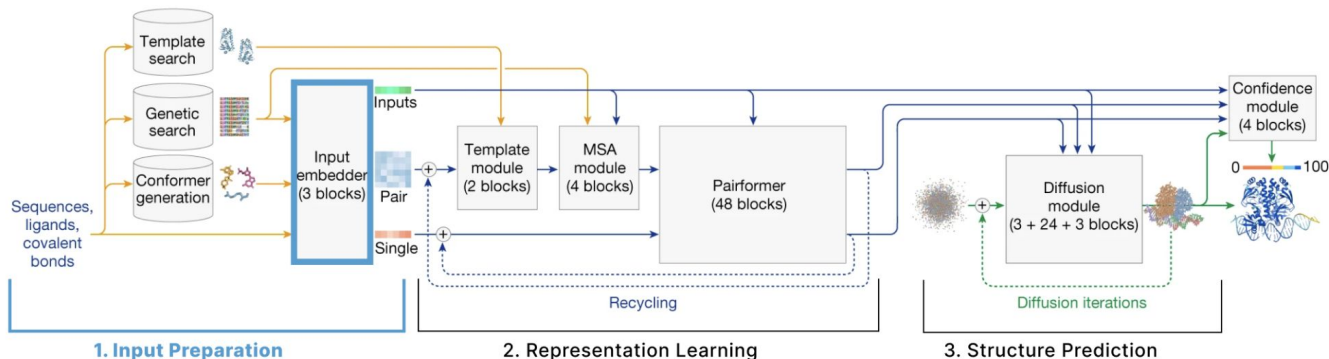


Token-Level Pair Representation



For Step 2, we will set aside the atom-level representations (**c**, **q**, **p**) and focus on updating our token-level representations **s** and **z** in the next section (with the help of **m** and **t**).

1. Input Preparation Recap



Tokenize input sequences

Retrieve similar sequences and structures to create MSA and templates

Create atom-level representation of sequences

Update atom-level representation (Atom Transformer)

Aggregate atom-level representation to token-level

AtomTransformer introduces building blocks used elsewhere

* Adaptive LayerNorm

* Attention with Pair Bias

* Sequence-local atom attention

* Conditioned Gating

* Conditioned Transition blocks

End of Part 1

Thanks for listening!

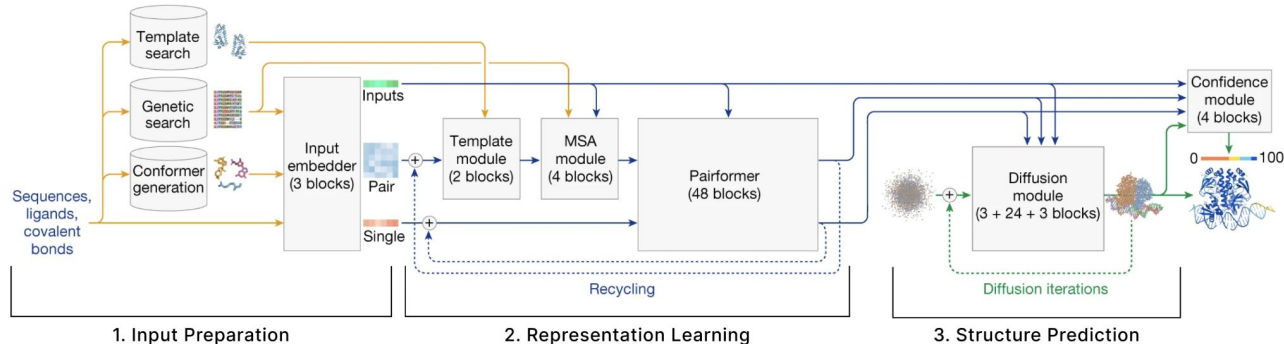
Questions - Comments?

Algorithm 1 Main Inference Loop

```

def MainInferenceLoop( $\{f^*\}$ ,  $N_{\text{cycle}} = 4$ ,  $c_s = 384$ ,  $c_z = 128$ ) :
1:  $\{s_i^{\text{inputs}}\} = \text{InputFeatureEmbedder}(\{f^*\})$ 
2:  $s_i^{\text{init}} = \text{LinearNoBias}(s_i^{\text{inputs}})$   $s_i^{\text{init}} \in \mathbb{R}^{c_s}$ 
3:  $z_{ij}^{\text{init}} = \text{LinearNoBias}(s_i^{\text{inputs}}) + \text{LinearNoBias}(s_j^{\text{inputs}})$   $z_{ij}^{\text{init}} \in \mathbb{R}^{c_z}$ 
4:  $z_{ij}^{\text{init}} += \text{RelativePositionEncoding}(\{f^*\})$ 
5:  $z_{ij}^{\text{init}} += \text{LinearNoBias}(r_{ij}^{\text{token\_bonds}})$ 
6:  $\{\hat{z}_{ij}\}, \{\hat{s}_i\} = \mathbf{0}, \mathbf{0}$ 
7: for all  $c \in [1, \dots, N_{\text{cycle}}]$  do
8:    $z_{ij} = z_{ij}^{\text{init}} + \text{LinearNoBias}(\text{LayerNorm}(\hat{z}_{ij}))$   $z_{ij} \in \mathbb{R}^{c_z}$ 
9:    $\{z_{ij}\} += \text{TemplateEmbedder}(\{f^*\}, \{z_{ij}\})$ 
10:   $\{z_{ij}\} += \text{MsaModule}(\{f_{s_i}^{\text{msa}}\}, \{z_{ij}\}, \{s_i^{\text{inputs}}\})$ 
11:   $s_i = s_i^{\text{init}} + \text{LinearNoBias}(\text{LayerNorm}(\hat{s}_i))$   $s_i \in \mathbb{R}^{c_s}$ 
12:   $\{s_i\}, \{z_{ij}\} = \text{PairformerStack}(\{s_i\}, \{z_{ij}\})$ 
13:   $\{\hat{s}_i\}, \{\hat{z}_{ij}\} \leftarrow \{s_i\}, \{z_{ij}\}$ 
14: end for
15:  $\{\bar{x}_i^{\text{pred}}\} = \text{SampleDiffusion}(\{f^*\}, \{s_i^{\text{inputs}}\}, \{s_i\}, \{z_{ij}\})$ 
16:  $\{p_i^{\text{pddt}}\}, \{p_i^{\text{pae}}\}, \{p_{ij}^{\text{pde}}\}, \{p_i^{\text{resolved}}\} = \text{ConfidenceHead}(\{s_i^{\text{inputs}}\}, \{s_i\}, \{z_{ij}\}, \{\bar{x}_i^{\text{pred}}\})$ 
17:  $p_{ij}^{\text{distogram}} = \text{DistogramHead}(z_{ij})$ 
18: return  $\{\bar{x}_i^{\text{pred}}\}, \{p_i^{\text{pddt}}\}, \{p_i^{\text{pae}}\}, \{p_{ij}^{\text{pde}}\}, \{p_i^{\text{resolved}}\}, \{p_{ij}^{\text{distogram}}\}$ 

```



Accurate structure prediction of biomolecular interactions with AlphaFold 3 (Part 2)

— Abramson, J., Adler, J., Dunger, J. et al. —

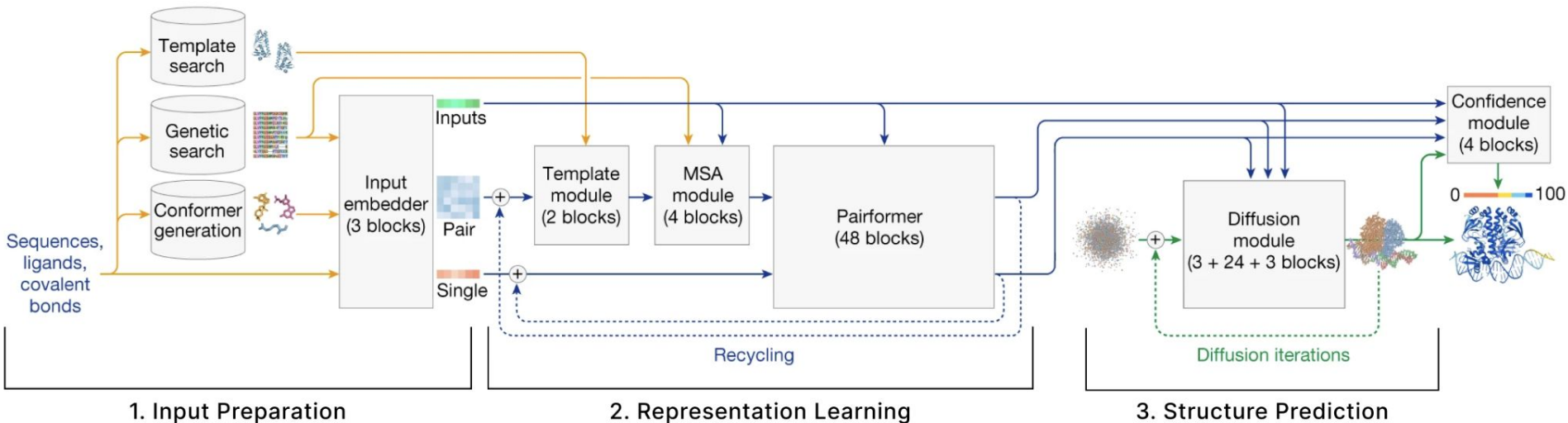
<https://www.nature.com/articles/s41586-024-07487-w>

Previously on Part 1

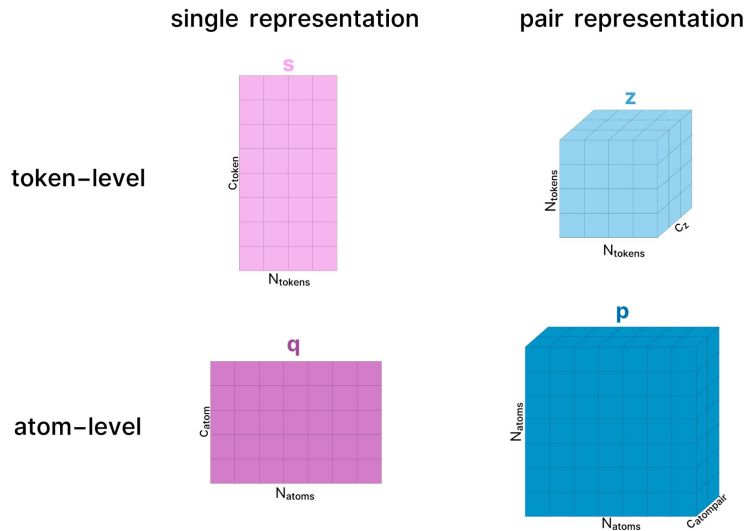
From AlphaFold2 to AlphaFold3 – A Paradigm Shift

Feature	AlphaFold2 (2021)	AlphaFold-Multimer (2021)	AlphaFold3 (2024)
Input types	Protein sequences	Protein sequences	Proteins, RNA, DNA, ligands
Complex prediction	No	Yes (proteins only)	Yes (all molecule types)
Ligand/RNA/DNA modeling	✗	✗	✓
Underlying architecture	Evoformer + structure module	Extended Evoformer	Pairformer + Diffusion
Training task	Supervised (PDB structures)	Supervised (PDB structures)	Conditional diffusion

Architecture Overview



Notes on the variables and diagrams



AlphaFold 3 represents protein complexes using two key tensors: **Single** (per-token/atom features) and **Pair** (relationships between token/atom pairs). These can be at the token or atom level, shown with consistent names and colors.

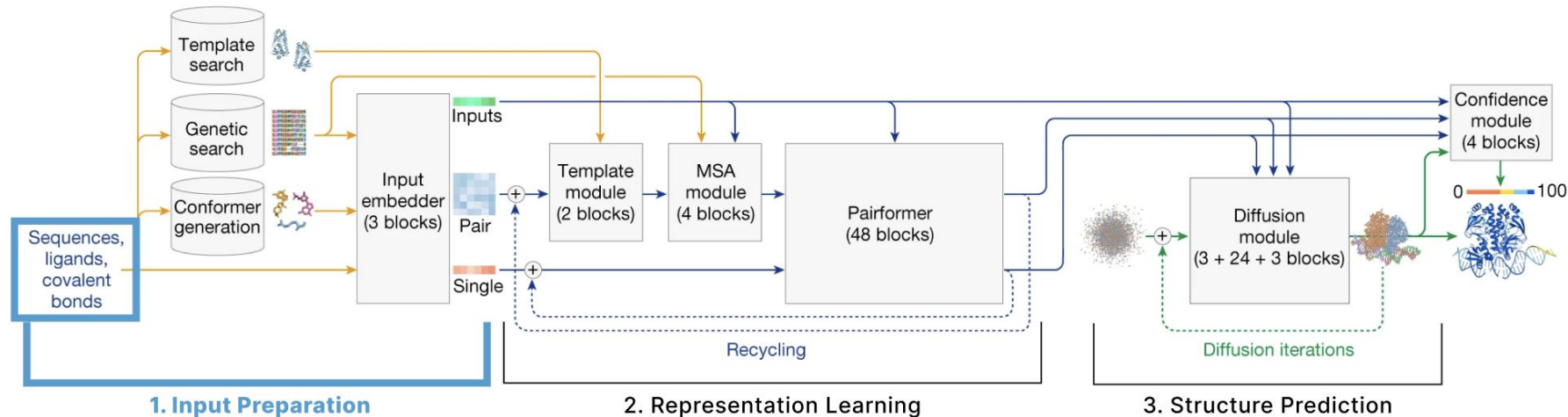
Diagrams show how activation shapes change—not model weights. Tensor labels match those in the AF3 paper. Names are mostly preserved, but some (e.g., **c** to **q** in atom-level single) indicate updates through processing. LayerNorms are used throughout but mostly omitted for clarity.

Input Features

Feature & Shape	Description
residue_index $[N_{\text{token}}]$	Residue number in the token's original input chain.
token_index $[N_{\text{token}}]$	Token number. Increases monotonically; does not restart at 1 for new chains.
asym_id $[N_{\text{token}}]$	Unique integer for each distinct chain.
entity_id $[N_{\text{token}}]$	Unique integer for each distinct sequence.
sym_id $[N_{\text{token}}]$	Unique integer within chains of this sequence. E.g. if chains A, B and C share a sequence but D does not, their sym_ids would be [0, 1, 2, 0].
restype $[N_{\text{token}}, 32]$	One-hot encoding of the sequence. 32 possible values: 20 amino acids + unknown, 4 RNA nucleotides + unknown, 4 DNA nucleotides + unknown, and gap. Ligands represented as “unknown amino acid”.
is_protein / rna / dna / ligand $[N_{\text{token}}]$	4 masks indicating the molecule type of a particular token.
ref_pos $[N_{\text{atom}}, 3]$	Atom positions in the reference conformer, with a random rotation and translation applied. Atom positions are given in Å.
ref_mask $[N_{\text{atom}}]$	Mask indicating which atom slots are used in the reference conformer.
ref_element $[N_{\text{atom}}, 128]$	One-hot encoding of the element atomic number for each atom in the reference conformer, up to atomic number 128.
ref_charge $[N_{\text{atom}}]$	Charge for each atom in the reference conformer.

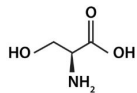
Feature & Shape	Description
ref_atom_name_chars $[N_{\text{atom}}, 4, 64]$	One-hot encoding of the unique atom names in the reference conformer. Each character is encoded as $\text{ord}(c) - 32$, and names are padded to length 4.
ref_space_uid $[N_{\text{atom}}]$	Numerical encoding of the chain id and residue index associated with this reference conformer. Each (chain id, residue index) tuple is assigned an integer on first appearance.
msa $[N_{\text{msa}}, N_{\text{token}}, 32]$	One-hot encoding of the processed MSA, using the same classes as restype.
has_deletion $[N_{\text{msa}}, N_{\text{token}}]$	Binary feature indicating if there is a deletion to the left of each position in the MSA.
deletion_value $[N_{\text{msa}}, N_{\text{token}}]$	Raw deletion counts (the number of deletions to the left of each MSA position) are transformed to $[0, 1]$ using $\frac{2}{\pi} \arctan \frac{d}{3}$.
profile $[N_{\text{token}}, 32]$	Distribution across restypes in the main MSA. Computed before MSA processing (subsection 2.3).
deletion_mean $[N_{\text{token}}]$	Mean number of deletions at each position in the main MSA. Computed before MSA processing (subsection 2.3).
template_restype $[N_{\text{templ}}, N_{\text{token}}]$	One-hot encoding of the template sequence, see restype.
template_pseudo_beta_mask $[N_{\text{templ}}, N_{\text{token}}]$	Mask indicating if the C^β (C^α for glycine) has coordinates for the template at this residue.
template_backbone_frame_mask $[N_{\text{templ}}, N_{\text{token}}]$	Mask indicating if coordinates exist for all atoms required to compute the backbone frame (used in the template_unit_vector feature).
template_distogram $[N_{\text{templ}}, N_{\text{token}}, N_{\text{token}}, 39]$	A one-hot pairwise feature indicating the distance between C^β atoms (C^α for glycine). Pairwise distances are discretized into 38 bins of equal width between 3.25 Å and 50.75 Å; one more bin contains any larger distances.
template_unit_vector $[N_{\text{templ}}, N_{\text{token}}, N_{\text{token}}, 3]$	The unit vector of the displacement of the C^α atom of all residues within the local frame of each residue. Local frames are computed as in [1].
token_bonds $[N_{\text{token}}, N_{\text{token}}]$	A 2D matrix indicating if there is a bond between any atom in token i and token j , restricted to just polymer-ligand and ligand-ligand bonds and bonds less than 2.4 Å during training.

Tokenization



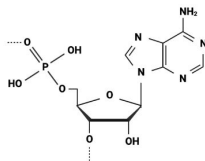
Tokenize input sequences

Standard Amino Acid
(serine)



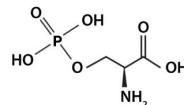
7 atoms*
1 tokens

Standard Nucleotide
(adenosine)



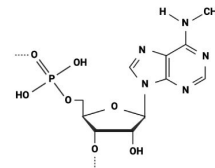
23 atoms*
1 token

Modified Amino Acid
(phosphoserine)



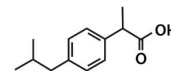
11 atoms*
11 tokens

Modified Nucleotide
(methyladenosine)



24 atoms*
24 tokens

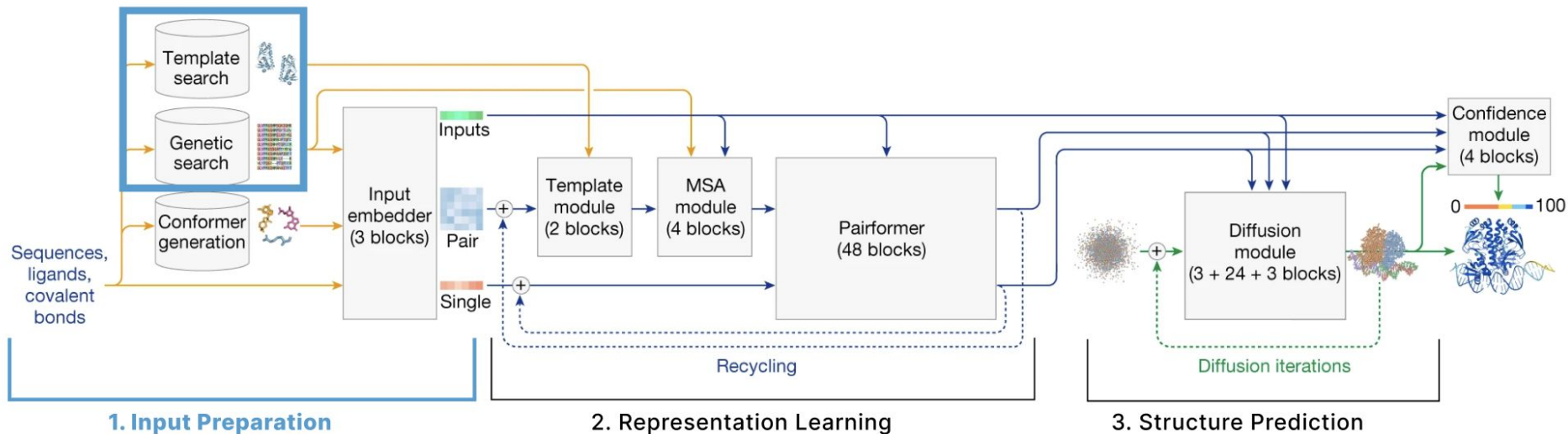
Ligand
(ibuprofen)



15 atoms*
15 tokens

*atoms=heavy atoms

Retrieval (Create MSA and Templates)



Tokenize input
sequences

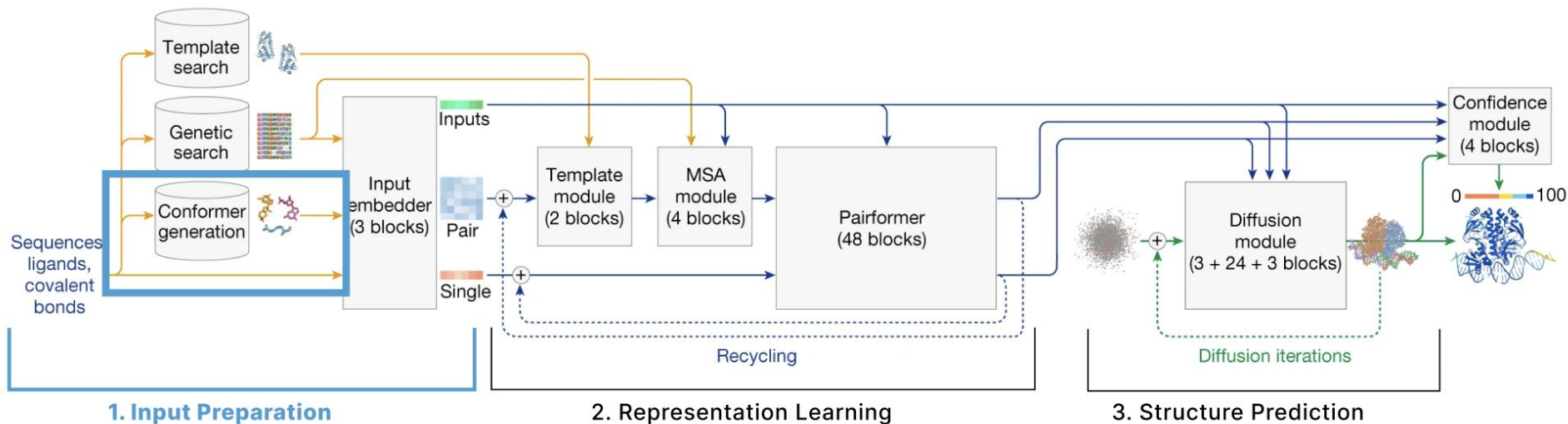
**Retrieve similar
sequences and
structures to create
MSA and templates**

Create atom-level
representation of
sequences

Update atom-level
representation (Atom
Transformer)

Aggregate atom-level
representation to
token-level

Create Atom-Level Representations



Tokenize input
sequences

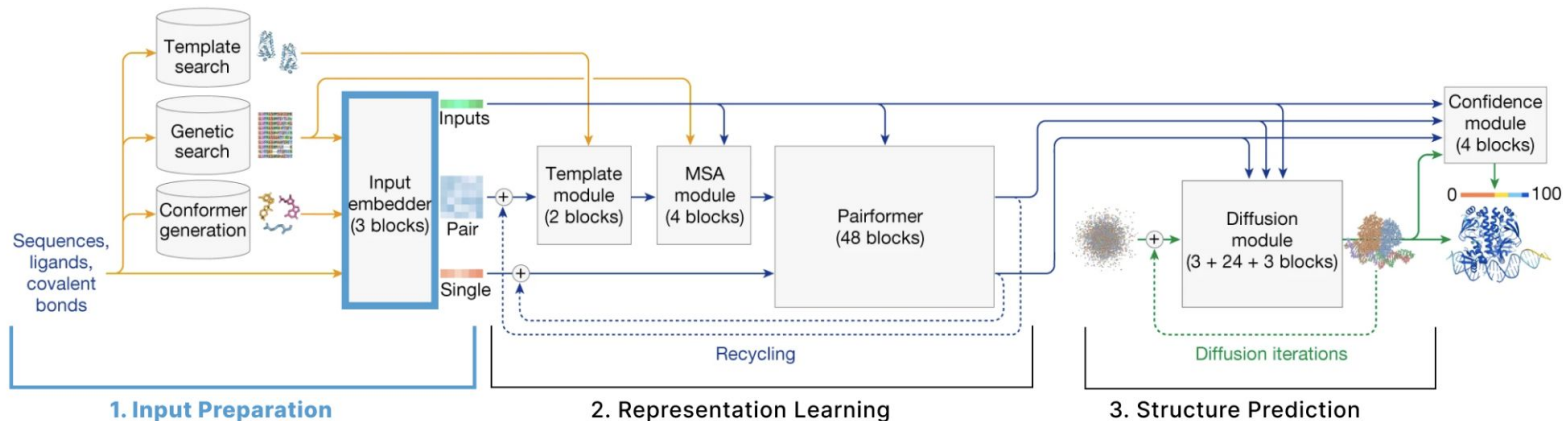
Retrieve similar
sequences and
structures to create
MSA and templates

Create atom-level
representation of
sequences

Update atom-level
representation (Atom
Transformer)

Aggregate atom-level
representation to
token-level

Update Atom-Level Representations (Atom Transformer)



Tokenize input
sequences

Retrieve similar
sequences and
structures to create
MSA and templates

Create atom-level
representation of
sequences

**Update atom-level
representation
(Atom Transformer)**

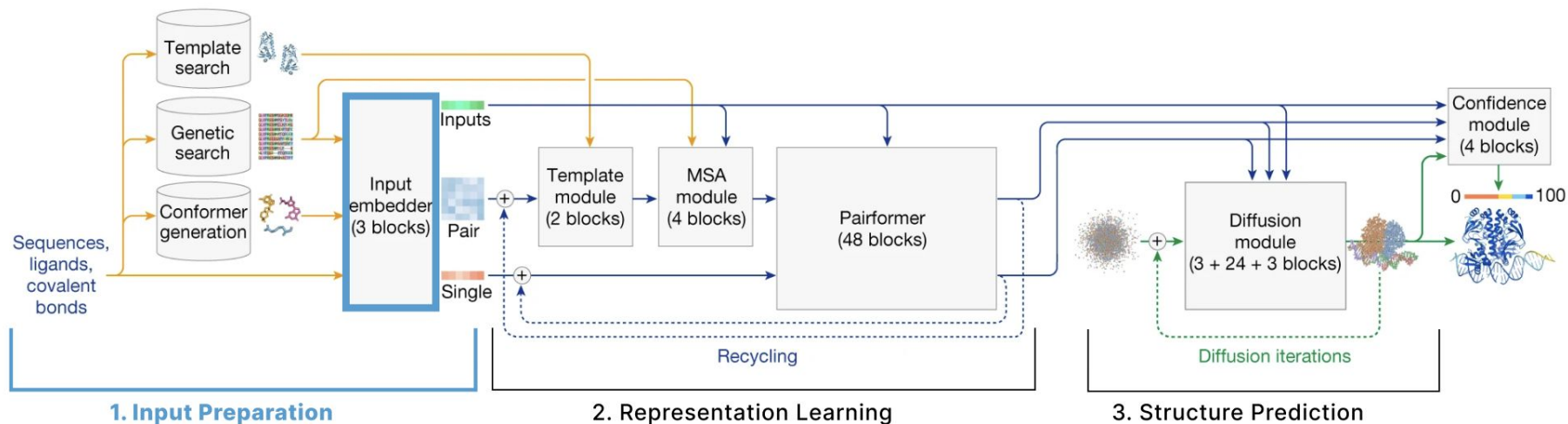
Aggregate atom-level
representation to
token-level

AtomTransformer introduces building blocks used elsewhere

- * Adaptive LayerNorm
- * Attention with Pair Bias
- * Sequence-local atom attention

- * Conditioned Gating
- * Conditioned Transition blocks

Aggregate Atom-Level → Token-Level



Tokenize input
sequences

Retrieve similar
sequences and
structures to create
MSA and templates

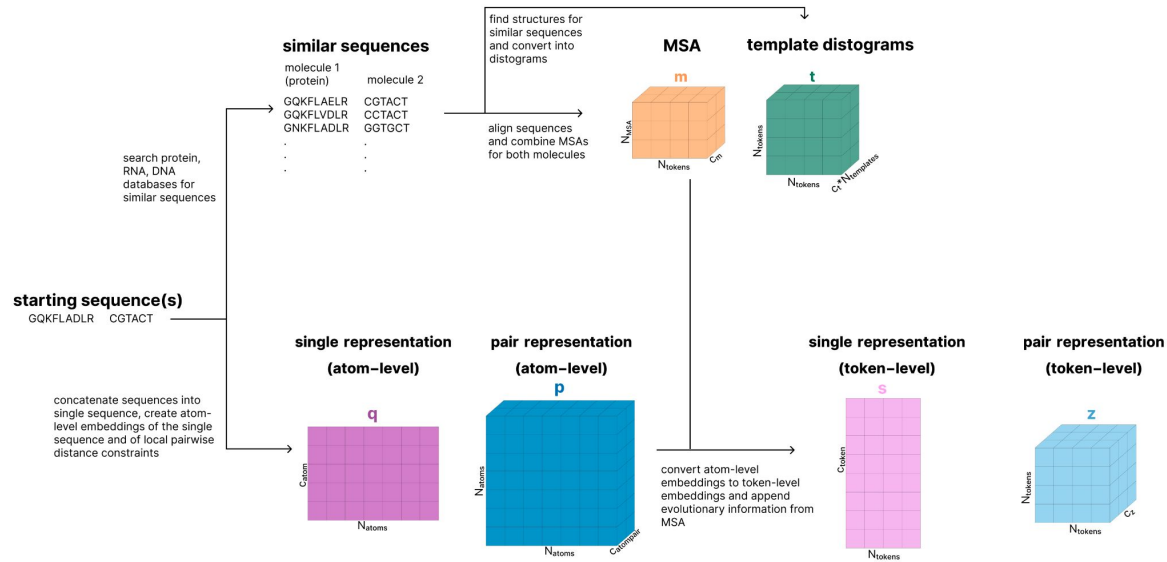
Create atom-level
representation of
sequences

Update atom-level
representation (Atom
Transformer)

**Aggregate atom-
level representation
to token-level**

Input Preparation Pipeline

Overview of Input Preparation Pipeline



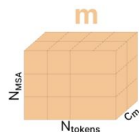
The user provides a protein sequence and optionally other molecules. This section turns inputs into 6 key tensors for the model: **s** (token-level single), **z** (token-level pair), **q** (atom-level single), **p** (atom-level pair), **m** (MSA), and **t** (template). Steps include:

- **Tokenization:** Molecules are split into tokens (e.g., amino acids, atoms); distinguishes atom-level vs. token-level representations.
- **Retrieval (MSA & Templates):** Uses external databases to find similar sequences (MSA, **m**) and structural templates (**t**) to guide prediction.
- **Create Atom-Level Representations:** Builds **q** and **p** from 3D conformers; encodes atom-level features and inter-atomic relationships.
- **Update Atom-Level Representations:** The Atom Transformer updates **q** using custom attention, LayerNorm, gating, and transition modules.
- **Aggregate Atom- to Token-Level:** Aggregates atom-level **q** and **p** into token-level **s** and **z**, incorporating MSA info and known ligand bonding data.

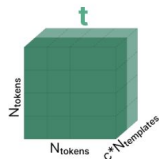
Aggregate Atom-Level → Token-Level

Information about related sequences and their structures

Multiple Sequence Alignment

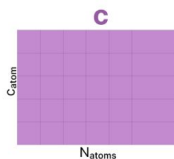


Structure Templates

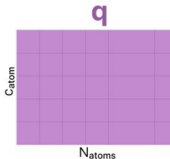


Information about all the atoms ("single")

Original Atom-Level Single Representation

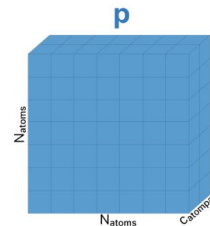


Updated Atom-Level Single Representation



Information about all the pairs of atoms ("pair")

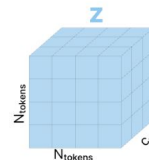
Atom-Level Pair Representation



Token-Level Single Representation



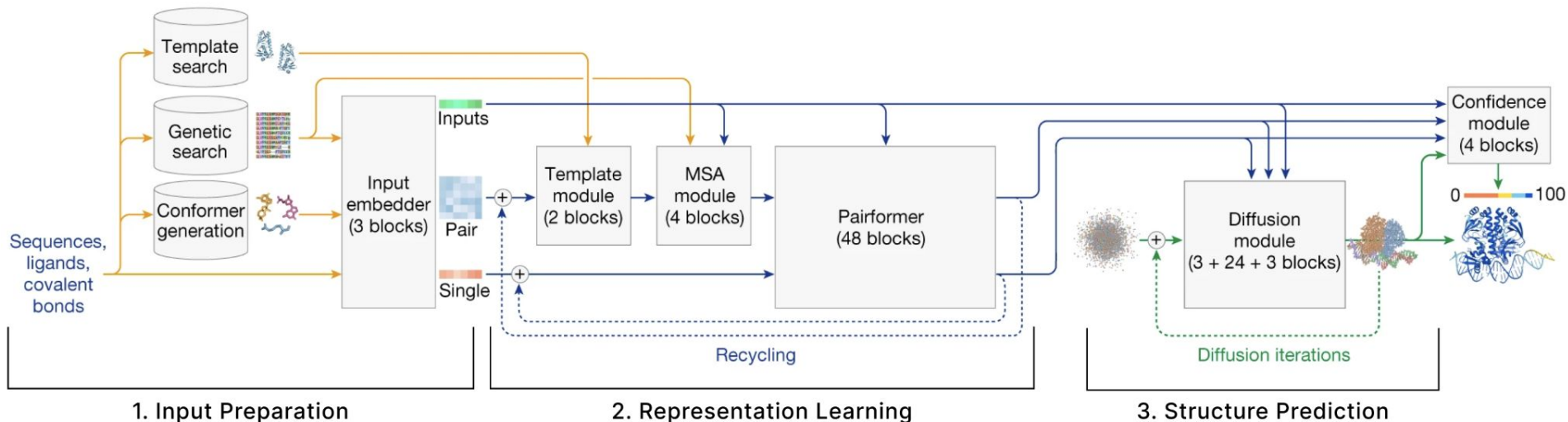
Token-Level Pair Representation



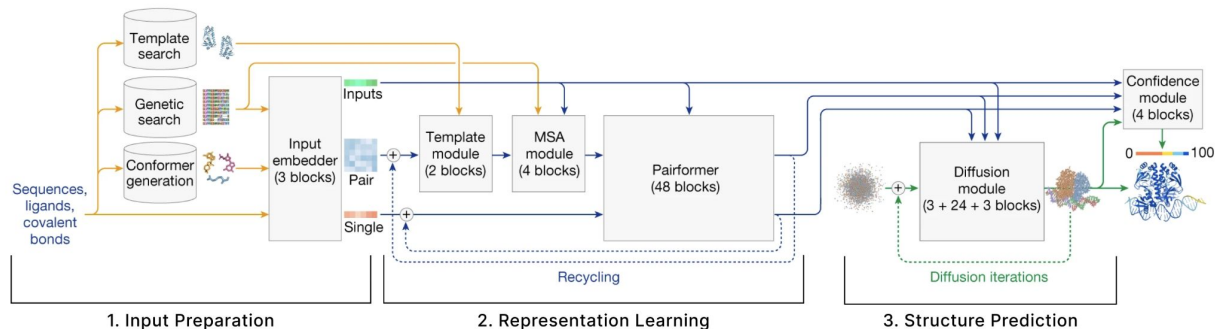
For Step 2, we will set aside the atom-level representations (**c**, **q**, **p**) and focus on updating our token-level representations **s** and **z** in the next section (with the help of **m** and **t**).

Let's Start Part 2

2. Representation Learning



Representation Learning ("Trunk") Overview

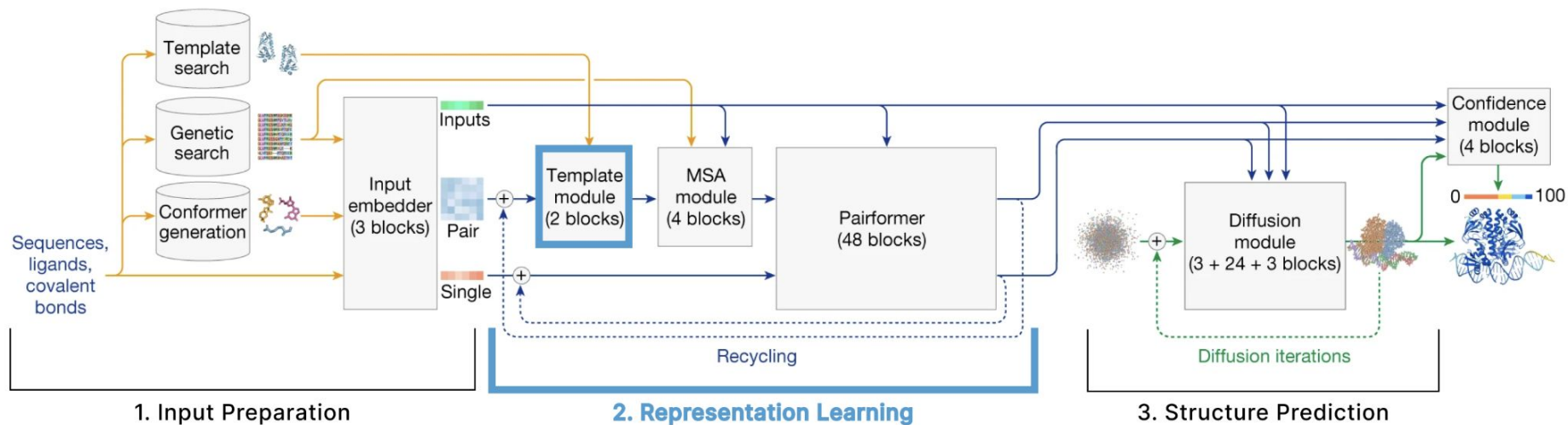


This is the main computational block, where token-level single (**s**) and pair (**z**) representations are iteratively refined. It includes:

- **Template Module:** Updates **z** using distance and identity features from structure templates (**t**).
- **MSA Module:** First updates the MSA representation (**m**), then incorporates it into **z**:
 - **Outer Product Mean:** Captures co-evolution by projecting interactions in **m** into **z**.
 - **MSA Row-wise Gated Self-Attention w/ Pair Bias:** Updates **m** using information from **z**, applying attention gated by **z** to reduce complexity.
- **Pairformer:** Refines **s** and **z** using geometric reasoning based on triangle relationships:
 - **Triangle Updates & Triangle Attention:** Two attention mechanisms updating **z**, inspired by the triangle inequality and structural motifs.
 - **Why Triangles?:** Models indirect interactions ($i \rightarrow j \rightarrow k$) crucial in 3D structures.
 - **Single Attention with Pair Bias:** Updates **s** using information in **z**, letting each token attend more to structurally important partners.

These modules are stacked and **recycled** multiple times, allowing the model to iteratively improve its internal structural representations.

Template Module

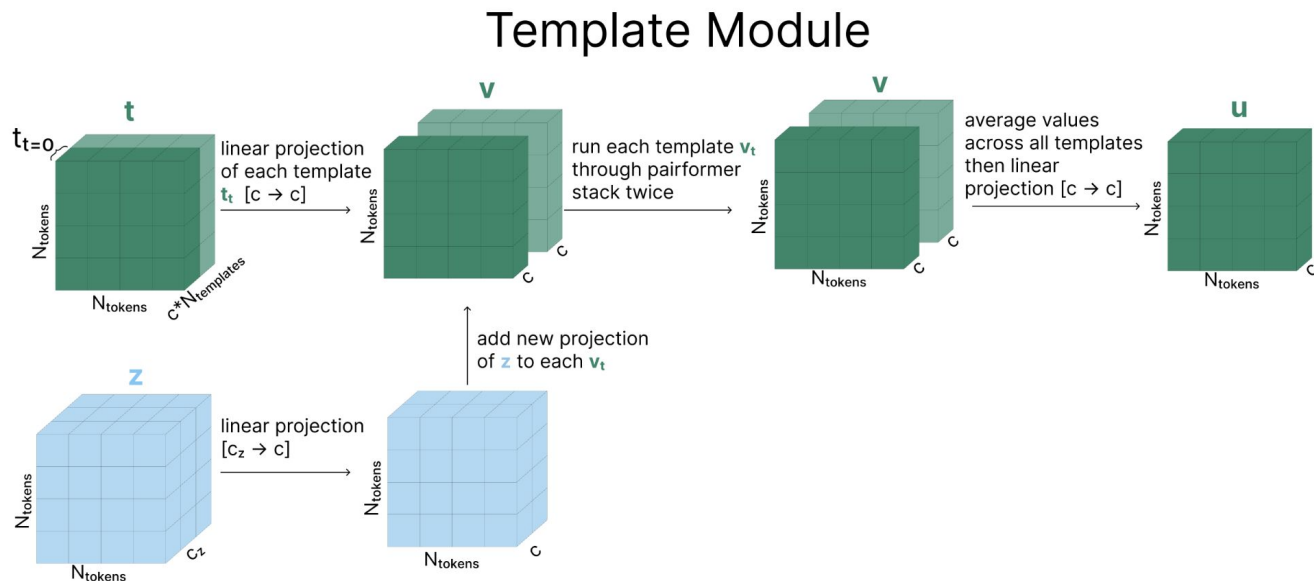


Template Module

MSA module

Pairformer

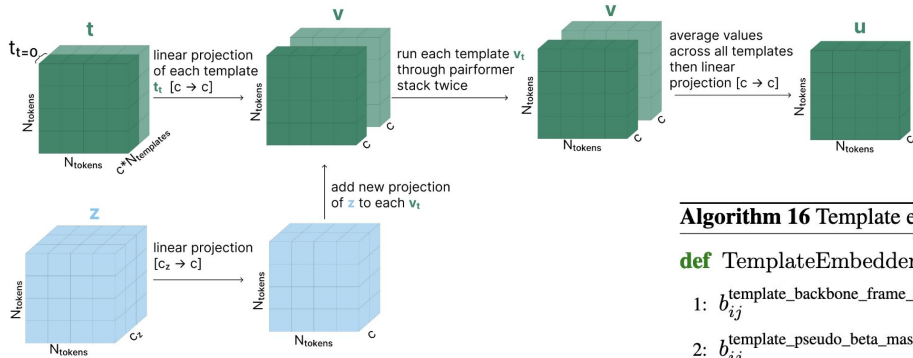
Template Module



- Each of the input templates (e.g., 2) is linearly projected and added to a projection of the pair representation $\mathbf{z}$.
- The result is passed through a **Pairformer Stack** to update structural relationships.
- Outputs from all templates are averaged and passed through a **ReLU-activated linear layer** (one of only two ReLUs in AF3).
- This module injects structural priors from templates into the pairwise representation.

Template Module

Template Module



Algorithm 16 Template embedder

def TemplateEmbedder($\{f^*\}, \{z_{ij}\}, N_{\text{block}} = 2, c = 64$) :

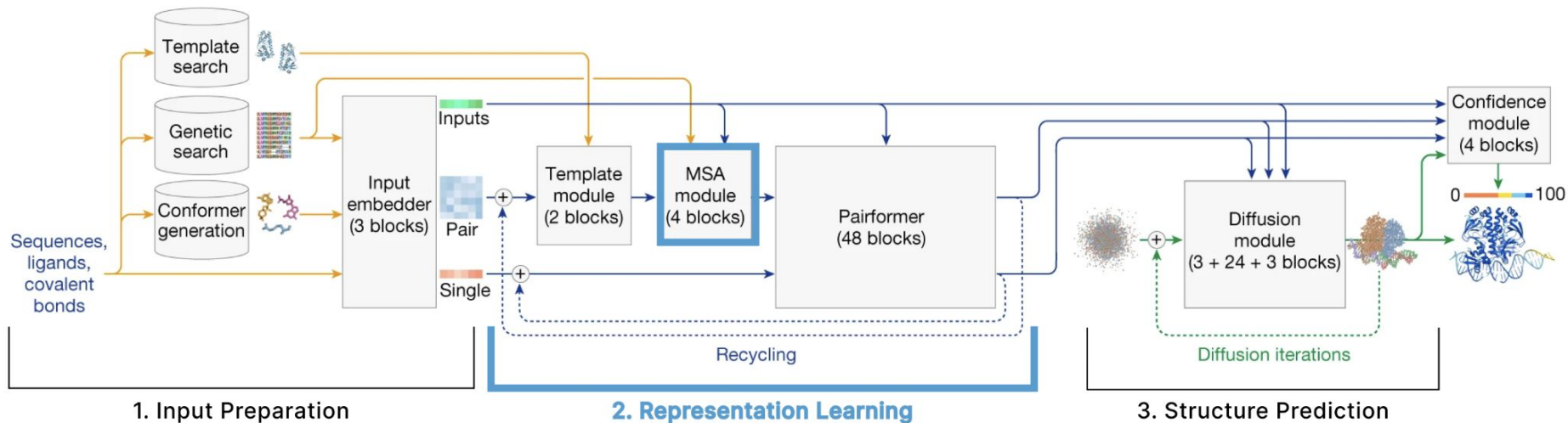
- 1: $b_{ij}^{\text{template_backbone_frame_mask}} = f_{ti}^{\text{template_backbone_frame_mask}} \cdot f_{tj}^{\text{template_backbone_frame_mask}}$
- 2: $b_{ij}^{\text{template_pseudo_beta_mask}} = f_{ti}^{\text{template_pseudo_beta_mask}} \cdot f_{tj}^{\text{template_pseudo_beta_mask}}$
- 3: $a_{tij} = \text{concat}(f_{tij}^{\text{template_distogram}}, b_{ij}^{\text{template_backbone_frame_mask}}, f_{tij}^{\text{template_unit_vector}}, b_{ij}^{\text{template_pseudo_beta_mask}})$
- 4: $a_{tij} \leftarrow a_{tij} \odot (f_i^{\text{asym_id}} == f_j^{\text{asym_id}})$
- 5: $a_{tij} \leftarrow \text{concat}(a_{tij}, f_{ti}^{\text{template_restype}}, f_{tj}^{\text{template_restype}})$
- 6: $u_{ij} = 0$
- 7: **for all** $t \in [1, \dots, N_{\text{templates}}]$ **do**
- 8: $v_{ij} = \text{LinearNoBias}(\text{LayerNorm}(z_{ij})) + \text{LinearNoBias}(a_{tij})$
- 9: $\{v_{ij}\} += \text{PairformerStack}(\{v_{ij}\}, N_{\text{block}})$
- 10: $u_{ij} += \text{LayerNorm}(v_{ij})$
- 11: **end for**
- 12: $u_{ij} \leftarrow \frac{u_{ij}}{N_{\text{templates}}}$
- 13: $u_{ij} \leftarrow \text{LinearNoBias}(\text{ReLU}(u_{ij}))$
- 14: **return** $\{u_{ij}\}$

$$u_{ij} \in \mathbb{R}^c$$

$$v_{ij} \in \mathbb{R}^c$$

$$u_{ij} \in \mathbb{R}^c$$

MSA Module



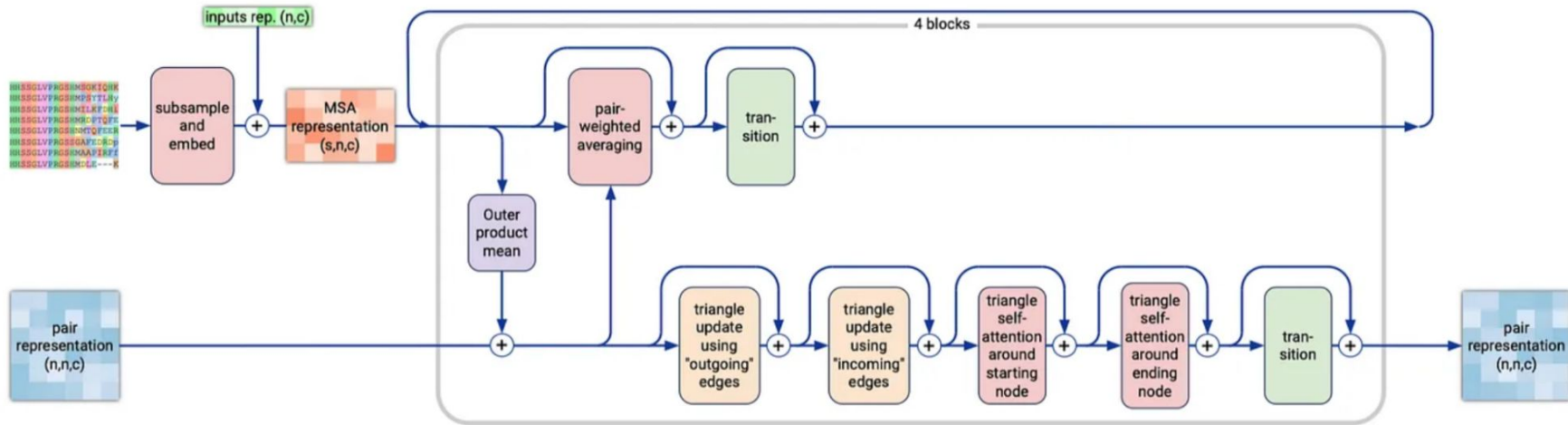
Template Module

MSA module

Pairformer

- Outer product mean
- Row-wise gated attention using only pair bias

MSA Module

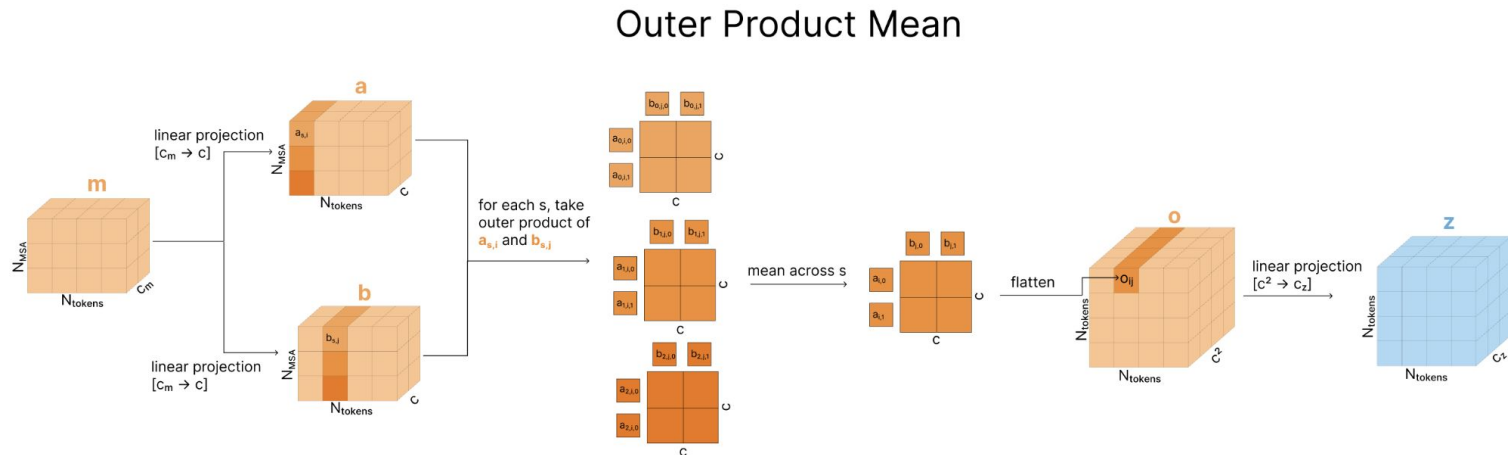


The MSA Module is the evolutionary reasoning engine of the model. It resembles the **Evoformer** from AlphaFold 2 but is optimized for speed and scalability by limiting cross-sequence operations.

➤ Step 1: MSA Subsampling and Initialization

- Instead of using the full MSA (which may have **up to 16k sequences**), AlphaFold 3 subsamples it to a smaller, computationally manageable set.
- The token-level single representation **s** is projected and **added to each row of the subsampled MSA**, seeding each sequence with positional context.

MSA Module - Outer Product Mean

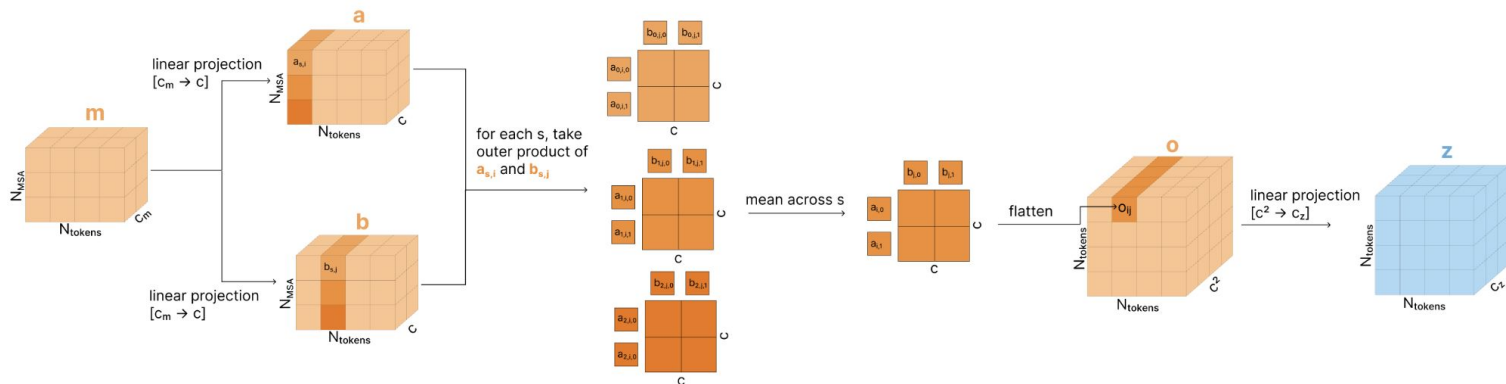


Step 2: Outer Product Mean (MSA $\rightarrow$ z)

- This step injects evolutionary coupling signals from the MSA into the pair representation z :
 - For each sequence and token pair (i, j) , compute the **outer product** of their MSA embeddings: `outer(ms, i, ms, j)`.
 - Average this across all MSA sequences to get a single feature per token pair.
 - Flatten and project this tensor back down to the pair channel size (e.g., $c_2 = 128$).
 - **Add to $z_{i,j}$**
- Crucially, **this is the only point** where cross-sequence information from the MSA is mixed — reducing memory costs compared to AF2.

MSA Module - Outer Product Mean

Outer Product Mean



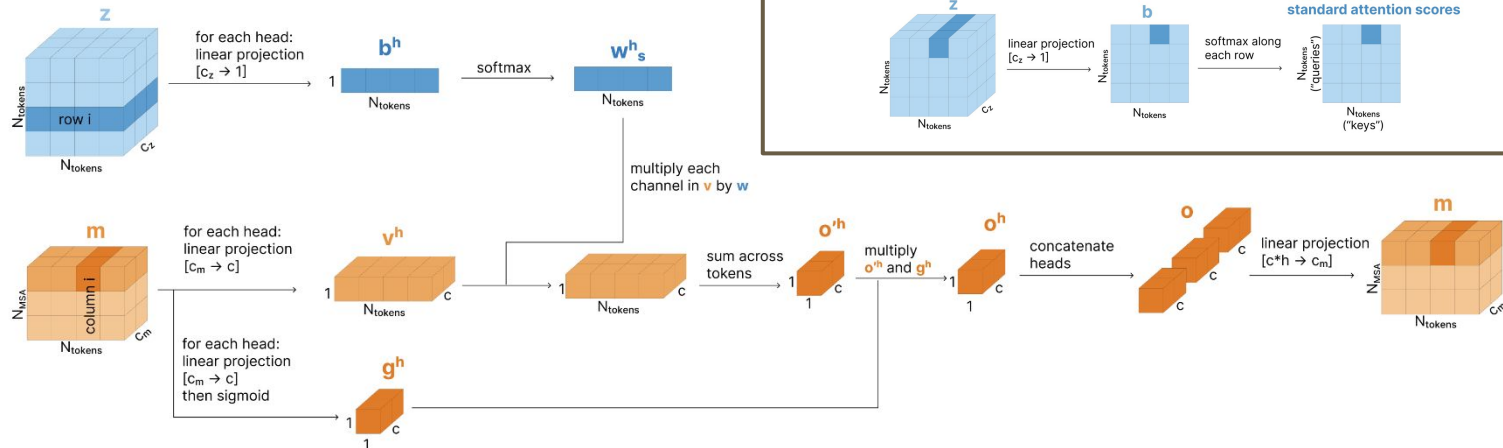
Algorithm 9 Outer product mean

def OuterProductMean($\{\mathbf{m}_{si}\}, c = 32, c_z = 128$) :

- 1: $\mathbf{m}_{si} \leftarrow \text{LayerNorm}(\mathbf{m}_{si})$
 - 2: $\mathbf{a}_{si}, \mathbf{b}_{si} = \text{LinearNoBias}(\mathbf{m}_{si})$
 - 3: $\mathbf{o}_{ij} = \text{flatten}(\text{mean}_s(\mathbf{a}_{si} \otimes \mathbf{b}_{sj}))$
 - 4: $\mathbf{z}_{ij} = \text{Linear}(\mathbf{o}_{ij})$
 - 5: **return** $\{\mathbf{z}_{ij}\}$
-

MSA Module - Row-wise gated self-attention using only pair bias

row i in z becomes the attention map specifying where column i in m should attend to

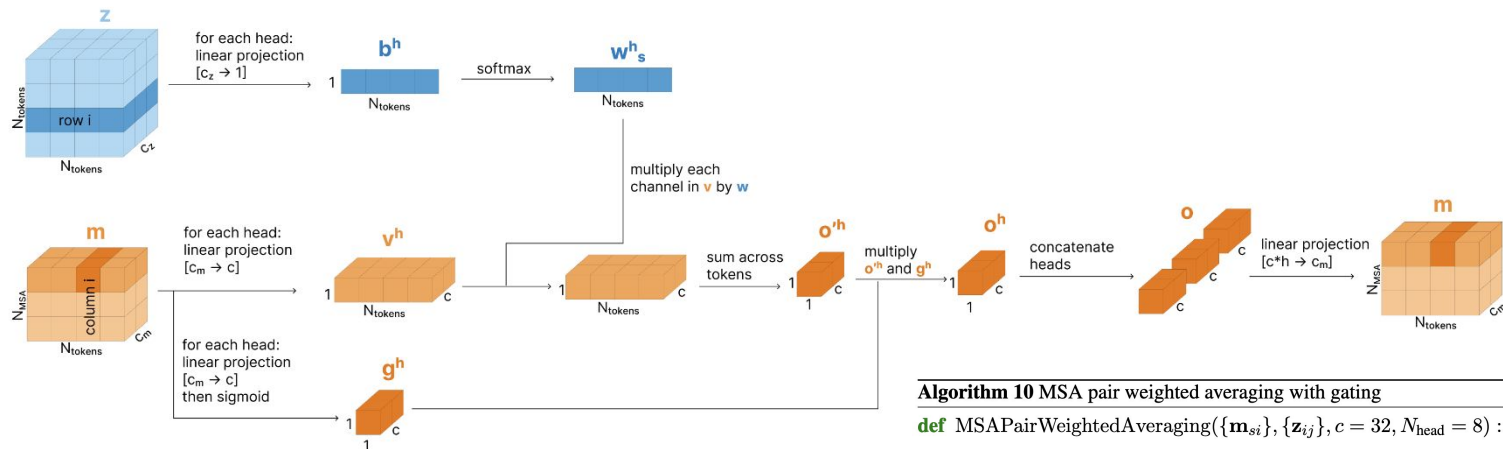


➤ Step 3: Row-wise Gated Self-Attention Using Only Pair Bias ($z \rightarrow m$)

- Updates each row of the MSA independently based on the current pairwise representation z :
 - Rather than using queries/keys as in standard attention, $z_{i,j}$ directly biases attention scores.
 - Each $z_{i,j}$ is projected to a scalar and passed through a **row-wise softmax** to act as attention weights.
 - The resulting scores are used to aggregate value vectors within each MSA row.
 - A **gating mechanism** (sigmoid) controls how much of the attention output is retained.
- Note: **No information is exchanged across different MSA rows** (i.e., no inter-sequence communication here).

MSA Module - Row-wise gated self-attention using only pair bias

row i in z becomes the attention map specifying where column i in m should attend to



Algorithm 10 MSA pair weighted averaging with gating

def MSAPairWeightedAveraging($\{\mathbf{m}_{si}\}, \{\mathbf{z}_{ij}\}, c = 32, N_{head} = 8$) :

Input projections

- 1: $\mathbf{m}_{si} \leftarrow \text{LayerNorm}(\mathbf{m}_{si})$
- 2: $\mathbf{v}_{si}^h = \text{LinearNoBias}(\mathbf{m}_{si})$
- 3: $\mathbf{b}_{ij}^h = \text{LinearNoBias}(\text{LayerNorm}(\mathbf{z}_{ij}))$
- 4: $\mathbf{g}_{si}^h = \text{sigmoid}(\text{LinearNoBias}(\mathbf{m}_{si}))$

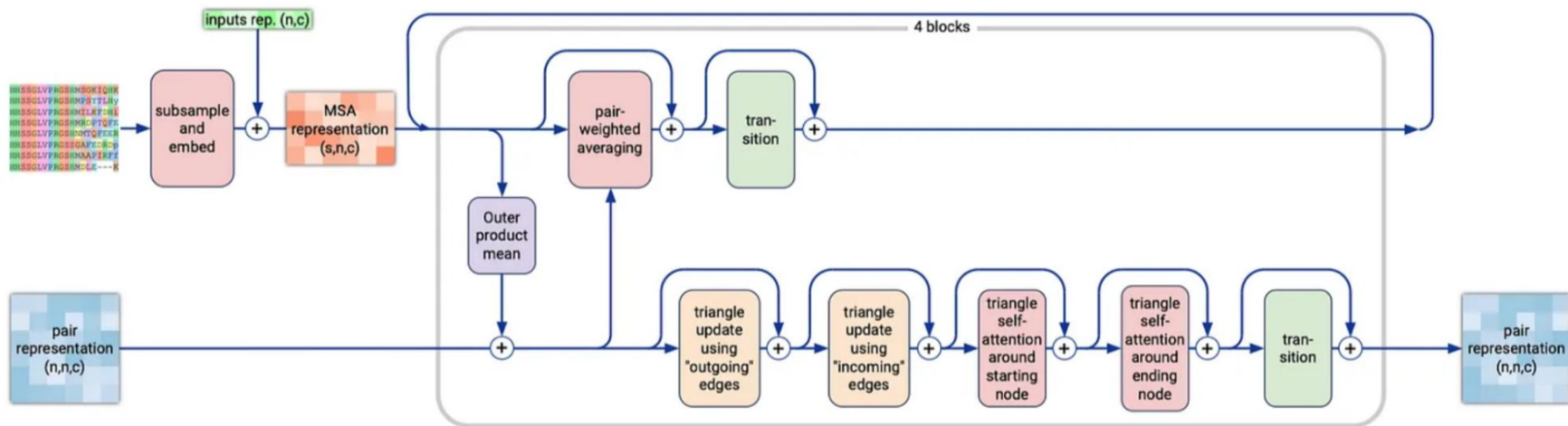
Weighted average with gating

- 5: $w_{ij}^h = \text{softmax}_j(\mathbf{b}_{ij}^h)$
- 6: $\mathbf{o}_{si}^h = \mathbf{g}_{si}^h \odot \sum_j w_{ij}^h \mathbf{v}_{sj}^h$

Output projection

- 7: $\tilde{\mathbf{m}}_{si} = \text{LinearNoBias}(\text{concat}_h(\mathbf{o}_{si}^h))$
- 8: **return** $\{\tilde{\mathbf{m}}_{si}\}$

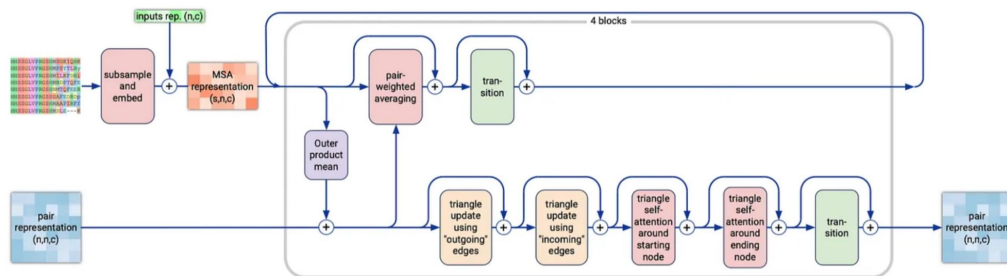
MSA Module - Updates to pair representation



➤ Step 4: Triangle Updates and SwiGLU Transitions ($m + z$)

- After mutual updates between **m** and **z**, a final refinement to **z** is performed:
 - **Triangle Multiplication, Triangle Attention** — use geometric reasoning to propagate indirect residue relationships (e.g., if *i* interacts with *j* and *j* with *k*, then *i* and *k* may interact).
 - **SwiGLU MLPs** refine feature dimensions post-attention.

MSA Module



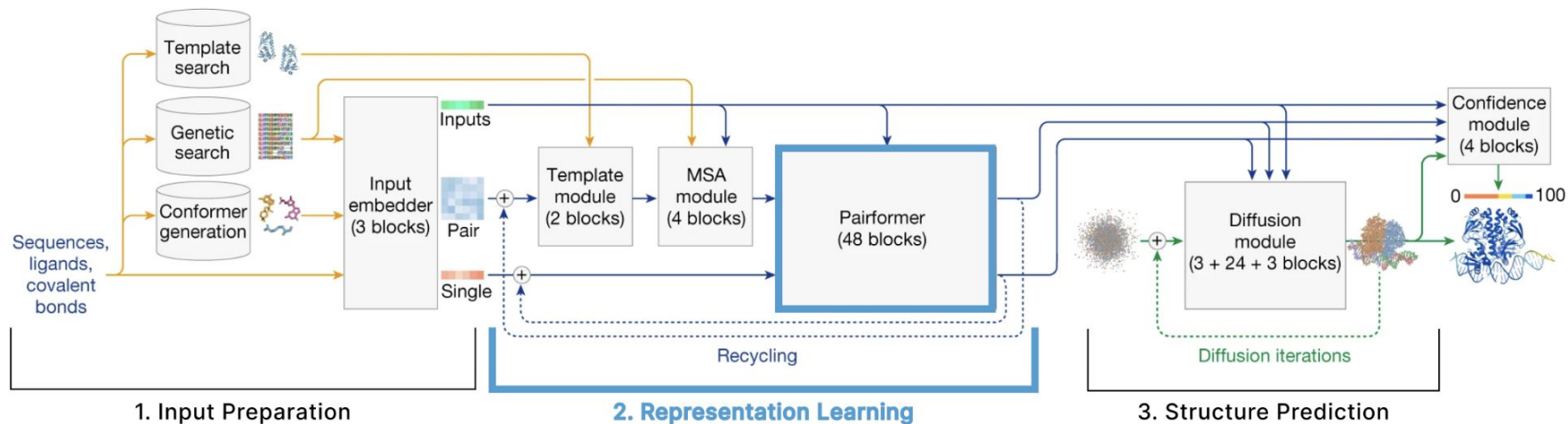
Algorithm 8 MSA Module

```

def MsaModule( $\{f^*\}$ ,  $\{z_{ij}\}$ ,  $\{s_i^{inputs}\}$ ,  $N_{block} = 4$ ,  $c_m = 64$ ) :
    1:  $m_{si} = \text{concat}(f_{Si}^{msa}, f_{Si}^{has\_deletion}, f_{Si}^{deletion\_value})$ 
    2:  $\{s\} = \text{SampleRandomWithoutReplacement}(\{S\})$ 
    3:  $m_{si} \leftarrow \text{LinearNoBias}(m_{si})$ 
    4:  $m_{si} += \text{LinearNoBias}(\{s_i^{inputs}\})$ 
    5: for all  $l \in [1, \dots, N_{block}]$  do
        # Communication
        6:  $\{z_{ij}\} += \text{OuterProductMean}(\{m_{si}\})$ 
        # MSA stack
        7:  $\{m_{si}\} += \text{DropoutRowwise}_{0.15}(\text{MSAPairWeightedAveraging}(\{m_{si}\}, \{z_{ij}\}, c = 8))$ 
        8:  $\{m_{si}\} += \text{Transition}(\{m_{si}\})$ 
        # Pair stack
        9:  $\{z_{ij}\} += \text{DropoutRowwise}_{0.25}(\text{TriangleMultiplicationOutgoing}(\{z_{ij}\}))$ 
        10:  $\{z_{ij}\} += \text{DropoutRowwise}_{0.25}(\text{TriangleMultiplicationIncoming}(\{z_{ij}\}))$ 
        11:  $\{z_{ij}\} += \text{DropoutRowwise}_{0.25}(\text{TriangleAttentionStartingNode}(\{z_{ij}\}))$ 
        12:  $\{z_{ij}\} += \text{DropoutColumnwise}_{0.25}(\text{TriangleAttentionEndingNode}(\{z_{ij}\}))$ 
        13:  $\{z_{ij}\} += \text{Transition}(\{z_{ij}\})$ 
    14: end for
    15: return  $\{z_{ij}\}$ 

```

Pairformer Module



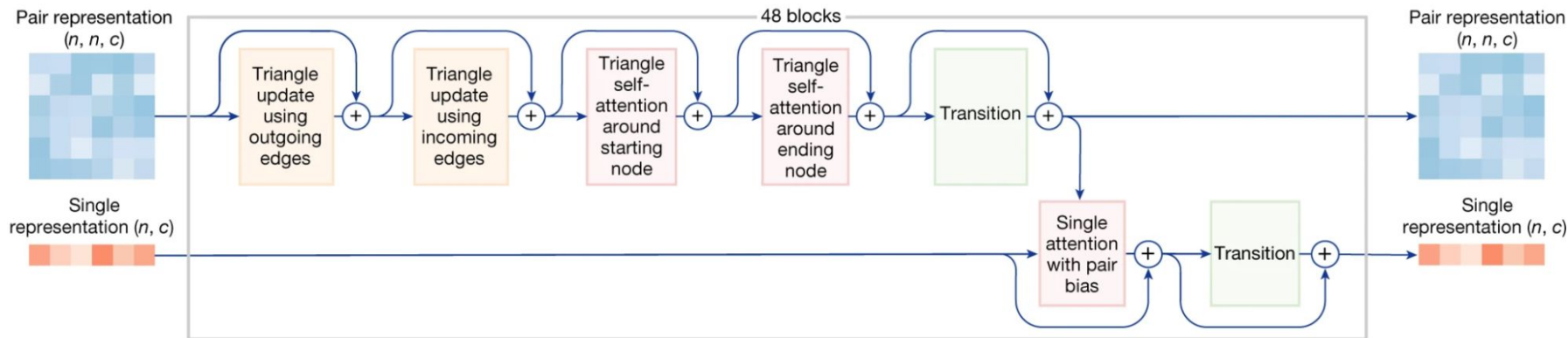
Template Module

MSA module

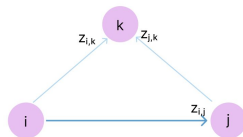
Pairformer

- Triangle Updates
- Triangle Attention
- Attention with Pair Bias (token-level)

Pairformer



After using templates and MSA to update the pair representation ($\mathbf{z}$), AF3 discards them and focuses only on $\mathbf{z}$ and the single representation $\mathbf{s}$. These are now updated via the **Pairformer**, a core architectural block rich with triangle-based reasoning, a key idea from AF2 that remains central in AF3.

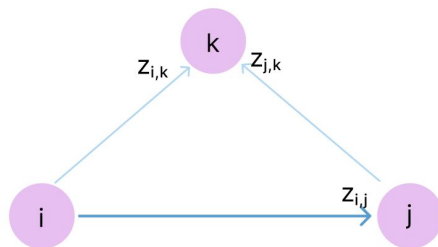


Why Triangles?

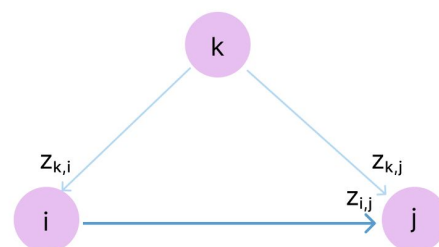
- **Inspired by the triangle inequality:** If you know the relation (e.g., distance) between $i \rightarrow j$ and $j \rightarrow k$, you can infer constraints on $i \rightarrow k$.
- Though **not physically enforced**, triangle consistency is encouraged through updates that consider all triplets (i, j, k) .
- This helps encode **geometric constraints** and encourages physical plausibility in structural predictions.

Pairformer

“outgoing edges”



“incoming edges”



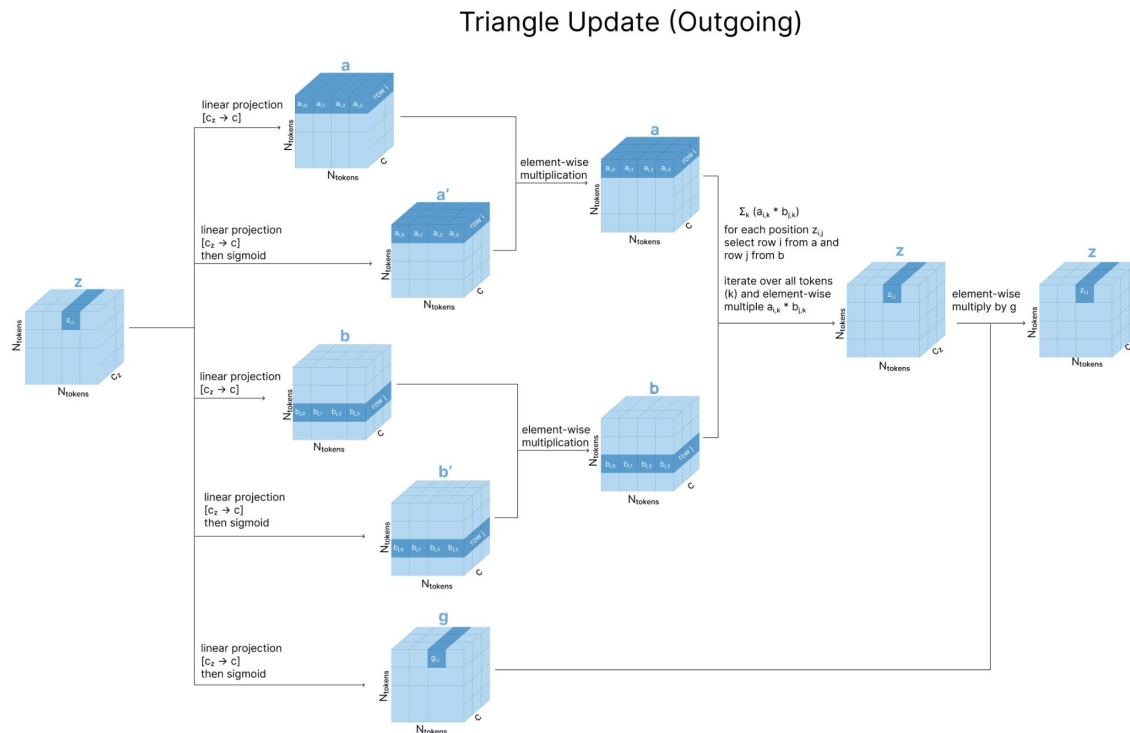
Triangle Operations:

- For each pairwise representation $z_{(i,j)}$, the model considers **intermediate nodes k** to update it:
 - Outgoing Triangle Attention/Update:** Uses $z_{(i,k)}$ and $z_{(j,k)}$.
 - Incoming Triangle Attention/Update:** Uses $z_{(k,i)}$ and $z_{(k,j)}$.
- These updates treat z as a **directed graph**, where attention flows along incoming or outgoing edges, encouraging relational consistency across triplets.

Single Attention with Pair Bias:

- Similar to earlier attention with pair bias (from Atom Transformer), but now operates at the **token-level**.
- s** (single) is updated using **z** (pair) as attention bias.
- Helps infuse sequence-level features with geometric and relational cues.

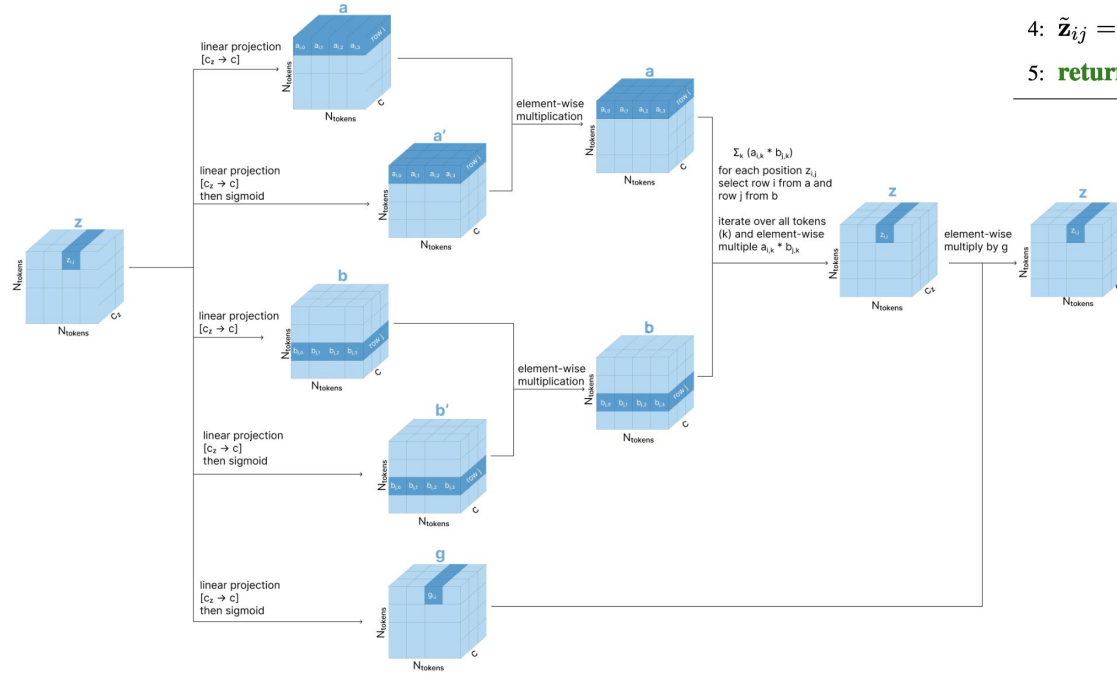
Pairformer - Triangle Updates



Practically, we take three linear projections of $\mathbf{z}$ (called $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{g}$). To update $\mathbf{z}_{i,j}$, we take an element-wise multiplication of **row i** from $\mathbf{a}$ and **row j** from $\mathbf{b}$. We then sum over all these rows (different values of k), and **gate** with our $\mathbf{g}$ projection.

Pairformer - Triangle Updates

Triangle Update (Outgoing)



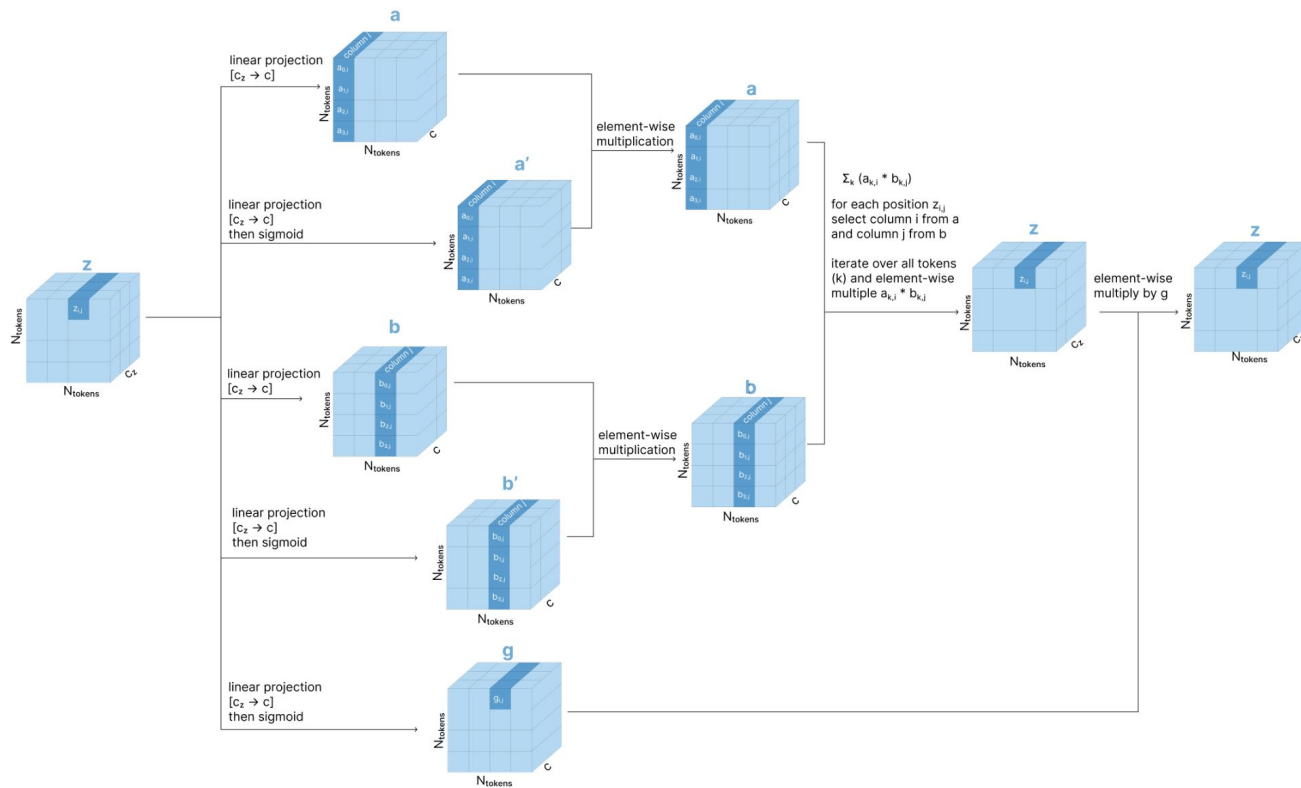
Algorithm 12 Triangular multiplicative update using “outgoing” edges

def TriangleMultiplicationOutgoing($\{z_{ij}\}$, $c = 128$) :

- 1: $z_{ij} \leftarrow \text{LayerNorm}(z_{ij})$
 - 2: $a_{ij}, b_{ij} = \text{sigmoid}(\text{LinearNoBias}(z_{ij})) \odot \text{LinearNoBias}(z_{ij})$
 - 3: $g_{ij} = \text{sigmoid}(\text{LinearNoBias}(z_{ij}))$
 - 4: $\tilde{z}_{ij} = g_{ij} \odot \text{LinearNoBias}(\text{LayerNorm}(\sum_k a_{ik} \odot b_{jk}))$
 - 5: **return** $\{\tilde{z}_{ij}\}$
-

Pairformer - Triangle Updates

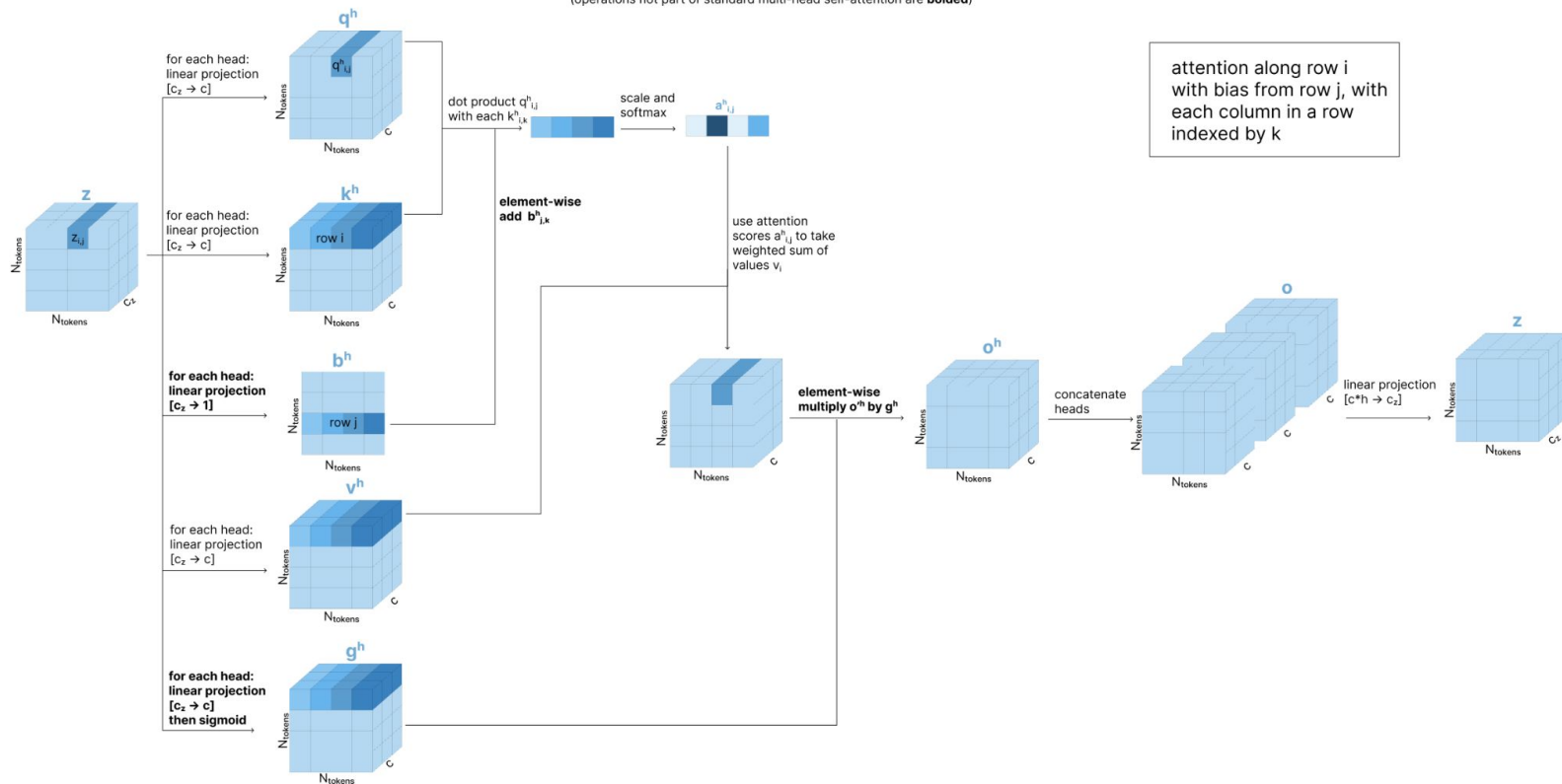
Triangle Update (Incoming)



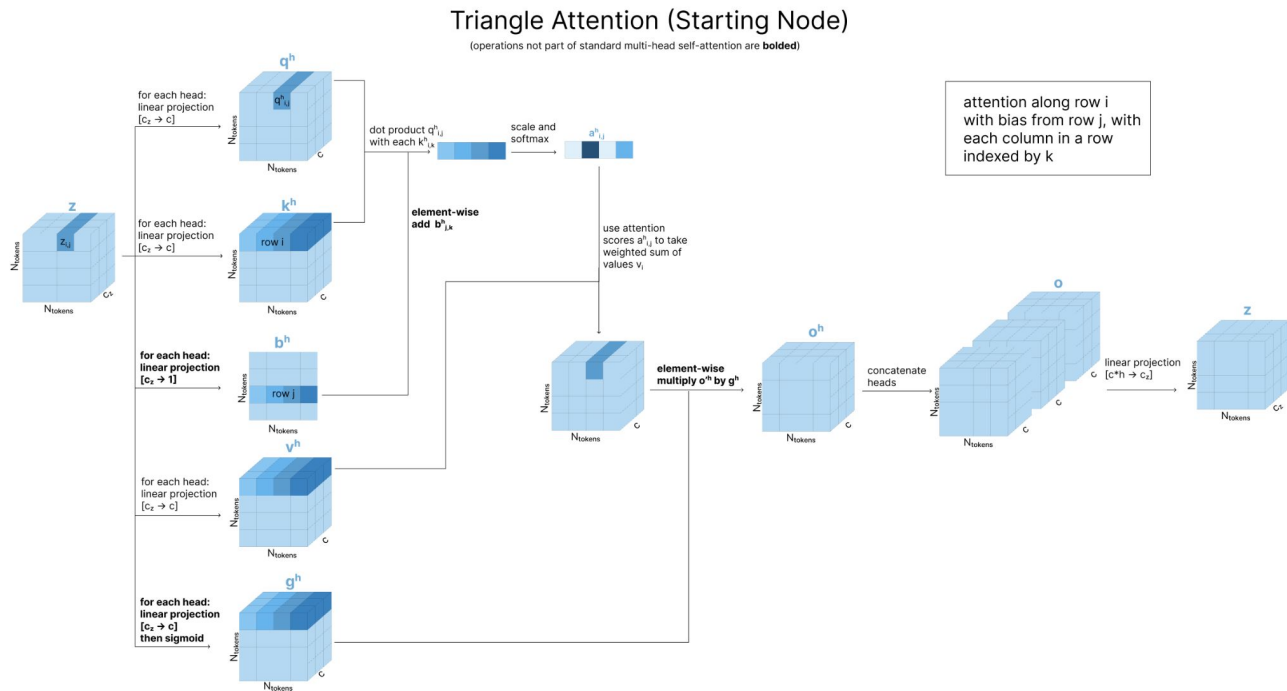
Pairformer - Triangle Attention

Triangle Attention (Starting Node)

(operations not part of standard multi-head self-attention are **bolded**)



Pairformer - Triangle Attention

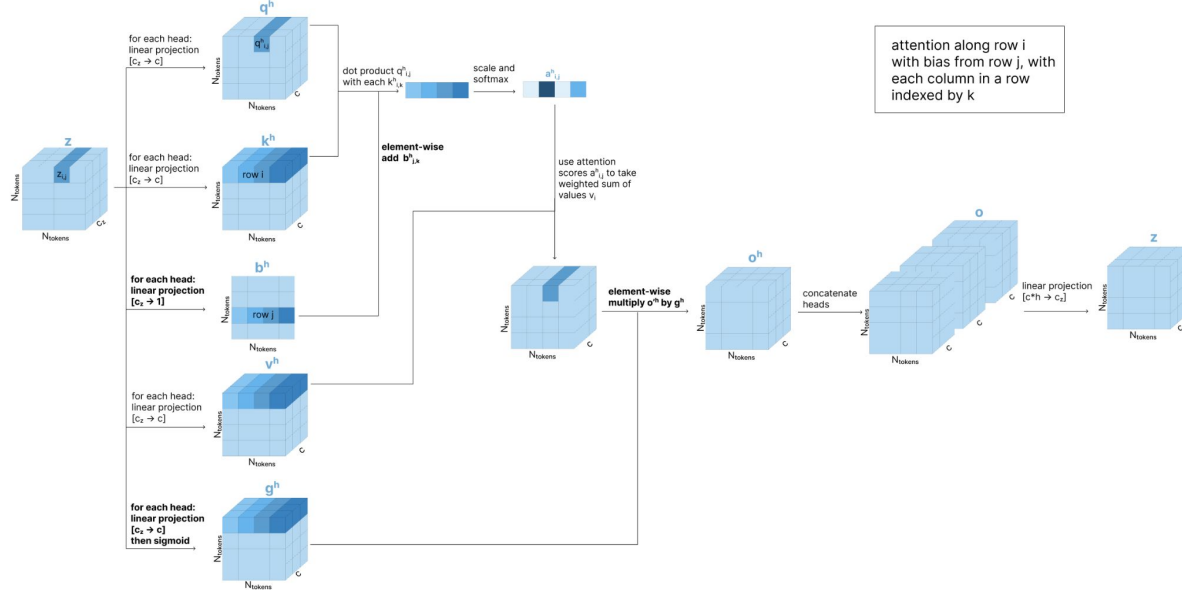


- **Built on standard self-attention:** Queries, keys, and values are derived from $\mathbf{z}$.
- **Adds triangle bias:** This means the attention score between two edges (e.g., $\mathbf{z}_{i,j}$ and $\mathbf{z}_{i,k}$) is **biased** based on a third relationship ($\mathbf{z}_{j,k}$).
- Think of it as **attention over a 2D matrix ($\mathbf{z}$)** guided by a third point in the triangle.

Pairformer - Triangle Attention

Triangle Attention (Starting Node)

(operations not part of standard multi-head self-attention are **bolded**)



Algorithm 14 Triangular gated self-attention around starting node

def TriangleAttentionStartingNode($\{\mathbf{z}_{ij}\}$, $c = 32$, $N_{\text{head}} = 4$) :

Input projections

1: $\mathbf{z}_{ij} \leftarrow \text{LayerNorm}(\mathbf{z}_{ij})$

2: $\mathbf{q}_{ij}^h, \mathbf{k}_{ij}^h, \mathbf{v}_{ij}^h = \text{LinearNoBias}(\mathbf{z}_{ij})$

3: $\mathbf{b}_{ij}^h = \text{LinearNoBias}(\mathbf{z}_{ij})$

4: $\mathbf{g}_{ij}^h = \text{sigmoid}(\text{LinearNoBias}(\mathbf{z}_{ij}))$

Attention

5: $\mathbf{a}_{ijk}^h = \text{softmax}_k \left(\frac{1}{\sqrt{c}} \mathbf{q}_{ij}^{h\top} \mathbf{k}_{ik}^h + \mathbf{b}_{jk}^h \right)$

6: $\mathbf{o}_{ij}^h = \mathbf{g}_{ij}^h \odot \sum_k \mathbf{a}_{ijk}^h \mathbf{v}_{ik}^h$

Output projection

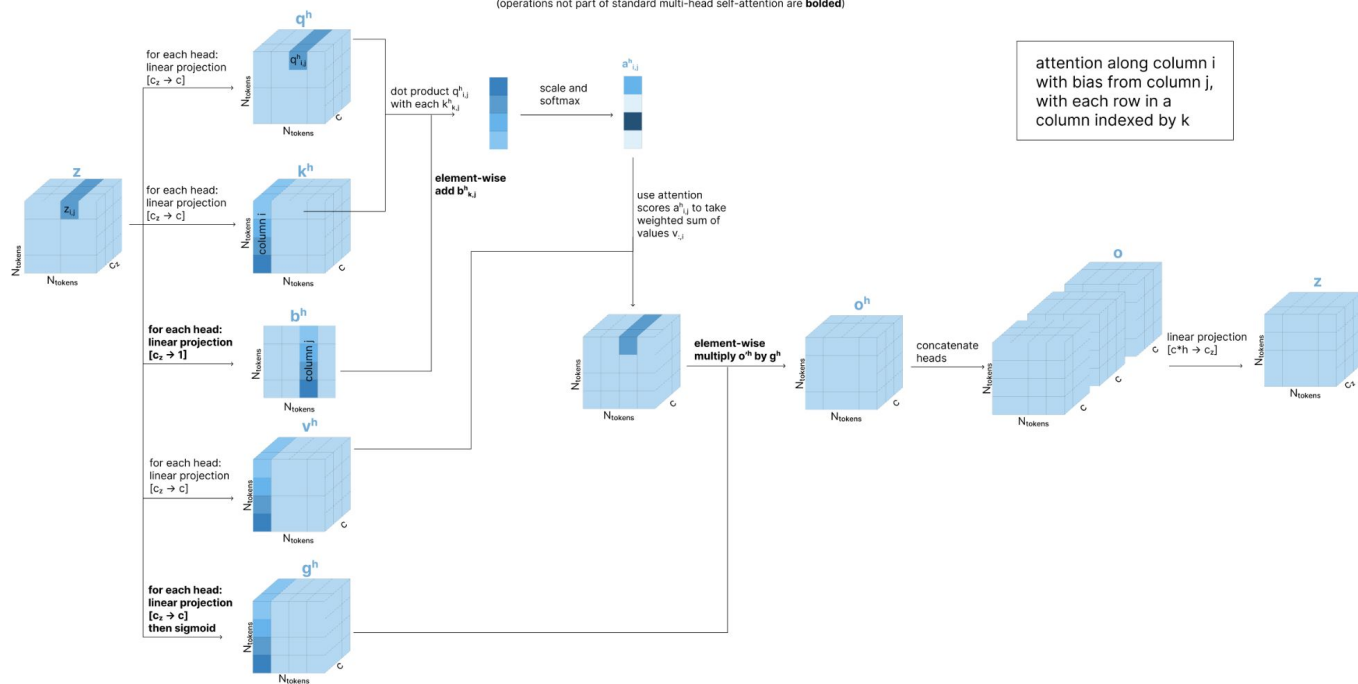
7: $\mathbf{\hat{z}}_{ij} = \text{LinearNoBias}(\text{concat}_h(\mathbf{o}_{ij}^h))$

8: **return** $\{\mathbf{\hat{z}}_{ij}\}$

Pairformer - Triangle Attention

Triangle Attention (Ending Node)

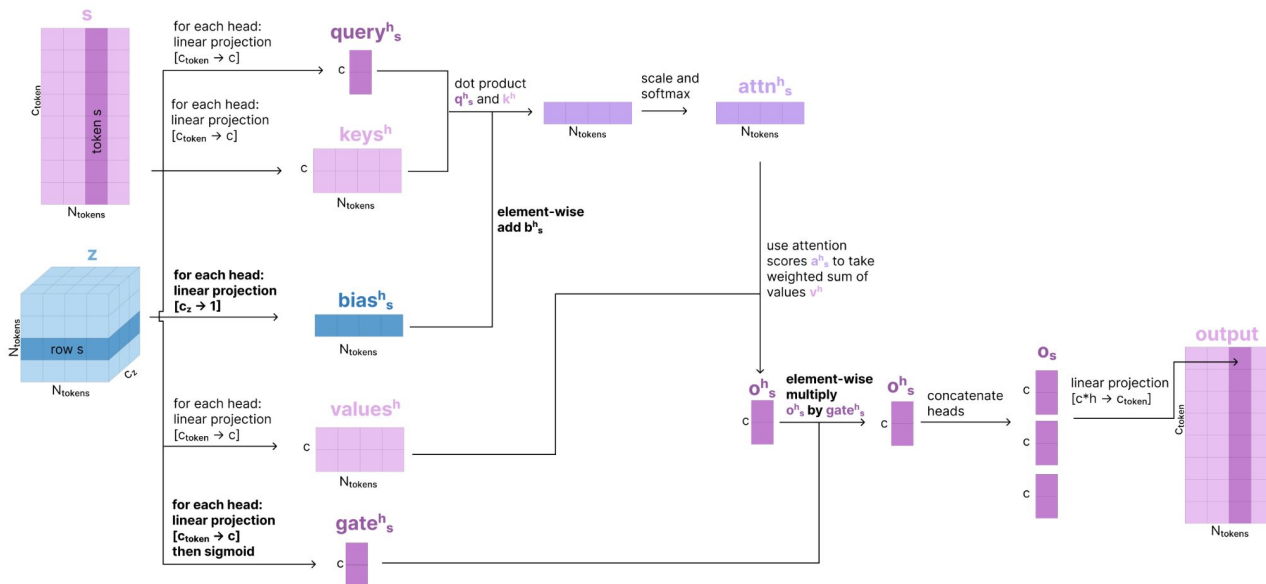
(operations not part of standard multi-head self-attention are **bolded**)



Pairformer - Single Attention with Pair Bias

Attention with pair-bias (Token-Level)

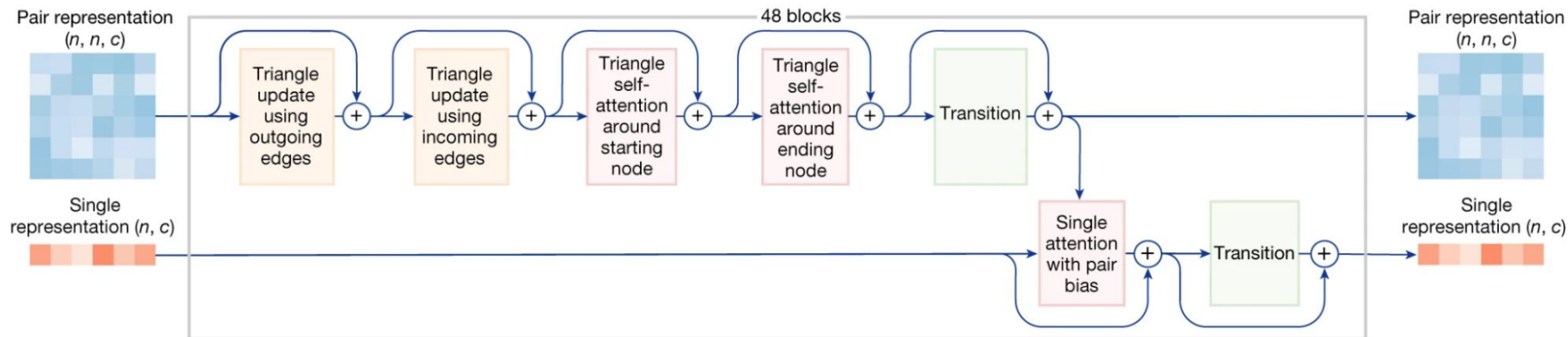
demonstrating the process for updating one token at a time (token s)
operations not part of standard multi-head self-attention are **bolded**



After the triangle updates and attentions, the pair representation z goes through a Transition block. Then, the single representation s is updated using **Single Attention with Pair Bias** at the token-level with full attention (unlike sparse atom-level attention).

This Pairformer block is repeated **48 times**, producing the final refined representations **s_trunk** and **z_trunk**.

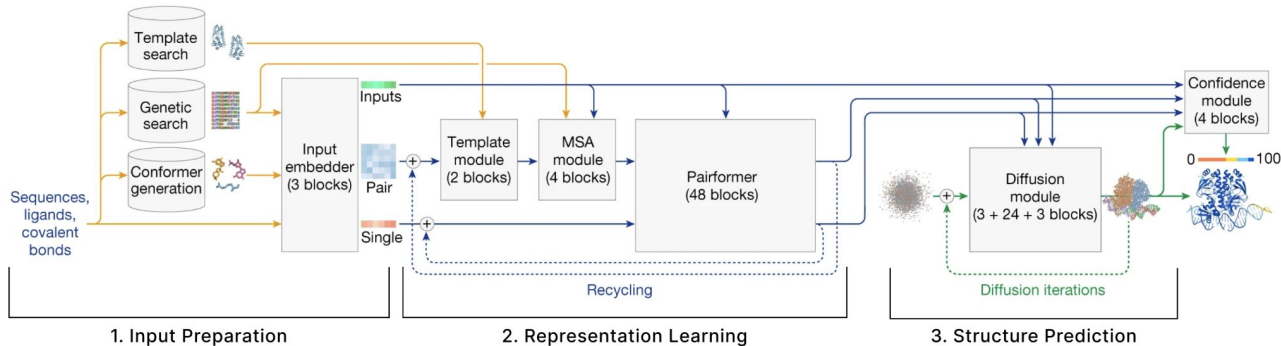
Pairformer Module



Algorithm 17 Pairformer stack

```

def PairformerStack( $\{\mathbf{s}_i\}, \{\mathbf{z}_{ij}\}, N_{\text{block}} = 48$ ) :
    1: for all  $l \in [1, \dots, N_{\text{block}}]$  do
    # Pair stack
    2:  $\{\mathbf{z}_{ij}\} \mathrel{+}= \text{DropoutRowwise}_{0.25}(\text{TriangleMultiplicationOutgoing}(\{\mathbf{z}_{ij}\}))$ 
    3:  $\{\mathbf{z}_{ij}\} \mathrel{+}= \text{DropoutRowwise}_{0.25}(\text{TriangleMultiplicationIncoming}(\{\mathbf{z}_{ij}\}))$ 
    4:  $\{\mathbf{z}_{ij}\} \mathrel{+}= \text{DropoutRowwise}_{0.25}(\text{TriangleAttentionStartingNode}(\{\mathbf{z}_{ij}\}))$ 
    5:  $\{\mathbf{z}_{ij}\} \mathrel{+}= \text{DropoutColumnwise}_{0.25}(\text{TriangleAttentionEndingNode}(\{\mathbf{z}_{ij}\}))$ 
    6:  $\{\mathbf{z}_{ij}\} \mathrel{+}= \text{Transition}(\{\mathbf{z}_{ij}\})$ 
    7:  $\{\mathbf{s}_i\} \mathrel{+}= \text{AttentionPairBias}(\{\mathbf{s}_i\}, \emptyset, \{\mathbf{z}_{ij}\}, \beta_{ij} = 0, N_{\text{head}} = 16)$ 
    8:  $\{\mathbf{s}_i\} \mathrel{+}= \text{Transition}(\{\mathbf{s}_i\})$ 
    9: end for
    10: return  $\{\mathbf{s}_i\}, \{\mathbf{z}_{ij}\}$ 
    
```



Algorithm 1 Main Inference Loop

def MainInferenceLoop($\{f^*\}$, $N_{\text{cycle}} = 4$, $c_s = 384$, $c_z = 128$) :

```

1:  $\{s_i^{\text{inputs}}\} = \text{InputFeatureEmbedder}(\{f^*\})$ 
2:  $s_i^{\text{init}} = \text{LinearNoBias}(s_i^{\text{inputs}})$ 
3:  $z_{ij}^{\text{init}} = \text{LinearNoBias}(s_i^{\text{inputs}}) + \text{LinearNoBias}(s_j^{\text{inputs}})$ 
4:  $z_{ij}^{\text{init}} += \text{RelativePositionEncoding}(\{f^*\})$ 
5:  $z_{ij}^{\text{init}} += \text{LinearNoBias}(f_{ij}^{\text{token\_bonds}})$ 
6:  $\{\hat{z}_{ij}\}, \{\hat{s}_i\} = \mathbf{0}, \mathbf{0}$ 
7: for all  $c \in [1, \dots, N_{\text{cycle}}]$  do
8:    $z_{ij} = z_{ij}^{\text{init}} + \text{LinearNoBias}(\text{LayerNorm}(\hat{z}_{ij}))$ 
9:    $\{z_{ij}\} += \text{TemplateEmbedder}(\{f^*\}, \{z_{ij}\})$ 
10:   $\{z_{ij}\} += \text{MsaModule}(\{f_{Si}^{\text{msa}}\}, \{z_{ij}\}, \{s_i^{\text{inputs}}\})$ 
11:   $s_i = s_i^{\text{init}} + \text{LinearNoBias}(\text{LayerNorm}(\hat{s}_i))$ 
12:   $\{s_i\}, \{z_{ij}\} = \text{PairformerStack}(\{s_i\}, \{z_{ij}\})$ 
13:   $\{\hat{s}_i\}, \{\hat{z}_{ij}\} \leftarrow \{s_i\}, \{z_{ij}\}$ 
14: end for
15:  $\{\bar{x}_i^{\text{pred}}\} = \text{SampleDiffusion}(\{f^*\}, \{s_i^{\text{inputs}}\}, \{s_i\}, \{z_{ij}\})$ 
16:  $\{p_i^{\text{plddt}}\}, \{p_i^{\text{pae}}\}, \{p_i^{\text{pde}}\}, \{p_i^{\text{resolved}}\} = \text{ConfidenceHead}(\{s_i^{\text{inputs}}\}, \{s_i\}, \{z_{ij}\}, \{\bar{x}_i^{\text{pred}}\})$ 
17:  $p_{ij}^{\text{distogram}} = \text{DistogramHead}(z_{ij})$ 
18: return  $\{\bar{x}_i^{\text{pred}}\}, \{p_i^{\text{plddt}}\}, \{p_i^{\text{pae}}\}, \{p_i^{\text{pde}}\}, \{p_i^{\text{resolved}}\}, \{p_{ij}^{\text{distogram}}\}$ 

```

$$s_i^{\text{init}} \in \mathbb{R}^{c_s}$$

$$z_{ij}^{\text{init}} \in \mathbb{R}^{c_z}$$

$$z_{ij} \in \mathbb{R}^{c_z}$$

$$s_i \in \mathbb{R}^{c_s}$$

End of Part 2

Thanks for listening!

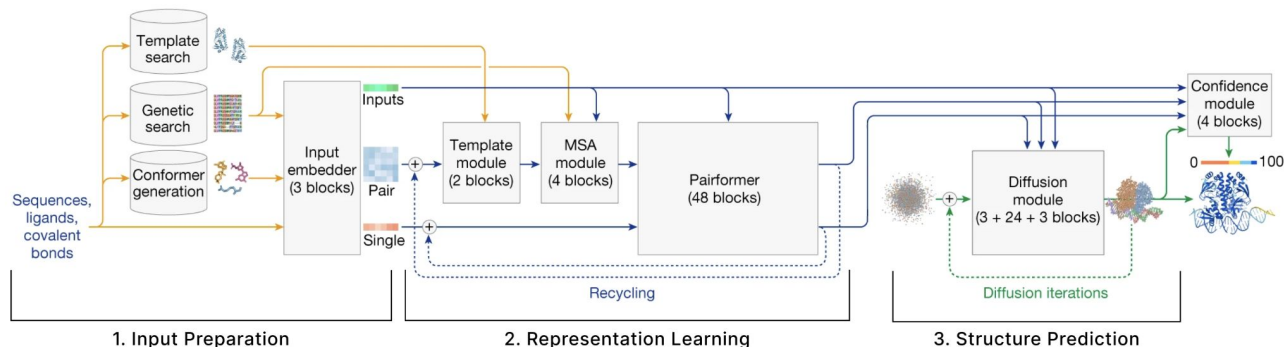
Questions - Comments?

Algorithm 1 Main Inference Loop

```

def MainInferenceLoop( $\{f^*\}$ ,  $N_{\text{cycle}} = 4$ ,  $c_s = 384$ ,  $c_z = 128$ ) :
1:  $\{s_i^{\text{inputs}}\} = \text{InputFeatureEmbedder}(\{f^*\})$ 
2:  $s_i^{\text{init}} = \text{LinearNoBias}(s_i^{\text{inputs}})$   $s_i^{\text{init}} \in \mathbb{R}^{c_s}$ 
3:  $z_{ij}^{\text{init}} = \text{LinearNoBias}(s_i^{\text{inputs}}) + \text{LinearNoBias}(s_j^{\text{inputs}})$   $z_{ij}^{\text{init}} \in \mathbb{R}^{c_z}$ 
4:  $z_{ij}^{\text{init}} += \text{RelativePositionEncoding}(\{f^*\})$ 
5:  $z_{ij}^{\text{init}} += \text{LinearNoBias}(r_{ij}^{\text{token\_bonds}})$ 
6:  $\{\hat{z}_{ij}\}, \{\hat{s}_i\} = \mathbf{0}, \mathbf{0}$ 
7: for all  $c \in [1, \dots, N_{\text{cycle}}]$  do
8:    $z_{ij} = z_{ij}^{\text{init}} + \text{LinearNoBias}(\text{LayerNorm}(\hat{z}_{ij}))$   $z_{ij} \in \mathbb{R}^{c_z}$ 
9:    $\{z_{ij}\} += \text{TemplateEmbedder}(\{f^*\}, \{z_{ij}\})$ 
10:   $\{z_{ij}\} += \text{MsaModule}(\{f_{s_i}^{\text{msa}}\}, \{z_{ij}\}, \{s_i^{\text{inputs}}\})$ 
11:   $s_i = s_i^{\text{init}} + \text{LinearNoBias}(\text{LayerNorm}(\hat{s}_i))$   $s_i \in \mathbb{R}^{c_s}$ 
12:   $\{s_i\}, \{z_{ij}\} = \text{PairformerStack}(\{s_i\}, \{z_{ij}\})$ 
13:   $\{\hat{s}_i\}, \{\hat{z}_{ij}\} \leftarrow \{s_i\}, \{z_{ij}\}$ 
14: end for
15:  $\{\bar{x}_i^{\text{pred}}\} = \text{SampleDiffusion}(\{f^*\}, \{s_i^{\text{inputs}}\}, \{s_i\}, \{z_{ij}\})$ 
16:  $\{p_i^{\text{pddt}}\}, \{p_i^{\text{pae}}\}, \{p_{ij}^{\text{pde}}\}, \{p_i^{\text{resolved}}\} = \text{ConfidenceHead}(\{s_i^{\text{inputs}}\}, \{s_i\}, \{z_{ij}\}, \{\bar{x}_i^{\text{pred}}\})$ 
17:  $p_{ij}^{\text{distogram}} = \text{DistogramHead}(z_{ij})$ 
18: return  $\{\bar{x}_i^{\text{pred}}\}, \{p_i^{\text{pddt}}\}, \{p_i^{\text{pae}}\}, \{p_{ij}^{\text{pde}}\}, \{p_i^{\text{resolved}}\}, \{p_{ij}^{\text{distogram}}\}$ 

```



Accurate structure prediction of biomolecular interactions with AlphaFold 3 (Part 3)

— Abramson, J., Adler, J., Dunger, J. et al. —

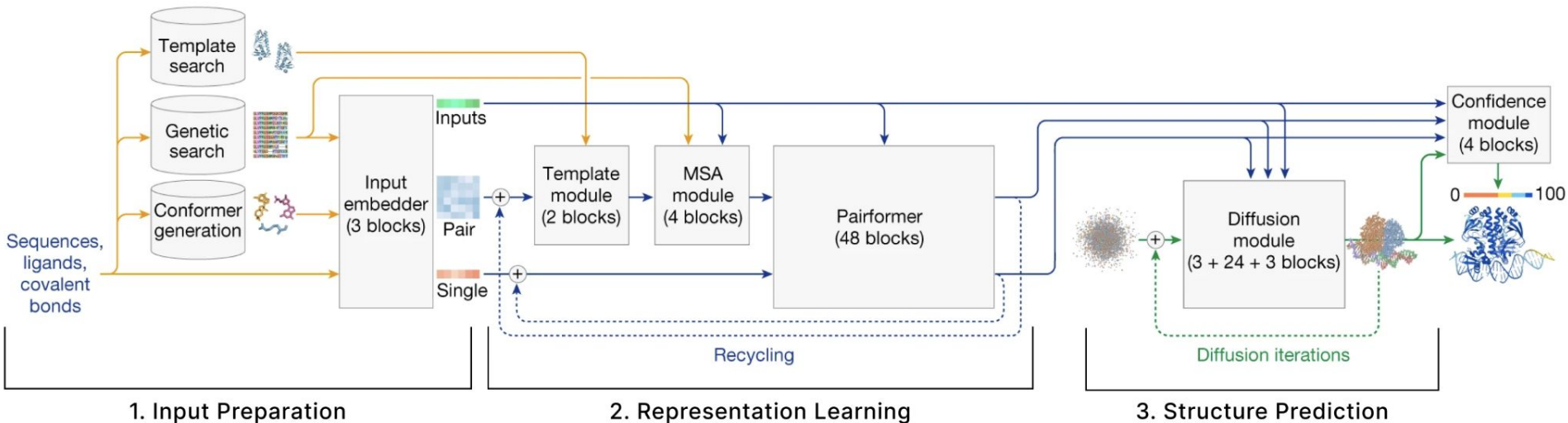
<https://www.nature.com/articles/s41586-024-07487-w>

Previously on Part 2

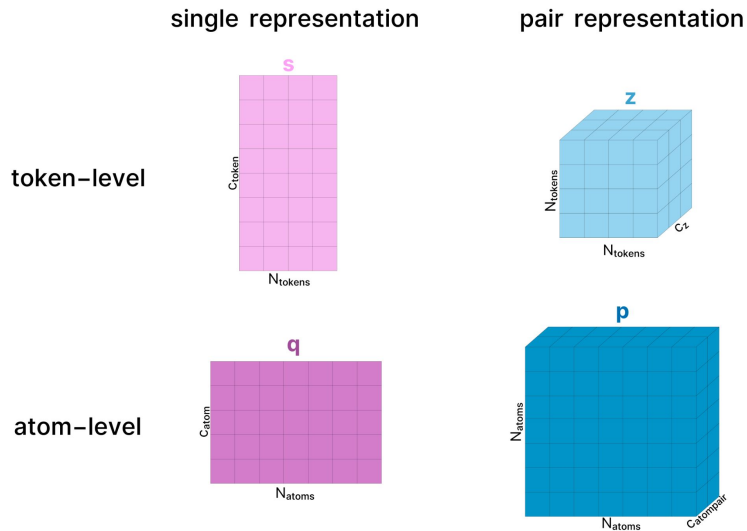
From AlphaFold2 to AlphaFold3 – A Paradigm Shift

Feature	AlphaFold2 (2021)	AlphaFold-Multimer (2021)	AlphaFold3 (2024)
Input types	Protein sequences	Protein sequences	Proteins, RNA, DNA, ligands
Complex prediction	No	Yes (proteins only)	Yes (all molecule types)
Ligand/RNA/DNA modeling	✗	✗	✓
Underlying architecture	Evoformer + structure module	Extended Evoformer	Pairformer + Diffusion
Training task	Supervised (PDB structures)	Supervised (PDB structures)	Conditional diffusion

Architecture Overview



Notes on the variables and diagrams



AlphaFold 3 represents protein complexes using two key tensors: **Single** (per-token/atom features) and **Pair** (relationships between token/atom pairs). These can be at the token or atom level, shown with consistent names and colors.

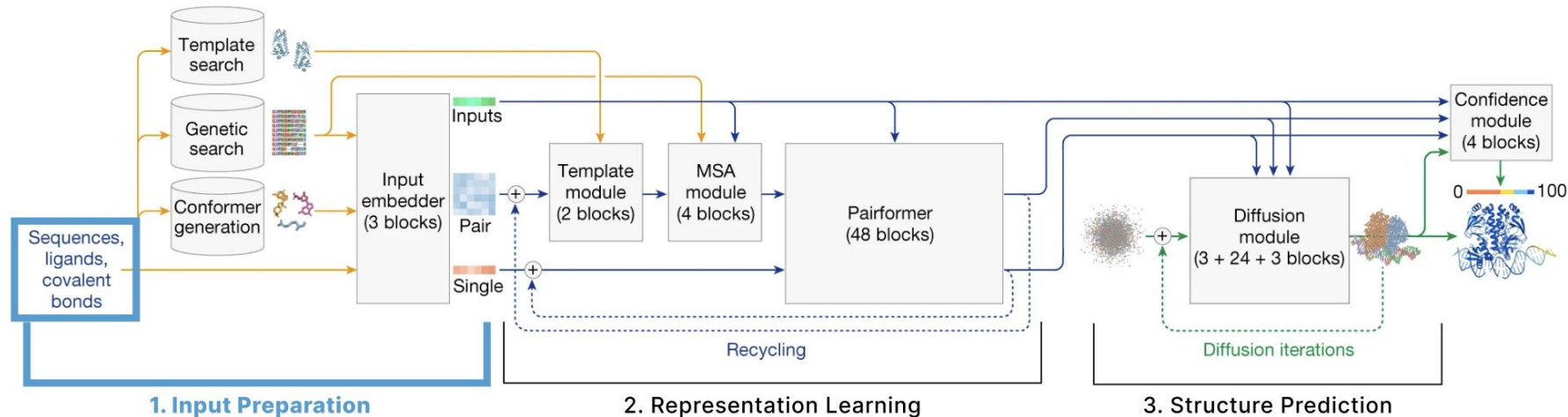
Diagrams show how activation shapes change—not model weights. Tensor labels match those in the AF3 paper. Names are mostly preserved, but some (e.g., **c** to **q** in atom-level single) indicate updates through processing. LayerNorms are used throughout but mostly omitted for clarity.

Input Features

Feature & Shape	Description
residue_index $[N_{\text{token}}]$	Residue number in the token's original input chain.
token_index $[N_{\text{token}}]$	Token number. Increases monotonically; does not restart at 1 for new chains.
asym_id $[N_{\text{token}}]$	Unique integer for each distinct chain.
entity_id $[N_{\text{token}}]$	Unique integer for each distinct sequence.
sym_id $[N_{\text{token}}]$	Unique integer within chains of this sequence. E.g. if chains A, B and C share a sequence but D does not, their sym_ids would be [0, 1, 2, 0].
restype $[N_{\text{token}}, 32]$	One-hot encoding of the sequence. 32 possible values: 20 amino acids + unknown, 4 RNA nucleotides + unknown, 4 DNA nucleotides + unknown, and gap. Ligands represented as “unknown amino acid”.
is_protein / rna / dna / ligand $[N_{\text{token}}]$	4 masks indicating the molecule type of a particular token.
ref_pos $[N_{\text{atom}}, 3]$	Atom positions in the reference conformer, with a random rotation and translation applied. Atom positions are given in Å.
ref_mask $[N_{\text{atom}}]$	Mask indicating which atom slots are used in the reference conformer.
ref_element $[N_{\text{atom}}, 128]$	One-hot encoding of the element atomic number for each atom in the reference conformer, up to atomic number 128.
ref_charge $[N_{\text{atom}}]$	Charge for each atom in the reference conformer.

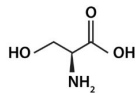
Feature & Shape	Description
ref_atom_name_chars $[N_{\text{atom}}, 4, 64]$	One-hot encoding of the unique atom names in the reference conformer. Each character is encoded as $\text{ord}(c) - 32$, and names are padded to length 4.
ref_space_uid $[N_{\text{atom}}]$	Numerical encoding of the chain id and residue index associated with this reference conformer. Each (chain id, residue index) tuple is assigned an integer on first appearance.
msa $[N_{\text{msa}}, N_{\text{token}}, 32]$	One-hot encoding of the processed MSA, using the same classes as restype.
has_deletion $[N_{\text{msa}}, N_{\text{token}}]$	Binary feature indicating if there is a deletion to the left of each position in the MSA.
deletion_value $[N_{\text{msa}}, N_{\text{token}}]$	Raw deletion counts (the number of deletions to the left of each MSA position) are transformed to $[0, 1]$ using $\frac{2}{\pi} \arctan \frac{d}{3}$.
profile $[N_{\text{token}}, 32]$	Distribution across restypes in the main MSA. Computed before MSA processing (subsection 2.3).
deletion_mean $[N_{\text{token}}]$	Mean number of deletions at each position in the main MSA. Computed before MSA processing (subsection 2.3).
template_restype $[N_{\text{templ}}, N_{\text{token}}]$	One-hot encoding of the template sequence, see restype.
template_pseudo_beta_mask $[N_{\text{templ}}, N_{\text{token}}]$	Mask indicating if the C^β (C^α for glycine) has coordinates for the template at this residue.
template_backbone_frame_mask $[N_{\text{templ}}, N_{\text{token}}]$	Mask indicating if coordinates exist for all atoms required to compute the backbone frame (used in the template_unit_vector feature).
template_distogram $[N_{\text{templ}}, N_{\text{token}}, N_{\text{token}}, 39]$	A one-hot pairwise feature indicating the distance between C^β atoms (C^α for glycine). Pairwise distances are discretized into 38 bins of equal width between 3.25 Å and 50.75 Å; one more bin contains any larger distances.
template_unit_vector $[N_{\text{templ}}, N_{\text{token}}, N_{\text{token}}, 3]$	The unit vector of the displacement of the C^α atom of all residues within the local frame of each residue. Local frames are computed as in [1].
token_bonds $[N_{\text{token}}, N_{\text{token}}]$	A 2D matrix indicating if there is a bond between any atom in token i and token j , restricted to just polymer-ligand and ligand-ligand bonds and bonds less than 2.4 Å during training.

Tokenization



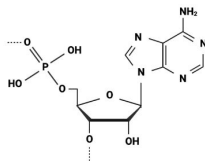
Tokenize input sequences

Standard Amino Acid
(serine)



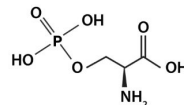
7 atoms*
1 tokens

Standard Nucleotide
(adenosine)



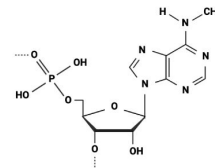
23 atoms*
1 token

Modified Amino Acid
(phosphoserine)



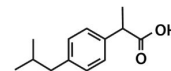
11 atoms*
11 tokens

Modified Nucleotide
(methyladenosine)



24 atoms*
24 tokens

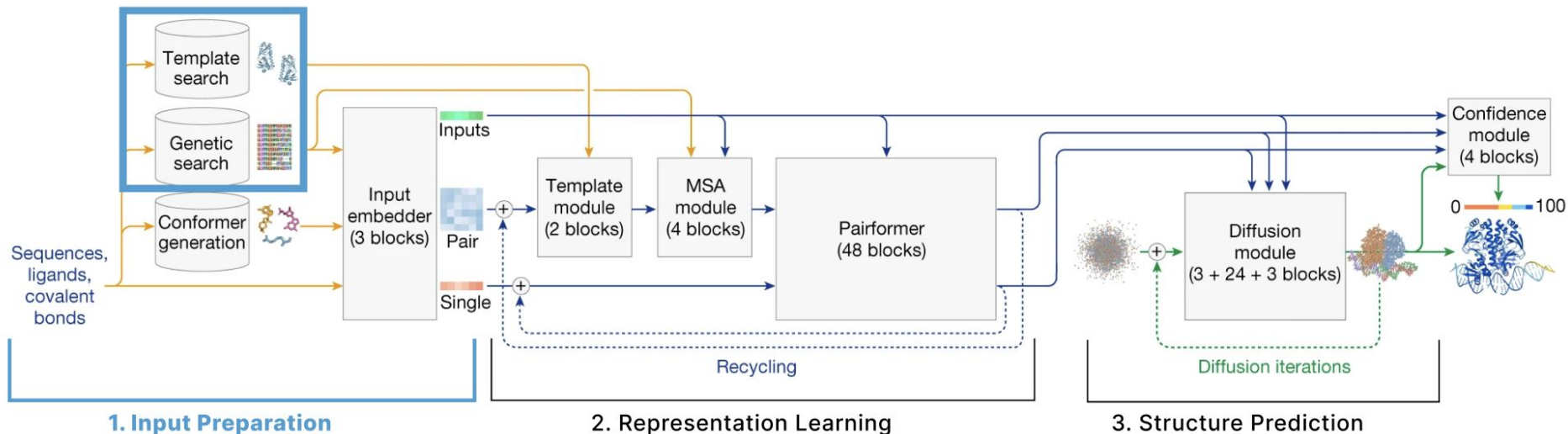
Ligand
(ibuprofen)



15 atoms*
15 tokens

*atoms=heavy atoms

Retrieval (Create MSA and Templates)



Tokenize input
sequences

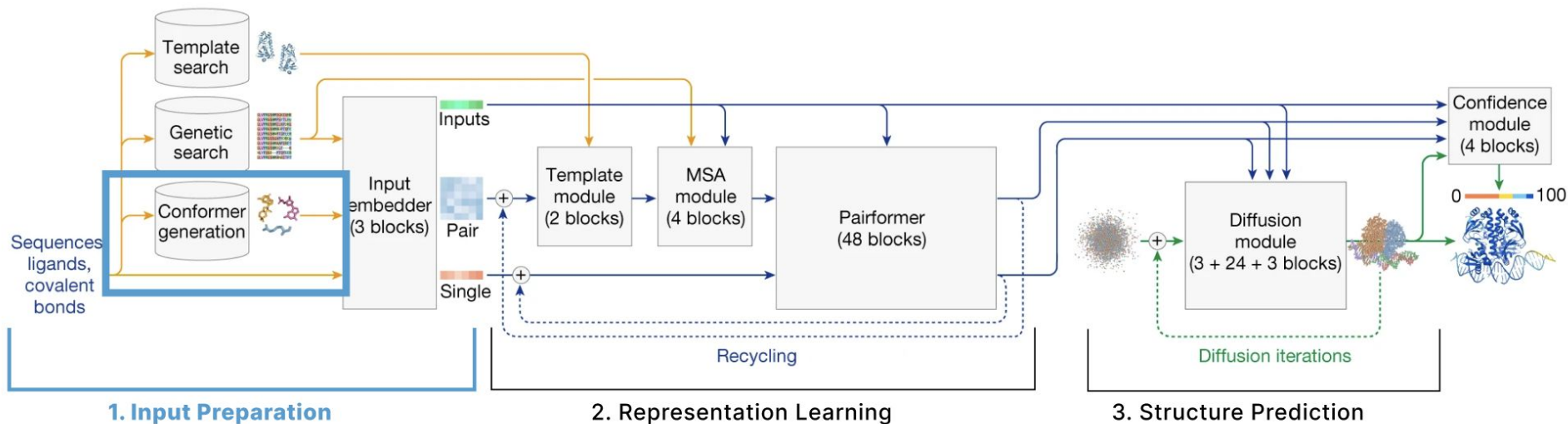
**Retrieve similar
sequences and
structures to create
MSA and templates**

Create atom-level
representation of
sequences

Update atom-level
representation (Atom
Transformer)

Aggregate atom-level
representation to
token-level

Create Atom-Level Representations



Tokenize input
sequences

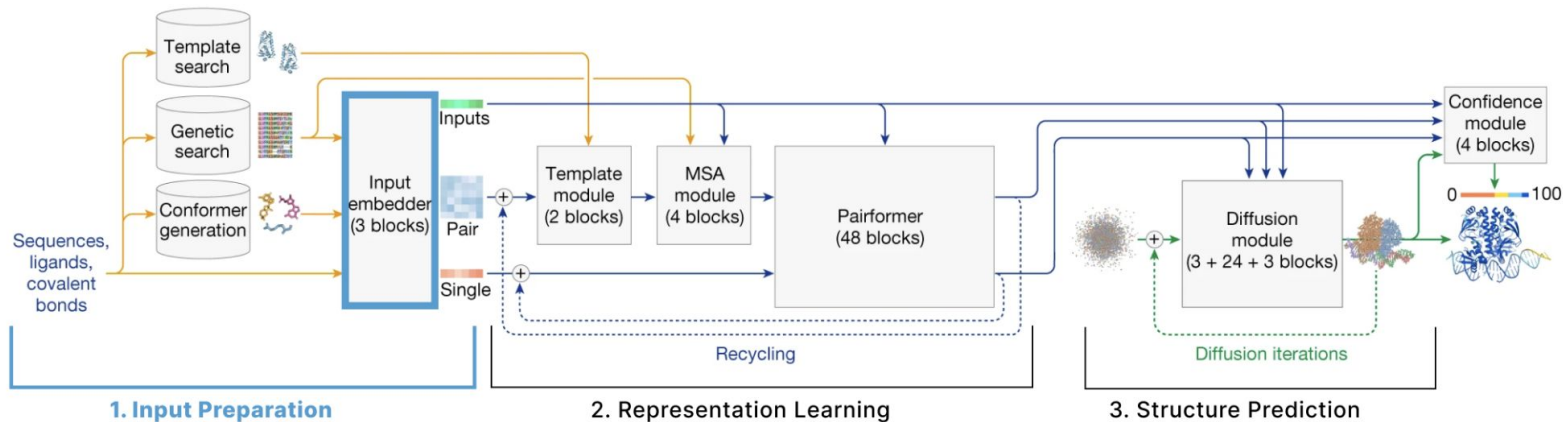
Retrieve similar
sequences and
structures to create
MSA and templates

Create atom-level
representation of
sequences

Update atom-level
representation (Atom
Transformer)

Aggregate atom-level
representation to
token-level

Update Atom-Level Representations (Atom Transformer)



Tokenize input
sequences

Retrieve similar
sequences and
structures to create
MSA and templates

Create atom-level
representation of
sequences

**Update atom-level
representation
(Atom Transformer)**

Aggregate atom-level
representation to
token-level

AtomTransformer introduces building blocks used elsewhere

* Adaptive LayerNorm

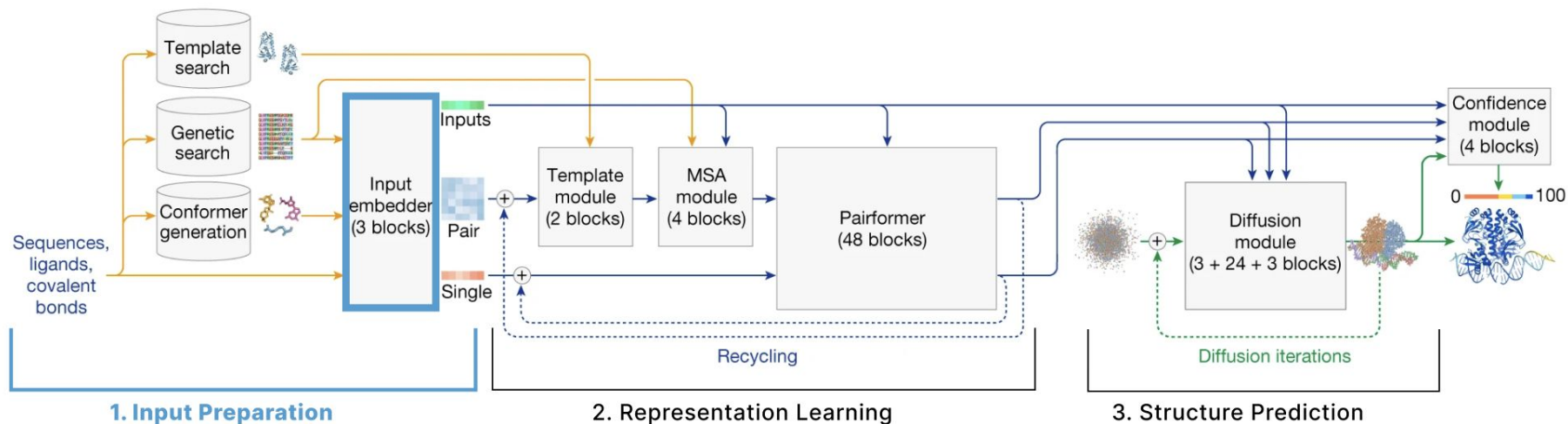
* Attention with Pair Bias

* Sequence-local atom attention

* Conditioned Gating

* Conditioned Transition blocks

Aggregate Atom-Level → Token-Level



Tokenize input
sequences

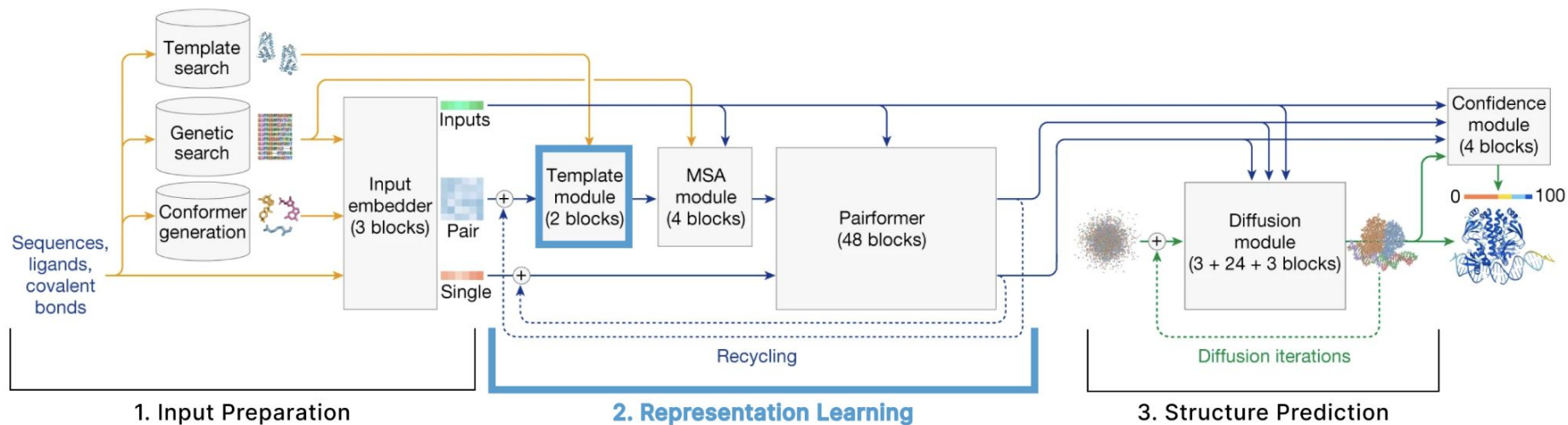
Retrieve similar
sequences and
structures to create
MSA and templates

Create atom-level
representation of
sequences

Update atom-level
representation (Atom
Transformer)

**Aggregate atom-
level representation
to token-level**

Template Module

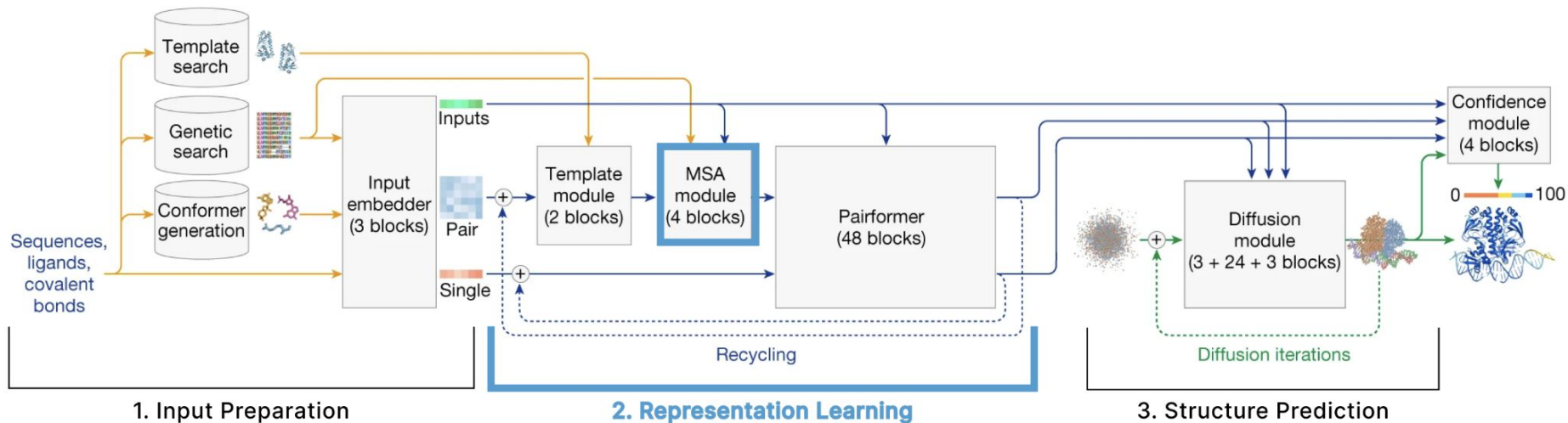


Template Module

MSA module

Pairformer

MSA Module



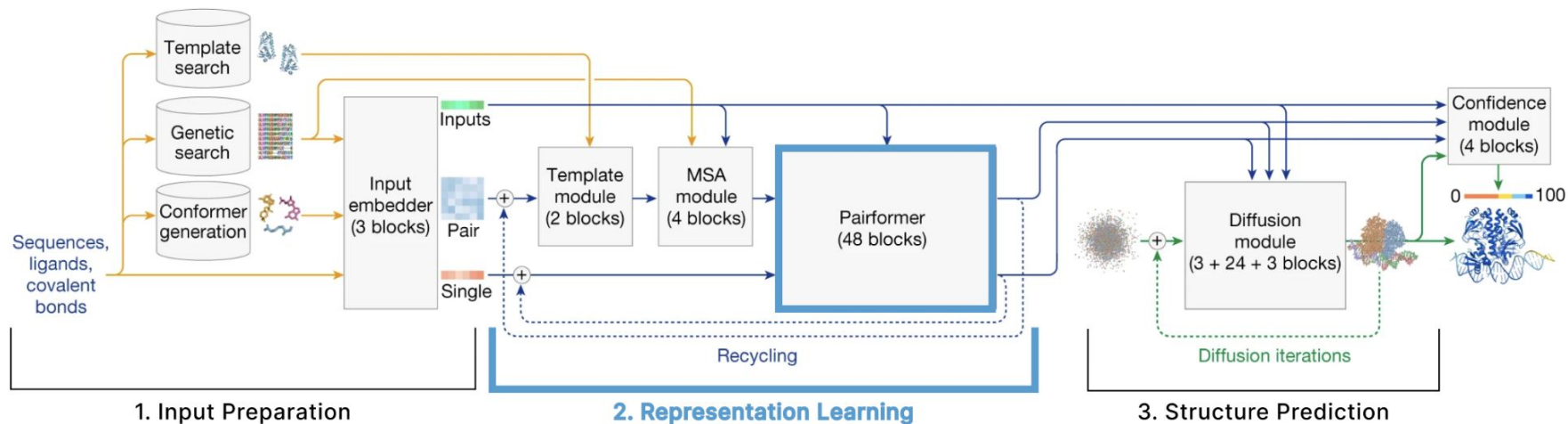
Template Module

MSA module

Pairformer

- Outer product mean
- Row-wise gated attention using only pair bias

Pairformer Module

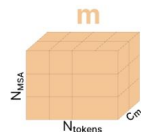


- Triangle Updates
- Triangle Attention
- Attention with Pair Bias (token-level)

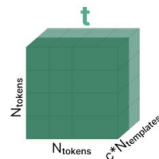
Aggregate Atom-Level → Token-Level

Information about related sequences and their structures

Multiple Sequence Alignment

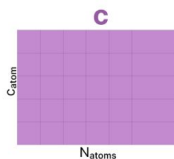


Structure Templates

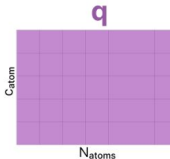


Information about all the atoms ("single")

Original Atom-Level Single Representation

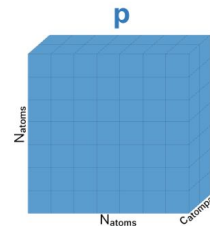


Updated Atom-Level Single Representation



Information about all the pairs of atoms ("pair")

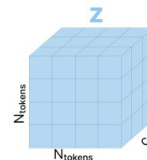
Atom-Level Pair Representation



Token-Level Single Representation



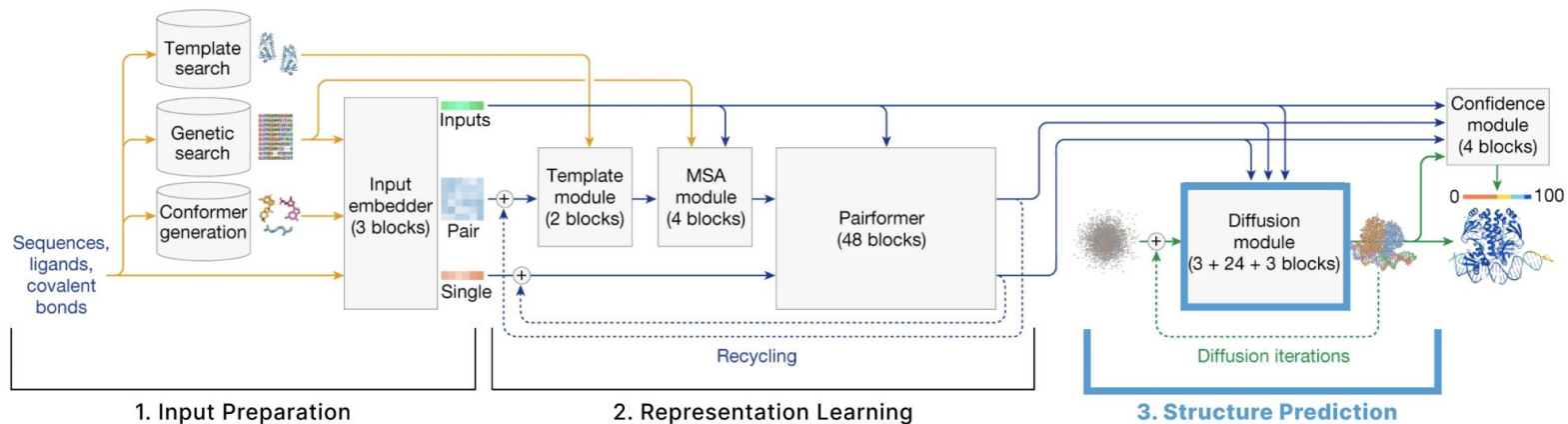
Token-Level Pair Representation



For Step 2, we will set aside the atom-level representations (**c**, **q**, **p**) and focus on updating our token-level representations **s** and **z** in the next section (with the help of **m** and **t**).

Let's Start Part 3

3. Structure Prediction

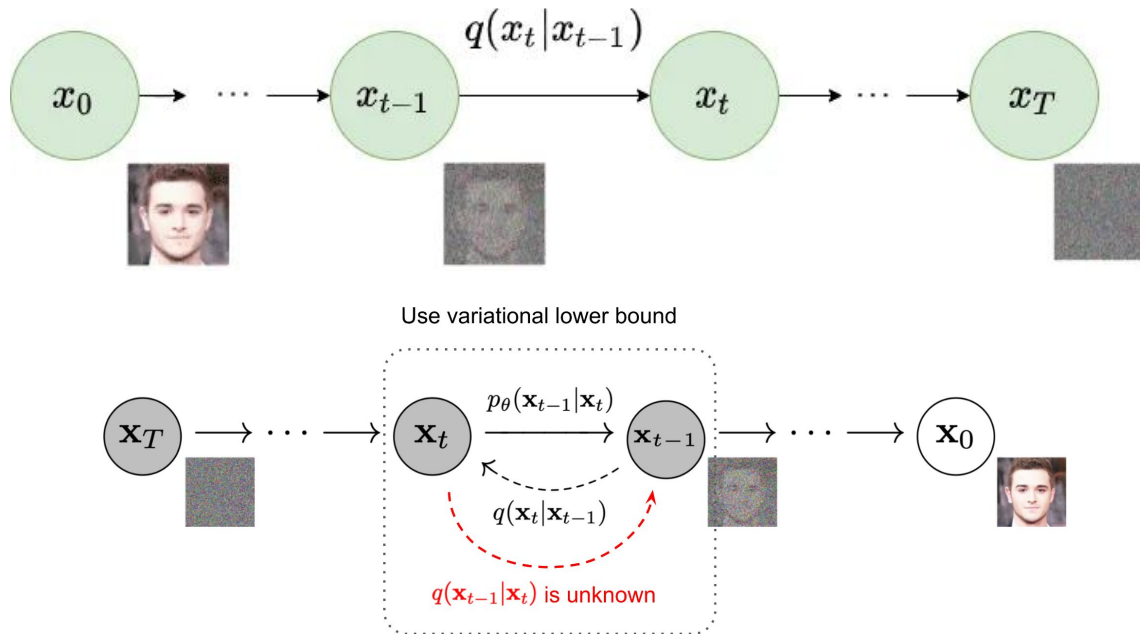


Basics of Diffusion

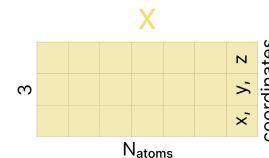
Diffusion Module

1. Prepare token-level conditioning tensors
2. Create atom-level conditioning tensors
3. Token-level attention biased by conditioning tensors
4. Apply attention at atom-level to predict atom-level noise updates

Basics of Diffusion



- **Diffusion model:** Adds noise to real data, then trains a model to reverse it step-by-step.
- **Training:** At each time step, the model predicts the noise added to atom coordinates.
- **Inference:** Starts from pure noise and iteratively removes predicted noise to recover structure.
- **Conditional diffusion:** Model conditions predictions on input info (e.g., protein properties).
- **AF3 input:** x,y,z coordinates of all atoms, with Gaussian noise added during training.
- **Data augmentation:** Random rotations/translations teach model rotational/positional invariance—replacing AF2's complex Invariant Point Attention.



Basics of Diffusion

1 Forward process (noising process):

- This process is **fixed** and **pre-defined**.
- We take clean data x_0 and gradually add noise to it across T steps according to a schedule.
- At each timestep t , a small amount of Gaussian noise is added:

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon$$

where:

- $\epsilon \sim \mathcal{N}(0, I)$ is standard Gaussian noise.
- β_t is the noise variance schedule at timestep t .
- $\alpha_t = 1 - \beta_t$, and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.
- **This forward process is not learned.** It is just a way of generating noisy training data.

2 Reverse process (denoising process):

- This is the part that we actually **train the model to learn**.
- The goal is to start from pure noise (x_T) and iteratively remove noise, reconstructing a clean sample x_0 .
- The model learns to predict the noise ϵ that was added at each step.

Summary Table

Process	What happens?	Is it learned?
Forward	$x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_T$ (add noise)	 No
Reverse	$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0$ (remove noise)	 Yes

🔧 What does the model learn?

At training time, the model receives a noisy sample x_t and the timestep t , and it tries to predict the noise that was originally added:

$$\hat{\epsilon}_\theta(x_t, t) \approx \epsilon$$

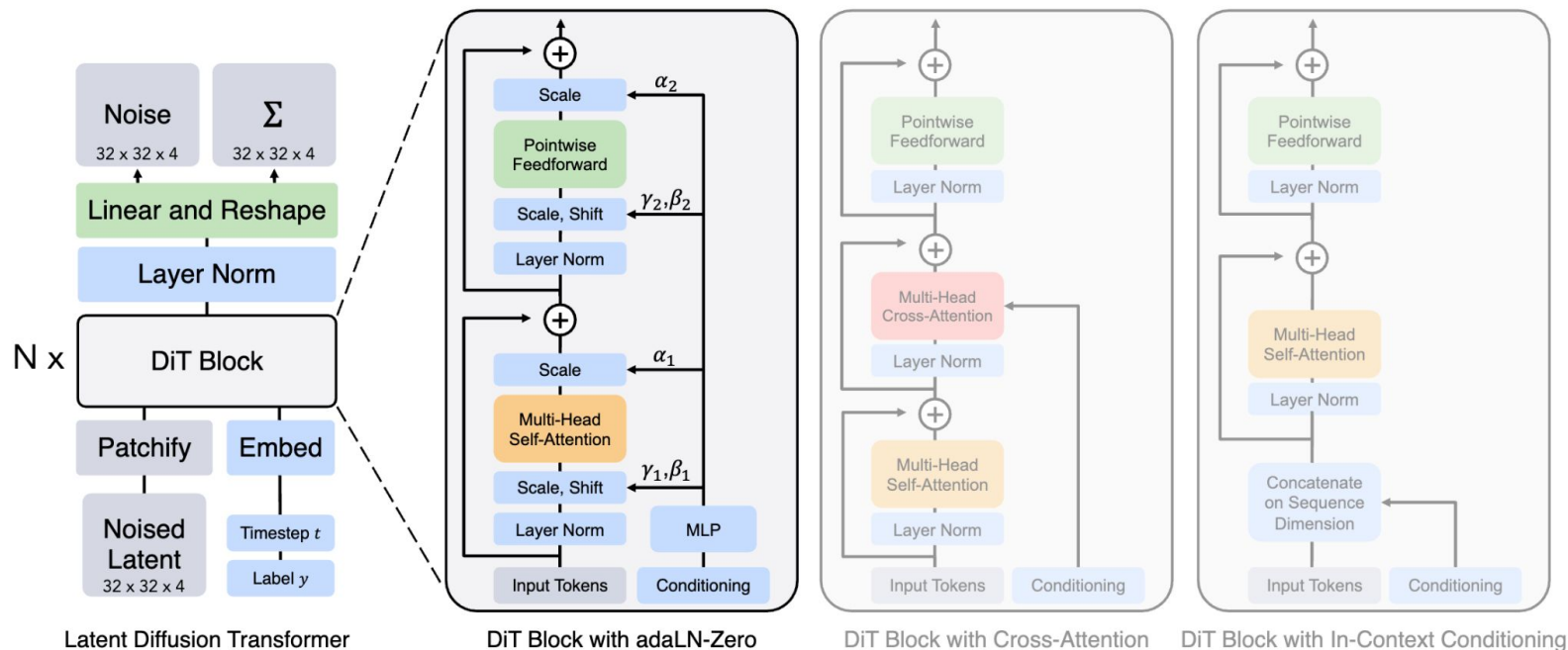
The loss function is usually a simple mean squared error (MSE):

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \hat{\epsilon}_\theta(x_t, t)\|^2]$$

So, in short:

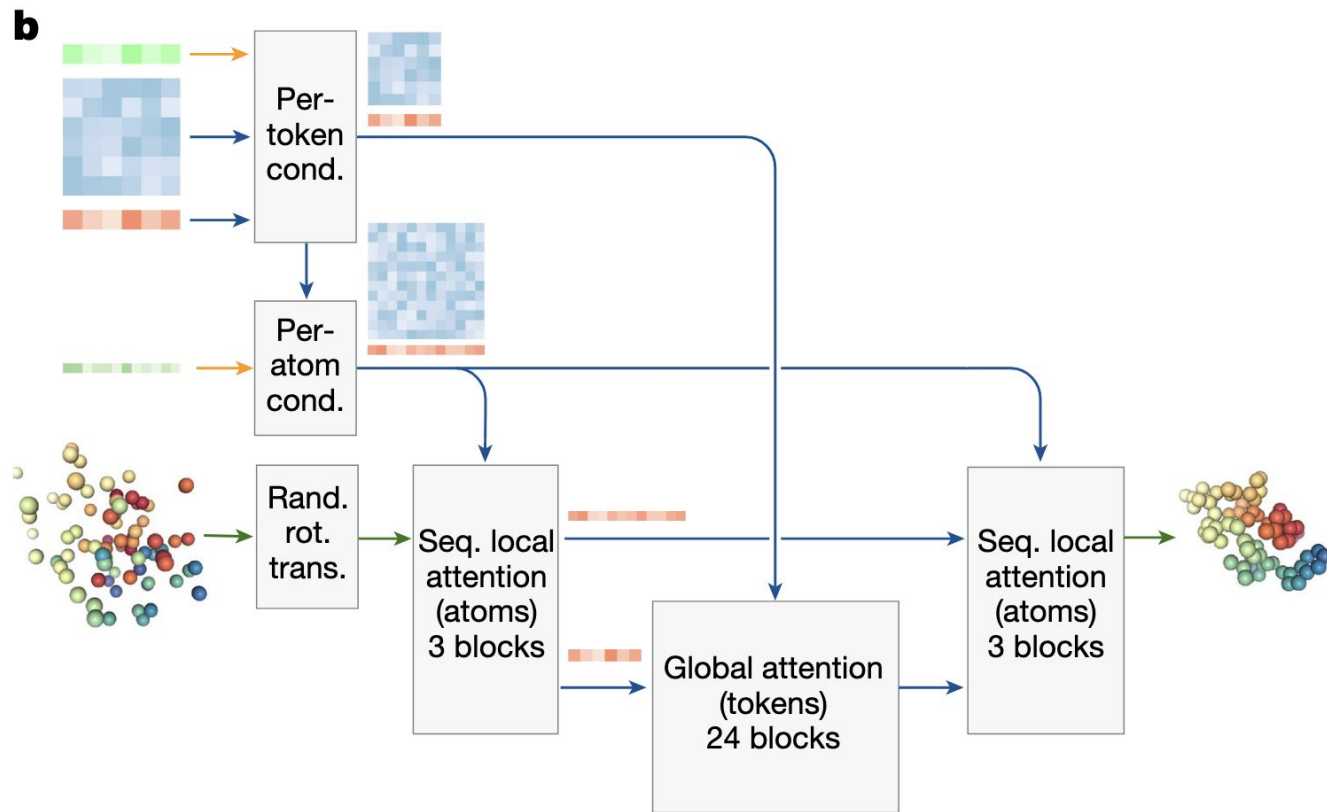
- The forward process generates training data.
- The reverse process is what we actually train.

Diffusion Transformer



<https://arxiv.org/pdf/2212.09748>

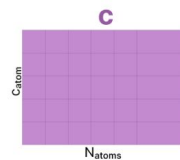
Diffusion Module



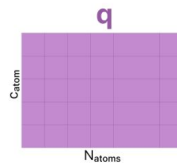
Atom-Level $\longleftrightarrow$ Token-Level

Information about all the atoms ("single")

Original Atom-Level Single Representation

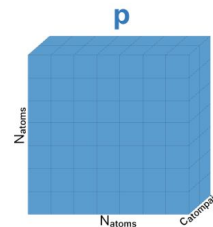


Positions of each atom

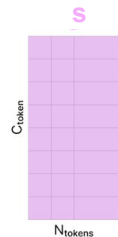


Information about all the pairs of atoms ("pair")

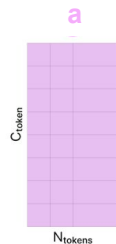
Atom-Level Pair Representation



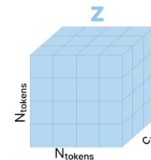
Token-level Single Representation



Positions of each Token



Token-Level Pair Representation



atom-level representations (**c**, **q**, **p**)

token-level representations (**s**, **a**, **z**)

Diffusion Module

Algorithm 18 Sample Diffusion

```

def SampleDiffusion( $\{\mathbf{f}^*\}, \{\mathbf{s}_i^{\text{inputs}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}^{\text{trunk}}\}, \text{Noise Schedule } [c_0, c_1, \dots, c_T],$ 
 $\gamma_0 = 0.8, \gamma_{\min} = 1.0, \text{noise scale } \lambda = 1.003, \text{step scale } \eta = 1.5$ ):
1:  $\tilde{\mathbf{x}}_t \sim c_0 \cdot \mathcal{N}(\vec{0}, \mathbf{I}_3)$ 
2: for all  $c_r \in [c_1, \dots, c_T]$  do
3:    $\{\tilde{\mathbf{x}}_i\} \leftarrow \text{CentreRandomAugmentation}(\{\tilde{\mathbf{x}}_i\})$ 
4:    $\gamma = \gamma_0$  if  $c_r > \gamma_{\min}$  else 0
5:    $\hat{t} = c_{r-1}(\gamma + 1)$ 
6:    $\tilde{\xi}_i = \lambda \sqrt{\hat{t}^2 - c_{r-1}^2} \cdot \mathcal{N}(\vec{0}, \mathbf{I}_3)$ 
7:    $\tilde{\mathbf{x}}_i^{\text{noisy}} = \tilde{\mathbf{x}}_i + \tilde{\xi}_i$ 
8:    $\{\tilde{\mathbf{x}}_i^{\text{denoised}}\} = \text{DiffusionModule}(\{\tilde{\mathbf{x}}_i^{\text{noisy}}\}, \hat{t}, \{\mathbf{f}^*\}, \{\mathbf{s}_i^{\text{inputs}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}^{\text{trunk}}\})$ 
9:    $\tilde{o}_i = (\tilde{\mathbf{x}}_i - \tilde{\mathbf{x}}_i^{\text{denoised}}) / \hat{t}$ 
10:   $dt = c_r - \hat{t}$ 
11:   $\tilde{\mathbf{x}}_i \leftarrow \tilde{\mathbf{x}}_i^{\text{noisy}} + \eta \cdot dt \cdot \tilde{o}_i$ 
12: end for
13: return  $\{\tilde{\mathbf{x}}_i\}$ 

```

Algorithm 20 Diffusion Module

```

def DiffusionModule( $\{\tilde{\mathbf{x}}_i^{\text{noisy}}\}, \hat{t}, \{\mathbf{f}^*\}, \{\mathbf{s}_i^{\text{inputs}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}^{\text{trunk}}\},$ 
 $\sigma_{\text{data}} = 16, c_{\text{atom}} = 128, c_{\text{atompair}} = 16, c_{\text{token}} = 768$ ):

```

$\tilde{\mathbf{x}}_i \in \mathbb{R}^3$

Conditioning

1: $\{\mathbf{s}_i\}, \{\mathbf{z}_{ij}\} = \text{DiffusionConditioning}(\hat{t}, \{\mathbf{f}^*\}, \{\mathbf{s}_i^{\text{inputs}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}^{\text{trunk}}\}, \sigma_{\text{data}})$

Scale positions to dimensionless vectors with approximately unit variance.

$\tilde{\mathbf{x}}_i \in \mathbb{R}^3$

2: $\mathbf{r}_i^{\text{noisy}} = \tilde{\mathbf{x}}_i^{\text{noisy}} / \sqrt{\hat{t}^2 + \sigma_{\text{data}}^2}$

$\mathbf{r}_i^{\text{noisy}} \in \mathbb{R}^3$

Sequence-local Atom Attention and aggregation to coarse-grained tokens

3: $\{\mathbf{a}_i\}, \{\mathbf{q}_i^{\text{skip}}\}, \{\mathbf{c}_i^{\text{skip}}\}, \{\mathbf{p}_{lm}^{\text{skip}}\} = \text{AtomAttentionEncoder}(\{\mathbf{f}^*\}, \{\mathbf{r}_i^{\text{noisy}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}^{\text{trunk}}\}, c_{\text{atom}}, c_{\text{atompair}}, c_{\text{token}})$
 $\mathbf{a}_i \in \mathbb{R}^{c_{\text{token}}}$

Full self-attention on token level.

4: $\mathbf{a}_i \leftarrow \text{LinearNoBias}(\text{LayerNorm}(\mathbf{s}_i))$

5: $\{\mathbf{a}_i\} \leftarrow \text{DiffusionTransformer}(\{\mathbf{a}_i\}, \{\mathbf{s}_i\}, \{\mathbf{z}_{ij}\}, \beta_{ij} = 0, N_{\text{block}} = 24, N_{\text{head}} = 16)$

6: $\mathbf{a}_i \leftarrow \text{LayerNorm}(\mathbf{a}_i)$

Broadcast token activations to atoms and run Sequence-local Atom Attention

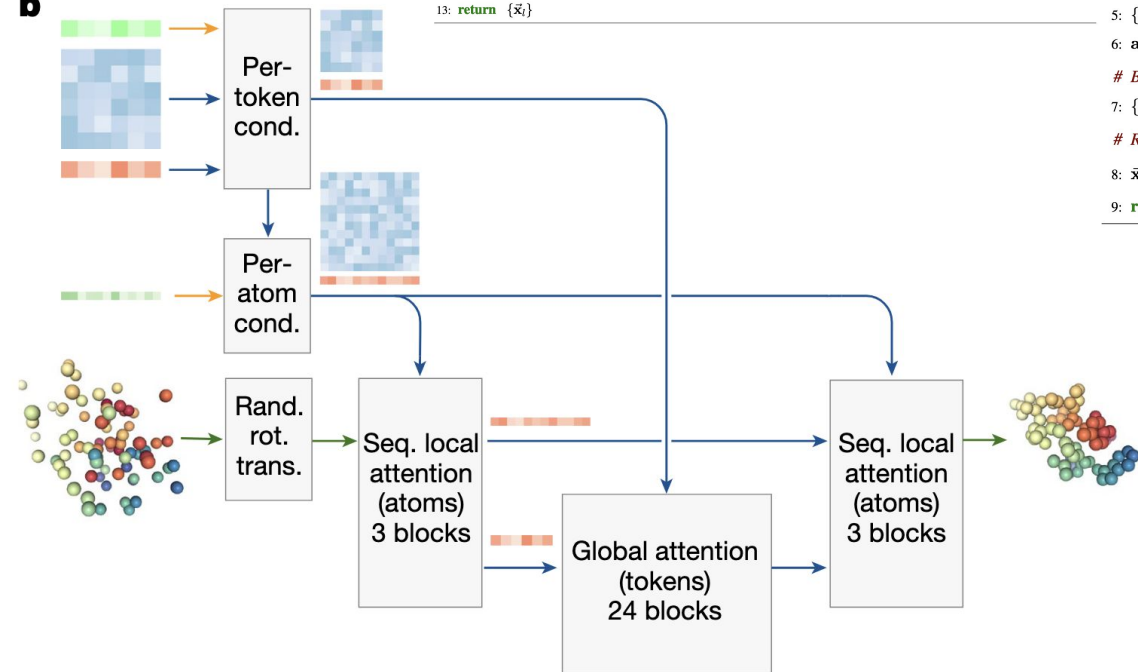
7: $\{\mathbf{r}_i^{\text{update}}\} = \text{AtomAttentionDecoder}(\{\mathbf{a}_i\}, \{\mathbf{q}_i^{\text{skip}}\}, \{\mathbf{c}_i^{\text{skip}}\}, \{\mathbf{p}_{lm}^{\text{skip}}\})$

Rescale updates to positions and combine with input positions

8: $\tilde{\mathbf{x}}_i^{\text{out}} = \sigma_{\text{data}}^2 / (\sigma_{\text{data}}^2 + \hat{t}^2) \cdot \tilde{\mathbf{x}}_i^{\text{noisy}} + \sigma_{\text{data}} \cdot \hat{t} / \sqrt{\sigma_{\text{data}}^2 + \hat{t}^2} \cdot \mathbf{r}_i^{\text{update}}$

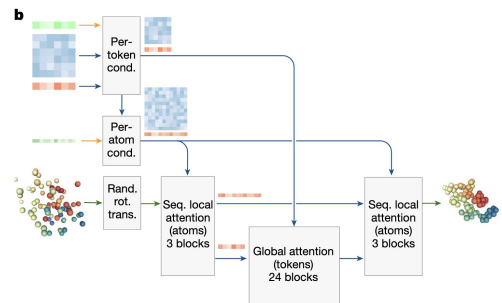
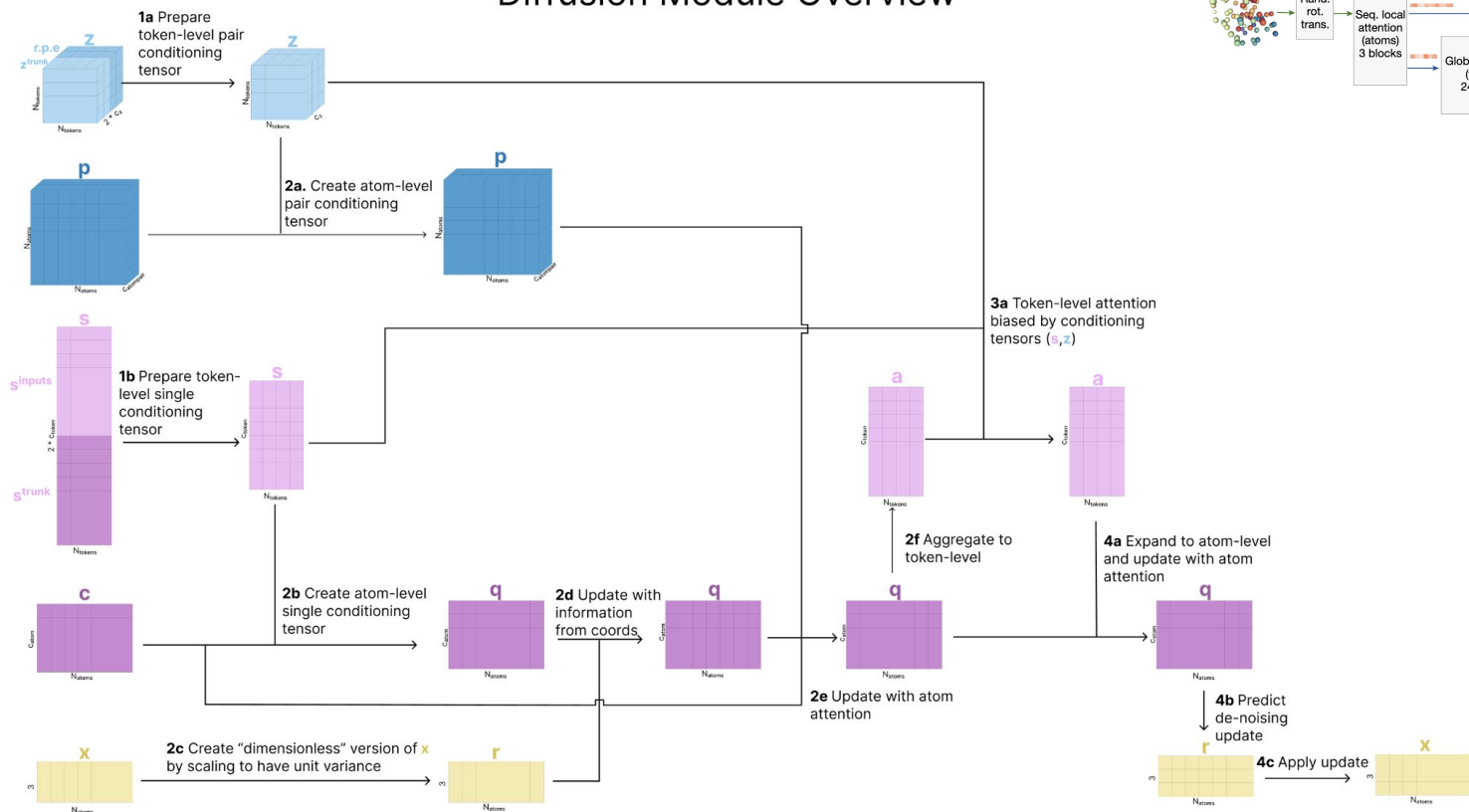
9: **return** $\{\tilde{\mathbf{x}}_i^{\text{out}}\}$

b



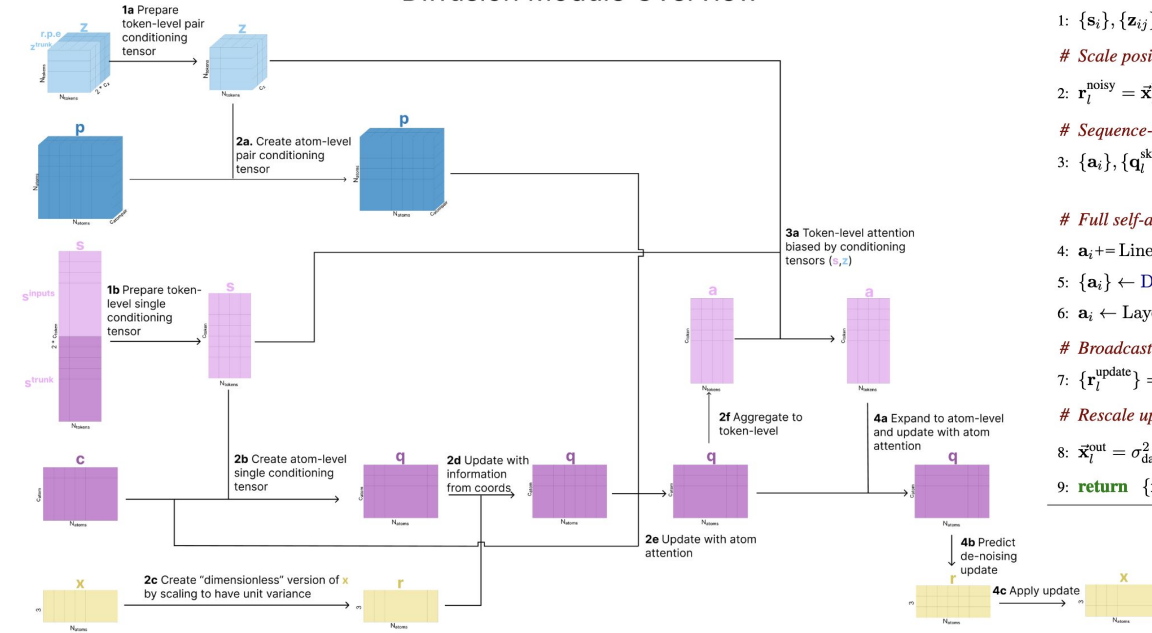
Diffusion Module

Diffusion Module Overview



Diffusion Module

Diffusion Module Overview



The AF3 paper breaks down its diffusion process into 4 steps that involve moving from tokens to atoms, back to tokens, and back to atoms:

1. Prepare token-level conditioning tensors
2. Prepare atom-level conditioning tensors, update them using the Atom Transformer, and aggregate them back to token-level
3. Apply attention at the token-level, and project back to atoms
4. Apply attention at the atom-level to predict atom-level noise updates

Algorithm 20 Diffusion Module

```
def DiffusionModule( $\{\mathbf{x}_l^{\text{noisy}}\}, \hat{t}, \{\mathbf{f}^*\}, \{\mathbf{s}_i^{\text{inputs}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}^{\text{trunk}}\},$   

 $\sigma_{\text{data}} = 16, c_{\text{atom}} = 128, c_{\text{atompair}} = 16, c_{\text{token}} = 768$ ) :
```

Conditioning

1: $\{\mathbf{s}_i\}, \{\mathbf{z}_{ij}\} = \text{DiffusionConditioning}(\hat{t}, \{\mathbf{f}^*\}, \{\mathbf{s}_i^{\text{inputs}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}^{\text{trunk}}\}, \sigma_{\text{data}})$

Scale positions to dimensionless vectors with approximately unit variance.

2: $\mathbf{r}_l^{\text{noisy}} = \mathbf{x}_l^{\text{noisy}} / \sqrt{\hat{t}^2 + \sigma_{\text{data}}^2}$ $\mathbf{r}_l^{\text{noisy}} \in \mathbb{R}^3$

Sequence-local Atom Attention and aggregation to coarse-grained tokens

3: $\{\mathbf{a}_i\}, \{\mathbf{q}_l^{\text{skip}}\}, \{\mathbf{c}_l^{\text{skip}}\}, \{\mathbf{p}_{lm}^{\text{skip}}\} = \text{AtomAttentionEncoder}(\{\mathbf{f}^*\}, \{\mathbf{r}_l^{\text{noisy}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}\}, c_{\text{atom}}, c_{\text{atompair}}, c_{\text{token}})$
 $\mathbf{a}_i \in \mathbb{R}^{c_{\text{token}}}$

Full self-attention on token level.

4: $\mathbf{a}_i += \text{LinearNoBias}(\text{LayerNorm}(\mathbf{s}_i))$

5: $\{\mathbf{a}_i\} \leftarrow \text{DiffusionTransformer}(\{\mathbf{a}_i\}, \{\mathbf{s}_i\}, \{\mathbf{z}_{ij}\}, \beta_{ij} = 0, N_{\text{block}} = 24, N_{\text{head}} = 16)$

6: $\mathbf{a}_i \leftarrow \text{LayerNorm}(\mathbf{a}_i)$

Broadcast token activations to atoms and run Sequence-local Atom Attention

7: $\{\mathbf{r}_l^{\text{update}}\} = \text{AtomAttentionDecoder}(\{\mathbf{a}_i\}, \{\mathbf{q}_l^{\text{skip}}\}, \{\mathbf{c}_l^{\text{skip}}\}, \{\mathbf{p}_{lm}^{\text{skip}}\})$

Rescale updates to positions and combine with input positions

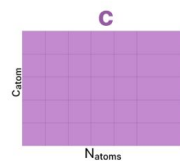
8: $\mathbf{x}_l^{\text{out}} = \sigma_{\text{data}}^2 / (\sigma_{\text{data}}^2 + \hat{t}^2) \cdot \mathbf{x}_l^{\text{noisy}} + \sigma_{\text{data}} \cdot \hat{t} / \sqrt{\sigma_{\text{data}}^2 + \hat{t}^2} \cdot \mathbf{r}_l^{\text{update}}$

9: return $\{\mathbf{x}_l^{\text{out}}\}$

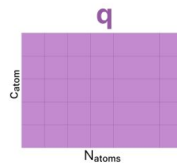
Atom-Level $\longleftrightarrow$ Token-Level

Information about all the atoms ("single")

Original Atom-Level Single Representation

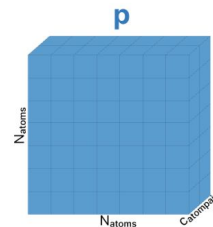


Positions of each atom

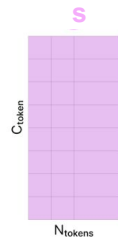


Information about all the pairs of atoms ("pair")

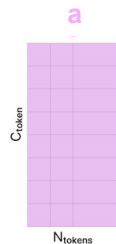
Atom-Level Pair Representation



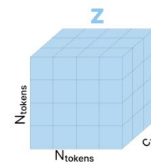
Token-level Single Representation



Positions of each Token



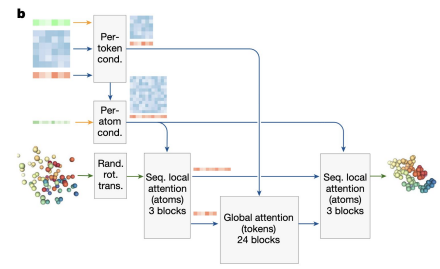
Token-Level Pair Representation



atom-level representations (**c**, **q**, **p**)

token-level representations (**s**, **a**, **z**)

Diffusion Module - Diffusion Conditioning



Algorithm 20 Diffusion Module

```
def DiffusionModule( $\{\mathbf{x}_l^{\text{noisy}}\}, \hat{t}, \{\mathbf{f}^*\}, \{\mathbf{s}_i^{\text{inputs}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}^{\text{trunk}}\},$   

 $\sigma_{\text{data}} = 16, c_{\text{atom}} = 128, c_{\text{atompair}} = 16, c_{\text{token}} = 768$ ) :
```

Conditioning

1: $\{\mathbf{s}_i\}, \{\mathbf{z}_{ij}\} = \text{DiffusionConditioning}(\hat{t}, \{\mathbf{f}^*\}, \{\mathbf{s}_i^{\text{inputs}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}^{\text{trunk}}\}, \sigma_{\text{data}})$

Scale positions to dimensionless vectors with approximately unit variance.

2: $\mathbf{r}_l^{\text{noisy}} = \mathbf{x}_l^{\text{noisy}} / \sqrt{\hat{t}^2 + \sigma_{\text{data}}^2}$ $\mathbf{r}_l^{\text{noisy}} \in \mathbb{R}^3$

Sequence-local Atom Attention and aggregation to coarse-grained tokens

3: $\{\mathbf{a}_i\}, \{\mathbf{q}_i^{\text{skip}}\}, \{\mathbf{c}_i^{\text{skip}}\}, \{\mathbf{p}_{lm}^{\text{skip}}\} = \text{AtomAttentionEncoder}(\{\mathbf{f}^*\}, \{\mathbf{r}_l^{\text{noisy}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}\}, c_{\text{atom}}, c_{\text{atompair}}, c_{\text{token}})$
 $\mathbf{a}_i \in \mathbb{R}^{c_{\text{token}}}$

Full self-attention on token level.

4: $\mathbf{a}_i += \text{LinearNoBias}(\text{LayerNorm}(\mathbf{s}_i))$

5: $\{\mathbf{a}_i\} \leftarrow \text{DiffusionTransformer}(\{\mathbf{a}_i\}, \{\mathbf{s}_i\}, \{\mathbf{z}_{ij}\}, \beta_{ij} = 0, N_{\text{block}} = 24, N_{\text{head}} = 16)$

6: $\mathbf{a}_i \leftarrow \text{LayerNorm}(\mathbf{a}_i)$

Broadcast token activations to atoms and run Sequence-local Atom Attention

7: $\{\mathbf{r}_l^{\text{update}}\} = \text{AtomAttentionDecoder}(\{\mathbf{a}_i\}, \{\mathbf{q}_i^{\text{skip}}\}, \{\mathbf{c}_i^{\text{skip}}\}, \{\mathbf{p}_{lm}^{\text{skip}}\})$

Rescale updates to positions and combine with input positions

8: $\mathbf{x}_l^{\text{out}} = \sigma_{\text{data}}^2 / (\sigma_{\text{data}}^2 + \hat{t}^2) \cdot \mathbf{x}_l^{\text{noisy}} + \sigma_{\text{data}} \cdot \hat{t} / \sqrt{\sigma_{\text{data}}^2 + \hat{t}^2} \cdot \mathbf{r}_l^{\text{update}}$

9: **return** $\{\mathbf{x}_l^{\text{out}}\}$

Algorithm 21 Diffusion Conditioning

```
def DiffusionConditioning( $\hat{t}, \{\mathbf{f}^*\}, \{\mathbf{s}_i^{\text{inputs}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}^{\text{trunk}}\}, \sigma_{\text{data}}, c_z = 128, c_s = 384$ ) :
```

Pair conditioning

1: $\mathbf{z}_{ij} = \text{concat}([\mathbf{z}_{ij}^{\text{trunk}}, \text{RelativePositionEncoding}(\{\mathbf{f}^*\})])$

2: $\mathbf{z}_{ij} \leftarrow \text{LinearNoBias}(\text{LayerNorm}(\mathbf{z}_{ij}))$ $\mathbf{z}_{ij} \in \mathbb{R}^{c_z}$

3: **for all** $b \in [1, 2]$ **do**

4: $\mathbf{z}_{ij} += \text{Transition}(\mathbf{z}_{ij}, n = 2)$

5: **end for**

Single conditioning

6: $\mathbf{s}_i = \text{concat}([\mathbf{s}_i^{\text{trunk}}, \mathbf{s}_i^{\text{inputs}}])$

7: $\mathbf{s}_i \leftarrow \text{LinearNoBias}(\text{LayerNorm}(\mathbf{s}_i))$ $\mathbf{s}_i \in \mathbb{R}^{c_s}$

8: $\mathbf{n} = \text{FourierEmbedding}(\frac{1}{4} \log(\hat{t} / \sigma_{\text{data}}), 256)$

9: $\mathbf{s}_i += \text{LinearNoBias}(\text{LayerNorm}(\mathbf{n}))$

10: **for all** $b \in [1, 2]$ **do**

11: $\mathbf{s}_i += \text{Transition}(\mathbf{s}_i, n = 2)$

12: **end for**

13: **return** $\{\mathbf{s}_i\}, \{\mathbf{z}_{ij}\}$

Diffusion Module

Algorithm 20 Diffusion Module

def DiffusionModule($\{\bar{\mathbf{x}}_l^{\text{noisy}}\}, \hat{t}, \{\mathbf{f}^*\}, \{\mathbf{s}_i^{\text{inputs}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}^{\text{trunk}}\}, \sigma_{\text{data}} = 16, c_{\text{atom}} = 128, c_{\text{atompair}} = 16, c_{\text{token}} = 768$) :

Conditioning

1: $\{\mathbf{s}_i\}, \{\mathbf{z}_{ij}\} = \text{DiffusionConditioning}(\hat{t}, \{\mathbf{f}^*\}, \{\mathbf{s}_i^{\text{inputs}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}^{\text{trunk}}\}, \sigma_{\text{data}})$

Scale positions to dimensionless vectors with approximately unit variance.

2: $\mathbf{r}_l^{\text{noisy}} = \bar{\mathbf{x}}_l^{\text{noisy}} / \sqrt{\hat{t}^2 + \sigma_{\text{data}}^2}$ $\mathbf{r}_l^{\text{noisy}} \in \mathbb{R}^3$

Sequence-local Atom Attention and aggregation to coarse-grained tokens

3: $\{\mathbf{a}_i\}, \{\mathbf{q}_l^{\text{skip}}\}, \{\mathbf{c}_l^{\text{skip}}\}, \{\mathbf{p}_{lm}^{\text{skip}}\} = \text{AtomAttentionEncoder}(\{\mathbf{f}^*\}, \{\mathbf{r}_l^{\text{noisy}}\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}\}, c_{\text{atom}}, c_{\text{atompair}}, c_{\text{token}})$ $\mathbf{a}_i \in \mathbb{R}^{c_{\text{token}}}$

Full self-attention on token level.

4: $\mathbf{a}_i += \text{LinearNoBias}(\text{LayerNorm}(\mathbf{s}_i))$

5: $\{\mathbf{a}_i\} \leftarrow \text{DiffusionTransformer}(\{\mathbf{a}_i\}, \{\mathbf{s}_i\}, \{\mathbf{z}_{ij}\}, \beta_{ij} = 0, N_{\text{block}} = 24, N_{\text{head}} = 16)$

6: $\mathbf{a}_i \leftarrow \text{LayerNorm}(\mathbf{a}_i)$

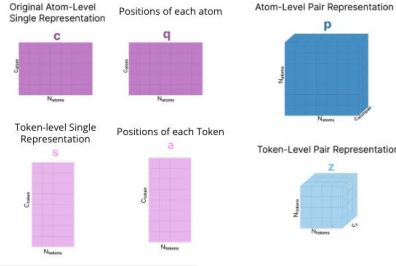
Broadcast token activations to atoms and run Sequence-local Atom Attention

7: $\{\mathbf{r}_l^{\text{update}}\} = \text{AtomAttentionDecoder}(\{\mathbf{a}_i\}, \{\mathbf{q}_l^{\text{skip}}\}, \{\mathbf{c}_l^{\text{skip}}\}, \{\mathbf{p}_{lm}^{\text{skip}}\})$

Rescale updates to positions and combine with input positions

8: $\bar{\mathbf{x}}_l^{\text{out}} = \sigma_{\text{data}}^2 / (\sigma_{\text{data}}^2 + \hat{t}^2) \cdot \mathbf{r}_l^{\text{noisy}} + \sigma_{\text{data}} \cdot \hat{t} / \sqrt{\sigma_{\text{data}}^2 + \hat{t}^2} \cdot \mathbf{r}_l^{\text{update}}$

9: **return** $\{\bar{\mathbf{x}}_l^{\text{out}}\}$



Algorithm 7 Atom Transformer

def AtomTransformer($\{\mathbf{q}_l\}, \{\mathbf{c}_l\}, \{\mathbf{p}_{lm}\}, N_{\text{block}} = 3, N_{\text{head}}, N_{\text{queries}} = 32, N_{\text{keys}} = 128, \mathcal{S}_{\text{subset centres}} = \{15.5, 47.5, 79.5, \dots\}$) :

sequence-local atom attention is equivalent to self attention within rectangular blocks along the diagonal.

1: $\beta_{lm} = \begin{cases} 0 & \text{if } |l - c| < N_{\text{queries}}/2 \wedge |m - c| < N_{\text{keys}}/2 \quad \forall c \in \mathcal{S}_{\text{subset centres}} \\ -10^{10} & \text{else} \end{cases}$

2: $\{\mathbf{q}_l\} = \text{DiffusionTransformer}(\{\mathbf{q}_l\}, \{\mathbf{c}_l\}, \{\mathbf{p}_{lm}\}, \{\beta_{lm}\}, N_{\text{block}}, N_{\text{head}})$

3: **return** $\{\mathbf{q}_l\}$

Algorithm 5 Atom attention encoder

def AtomAttentionEncoder($\{\mathbf{f}^*\}, \{\mathbf{r}_l\}, \{\mathbf{s}_i^{\text{trunk}}\}, \{\mathbf{z}_{ij}\}, c_{\text{atom}}, c_{\text{atompair}}, c_{\text{token}})$:

Create the atom single conditioning: Embed per-atom meta data

1: $\mathbf{c}_l = \text{LinearNoBias}(\text{concat}(\hat{\mathbf{f}}_l^{\text{ref_pos}}, \hat{\mathbf{f}}_l^{\text{ref_charge}}, \hat{\mathbf{f}}_l^{\text{ref_mask}}, \hat{\mathbf{f}}_l^{\text{ref_element}}, \hat{\mathbf{f}}_l^{\text{ref_atom_name_chars}}))$ $\mathbf{c}_l \in \mathbb{R}^{c_{\text{atom}}}$
 $l \in \{1, \dots, N_{\text{atoms}}\}$

Cross attention transformer.

15: $\{\mathbf{q}_l\} = \text{AtomTransformer}(\{\mathbf{q}_l\}, \{\mathbf{c}_l\}, \{\mathbf{p}_{lm}\}, N_{\text{block}} = 3, N_{\text{head}} = 4)$

Aggregate per-atom representation to per-token representation

16: $\mathbf{a}_i = \frac{1}{N_{\text{atoms}}} \sum_{l \in \{1, \dots, N_{\text{atoms}}\}} \text{mean}_{\text{tok_idx}(l)=i}(\text{relu}(\text{LinearNoBias}(\mathbf{q}_l)))$ $\mathbf{a}_i \in \mathbb{R}^{c_{\text{token}}}$

17: $\mathbf{q}_l^{\text{skip}}, \mathbf{c}_l^{\text{skip}}, \mathbf{p}_{lm}^{\text{skip}} = \mathbf{q}_l, \mathbf{c}_l, \mathbf{p}_{lm}$

18: **return** $\{\mathbf{a}_i\}, \{\mathbf{q}_l^{\text{skip}}\}, \{\mathbf{c}_l^{\text{skip}}\}, \{\mathbf{p}_{lm}^{\text{skip}}\}$

Algorithm 6 Atom attention decoder

def AtomAttentionDecoder($\{\mathbf{a}_i\}, \{\mathbf{q}_l^{\text{skip}}\}, \{\mathbf{c}_l^{\text{skip}}\}, \{\mathbf{p}_{lm}^{\text{skip}}\}$) :

Broadcast per-token activations to per-atom activations and add the skip connection

1: $\mathbf{q}_l = \text{LinearNoBias}(\mathbf{a}_{\text{tok_idx}(l)}) + \mathbf{q}_l^{\text{skip}}$

Cross attention transformer.

2: $\{\mathbf{q}_l\} = \text{AtomTransformer}(\{\mathbf{q}_l\}, \{\mathbf{c}_l^{\text{skip}}\}, \{\mathbf{p}_{lm}^{\text{skip}}\}, N_{\text{block}} = 3, N_{\text{head}} = 4)$

Map to positions update.

3: $\mathbf{r}_l^{\text{update}} = \text{LinearNoBias}(\text{LayerNorm}(\mathbf{q}_l))$

4: **return** $\{\mathbf{r}_l^{\text{update}}\}$

Algorithm 23 Diffusion Transformer

def DiffusionTransformer($\{\mathbf{a}_i\}, \{\mathbf{s}_i\}, \{\mathbf{z}_{ij}\}, \{\beta_{ij}\}, N_{\text{block}}, N_{\text{head}})$:

1: **for all** $n \in [1, \dots, N_{\text{block}}]$ **do**

2: $\{\mathbf{b}_i\} = \text{AttentionPairBias}(\{\mathbf{a}_i\}, \{\mathbf{s}_i\}, \{\mathbf{z}_{ij}\}, \{\beta_{ij}\}, N_{\text{head}})$

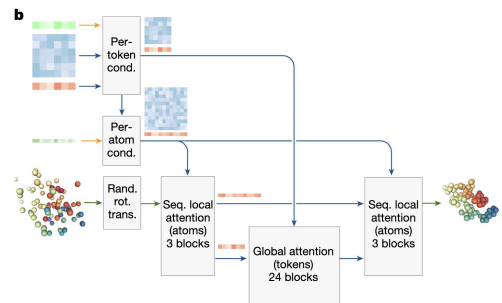
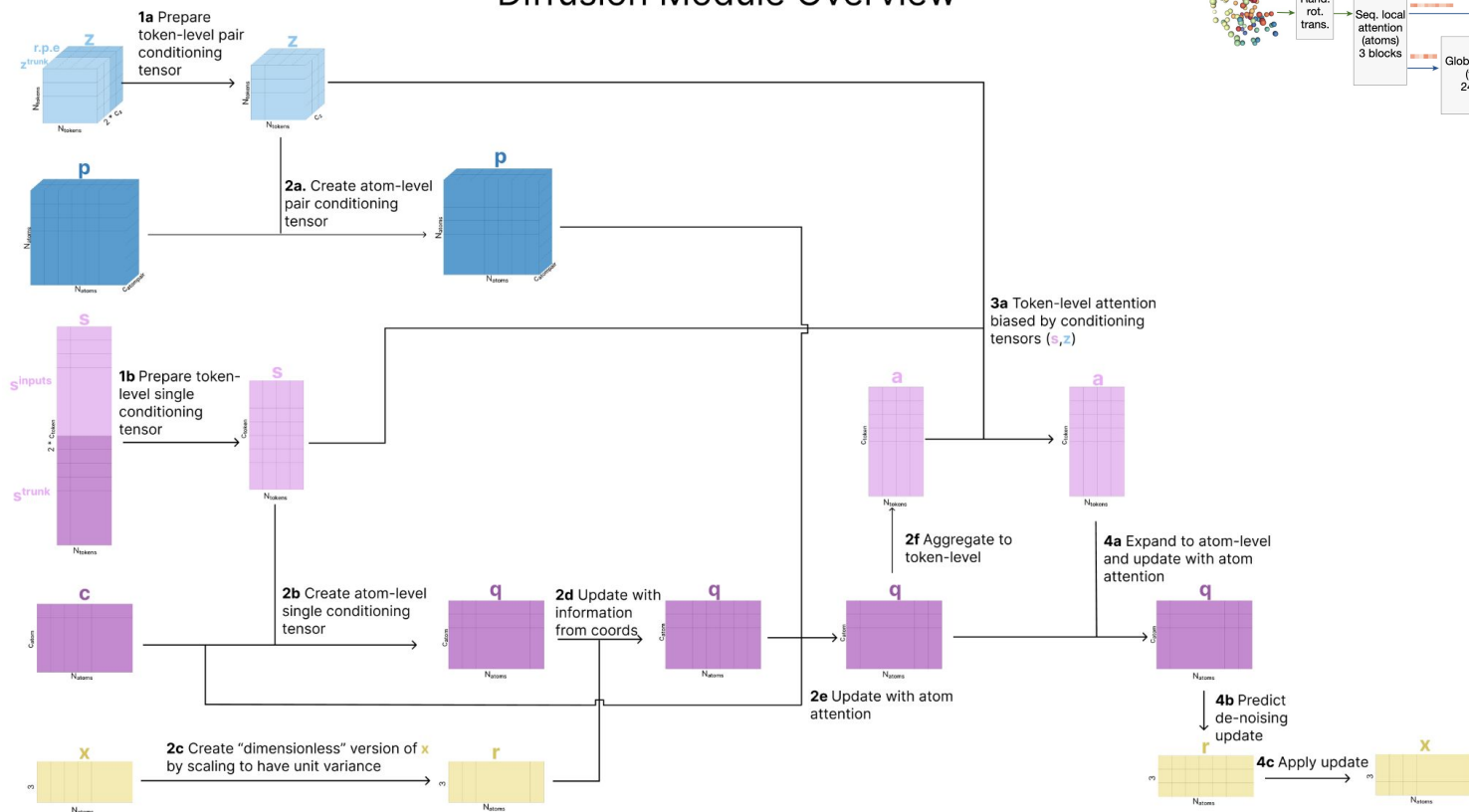
3: $\mathbf{a}_i \leftarrow \mathbf{b}_i + \text{ConditionedTransitionBlock}(\mathbf{a}_i, \mathbf{s}_i)$

4: **end for**

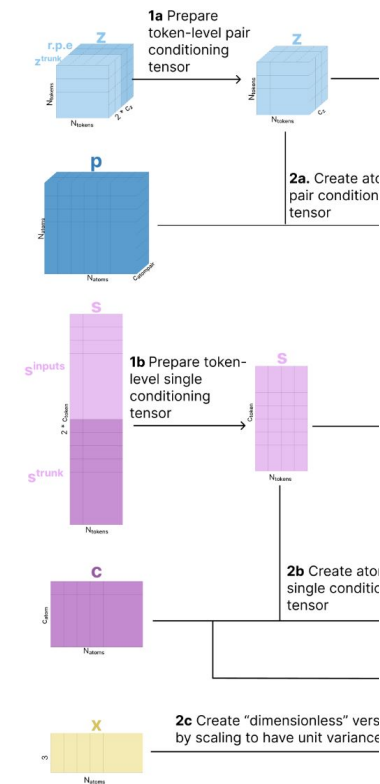
5: **return** $\{\mathbf{a}_i\}$

Diffusion Module

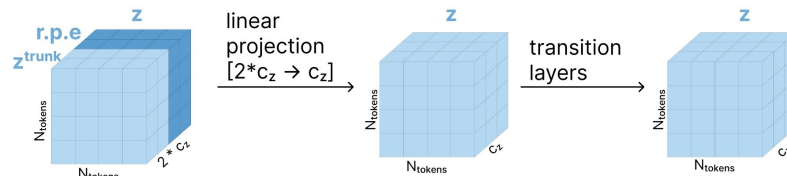
Diffusion Module Overview



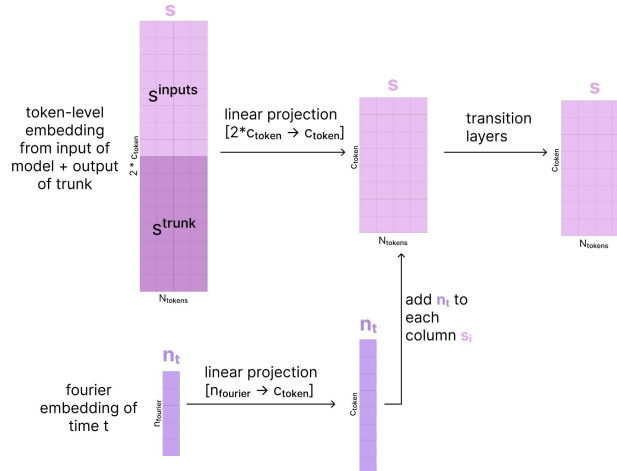
Diffusion Module - Part 1



1a Create token-level pair conditioning input ($\mathbf{z}$)

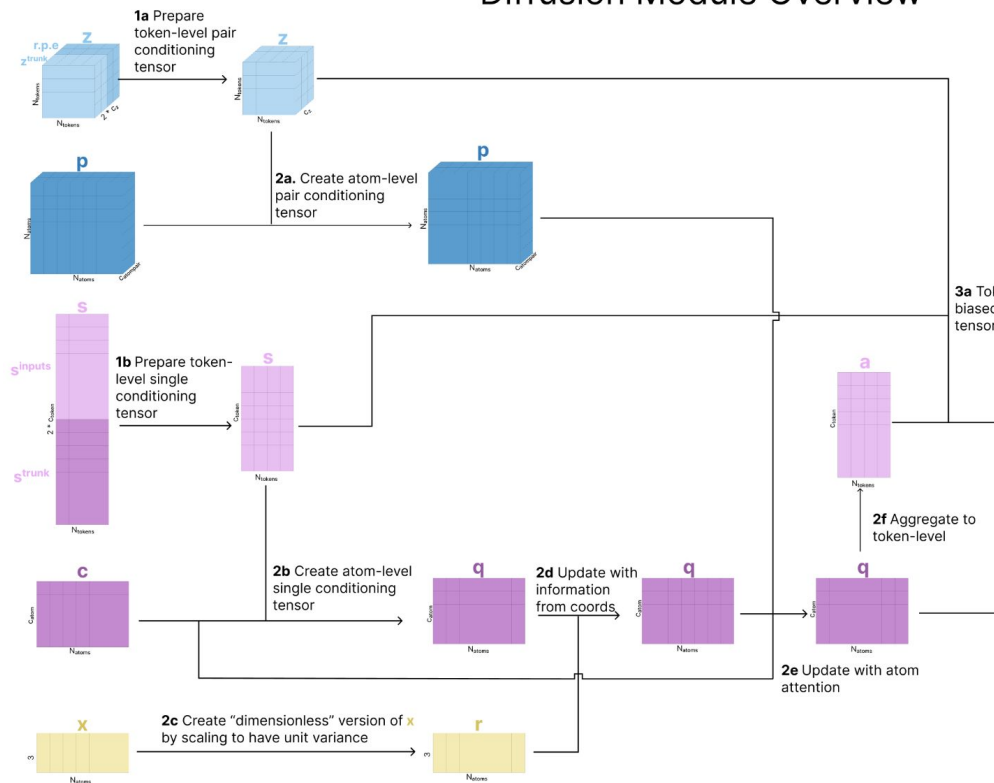


1b Create token-level single conditioning input ($\mathbf{s}$)



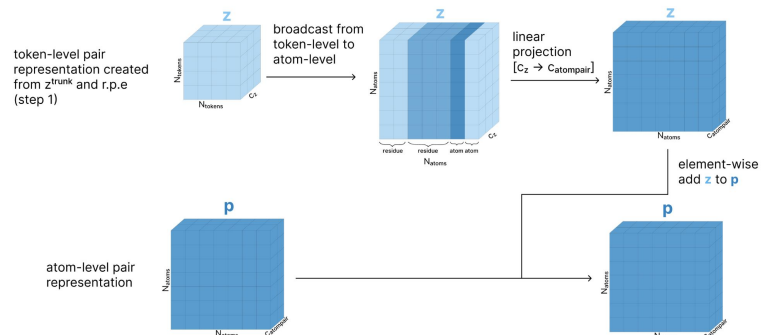
Diffusion Module - Part 2

Diffusion Module Overview



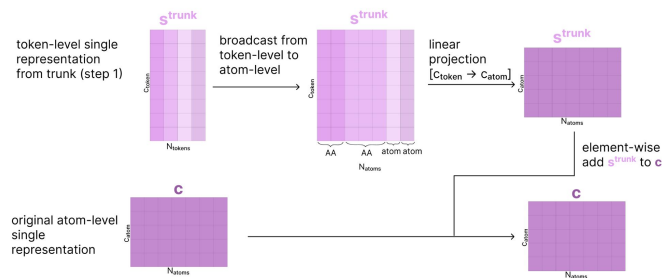
2a

Create atom-level pair conditioning input ($\mathbf{p}$)



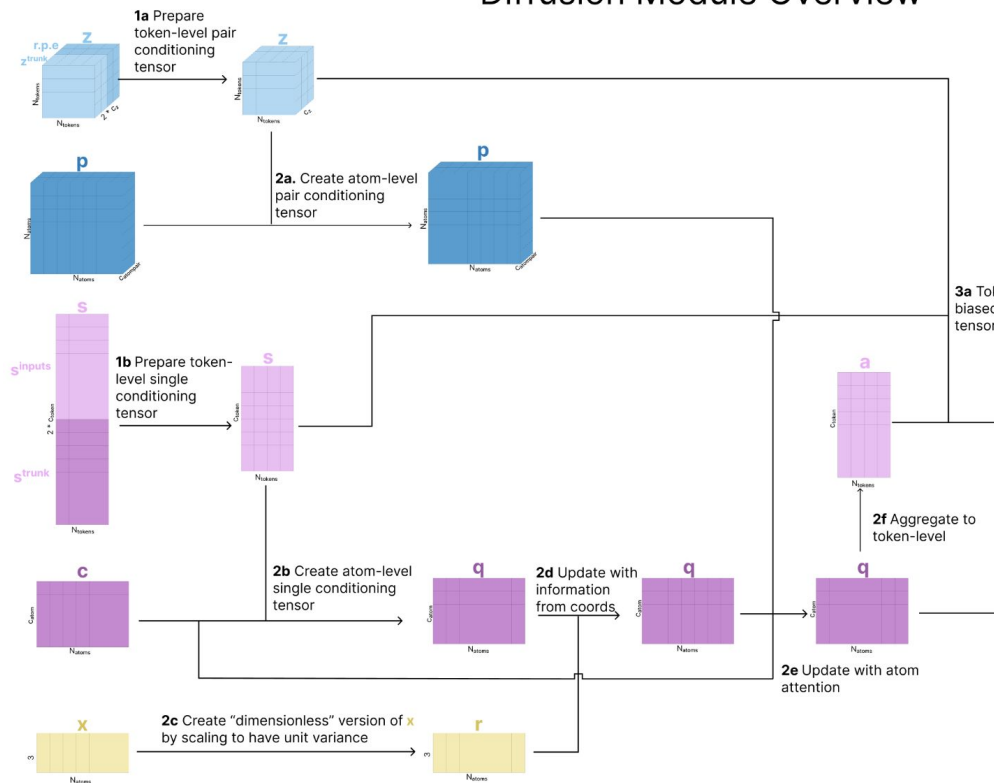
2b

Create atom-level single conditioning input ($\mathbf{c}$)



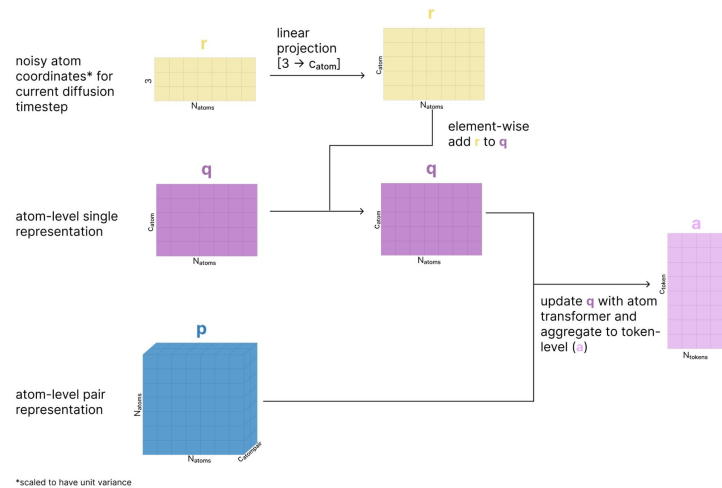
Diffusion Module - Part 2

Diffusion Module Overview



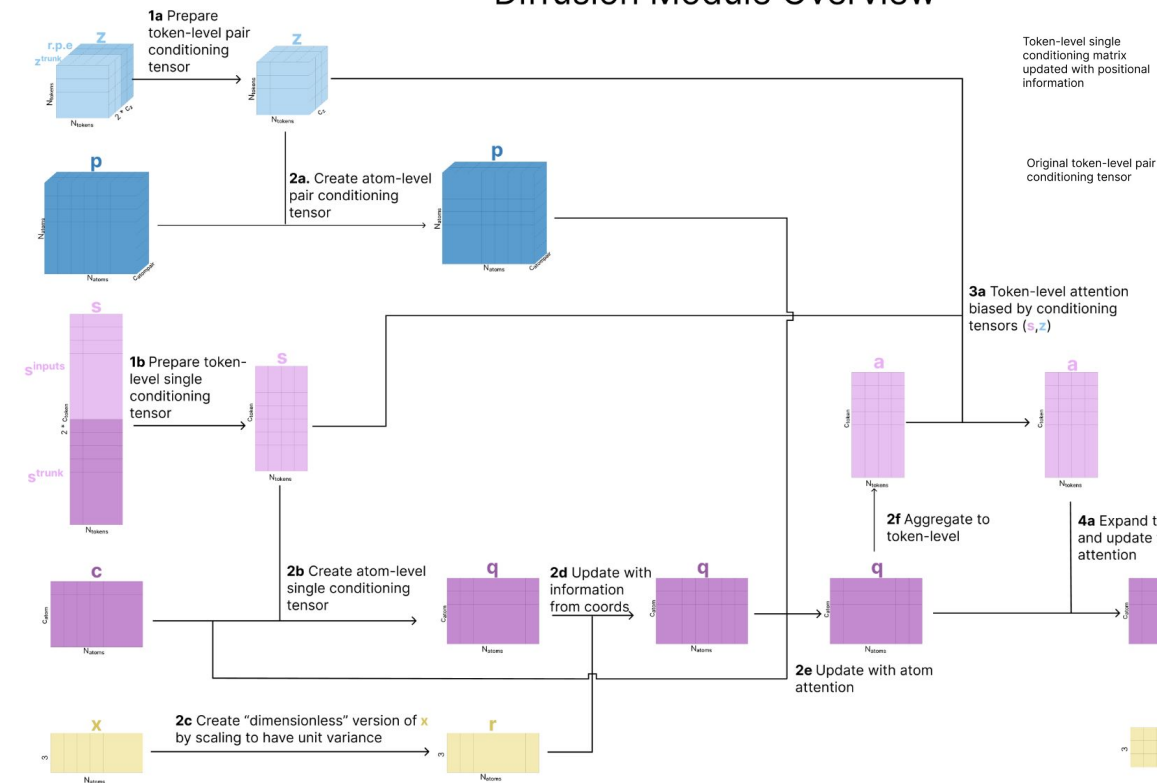
2c-f

Add noisy coordinates to atom-level representation and convert to token-level



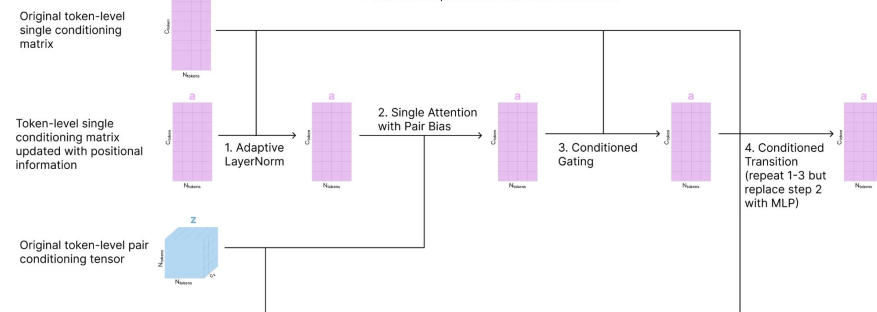
Diffusion Module - Part 3

Diffusion Module Overview



Overview of Diffusion Transformer

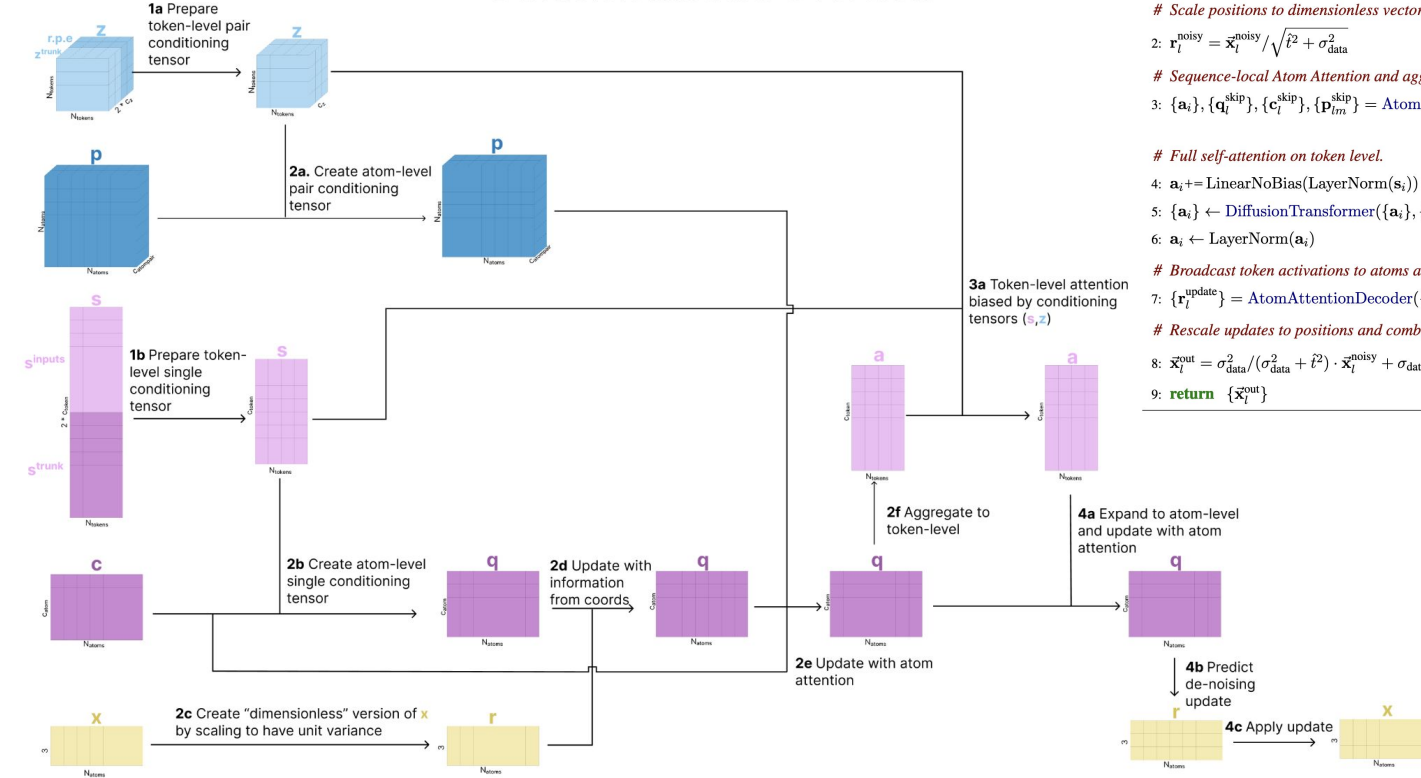
Token-level equivalent of the Atom Transformer



3a

Diffusion Module - Part 4

Diffusion Module Overview



Algorithm 20 Diffusion Module

```
def DiffusionModule( $\{\tilde{x}_l^{\text{noisy}}\}, \hat{t}, \{f^*\}, \{s_i^{\text{inputs}}\}, \{s_i^{\text{trunk}}\}, \{z_{ij}^{\text{trunk}}\}, \sigma_{\text{data}} = 16, c_{\text{atom}} = 128, c_{\text{atompair}} = 16, c_{\text{token}} = 768$ ) :
```

Conditioning

1: $\{s_i\}, \{z_{ij}\} = \text{DiffusionConditioning}(\hat{t}, \{f^*\}, \{s_i^{\text{inputs}}\}, \{s_i^{\text{trunk}}\}, \{z_{ij}^{\text{trunk}}\}, \sigma_{\text{data}})$

Scale positions to dimensionless vectors with approximately unit variance.

2: $r_l^{\text{noisy}} = \tilde{x}_l^{\text{noisy}} / \sqrt{\hat{t}^2 + \sigma_{\text{data}}^2}$

$r_l^{\text{noisy}} \in \mathbb{R}^3$

Sequence-local Atom Attention and aggregation to coarse-grained tokens

3: $\{a_i\}, \{q_i^{\text{skip}}\}, \{c_i^{\text{skip}}\}, \{p_{lm}^{\text{skip}}\} = \text{AtomAttentionEncoder}(\{f^*\}, \{r_l^{\text{noisy}}\}, \{s_i^{\text{trunk}}\}, \{z_{ij}\}, c_{\text{atom}}, c_{\text{atompair}}, c_{\text{token}})$
 $a_i \in \mathbb{R}^{c_{\text{token}}}$

Full self-attention on token level.

4: $a_i += \text{LinearNoBias}(\text{LayerNorm}(s_i))$

5: $\{a_i\} \leftarrow \text{DiffusionTransformer}(\{a_i\}, \{s_i\}, \{z_{ij}\}, \beta_{ij} = 0, N_{\text{block}} = 24, N_{\text{head}} = 16)$

6: $a_i \leftarrow \text{LayerNorm}(a_i)$

Broadcast token activations to atoms and run Sequence-local Atom Attention

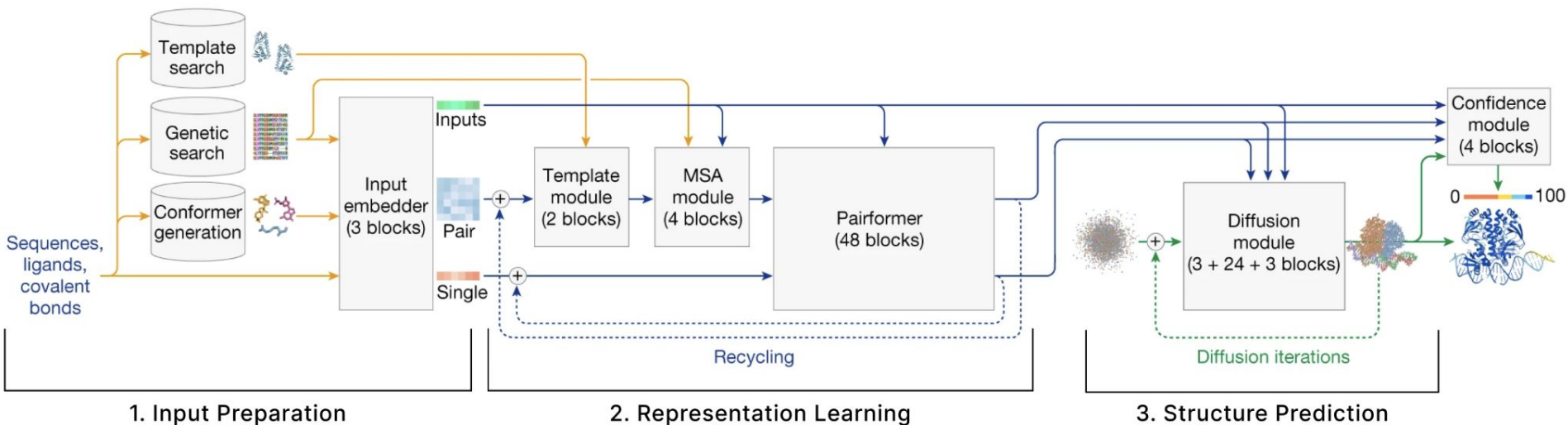
7: $\{r_l^{\text{update}}\} = \text{AtomAttentionDecoder}(\{a_i\}, \{q_i^{\text{skip}}\}, \{c_i^{\text{skip}}\}, \{p_{lm}^{\text{skip}}\})$

Rescale updates to positions and combine with input positions

8: $\tilde{x}_l^{\text{put}} = \sigma_{\text{data}}^2 / (\sigma_{\text{data}}^2 + \hat{t}^2) \cdot \tilde{x}_l^{\text{noisy}} + \sigma_{\text{data}} \cdot \hat{t} / \sqrt{\sigma_{\text{data}}^2 + \hat{t}^2} \cdot r_l^{\text{update}}$

9: **return** $\{\tilde{x}_l^{\text{put}}\}$

Architecture Overview



Loss function and confidence heads

$$L_{\text{loss}} = L_{\text{distogram}} * \alpha_{\text{distogram}} + L_{\text{diffusion}} * \alpha_{\text{diffusion}} + L_{\text{confidence}} * \alpha_{\text{confidence}}$$

The loss is a weighted sum of 3 terms:

- $L_{\text{distogram}}$ which evaluates the accuracy of the predicted distogram at a token-level
- $L_{\text{diffusion}}$ which evaluates the accuracy of the predicted distogram at an atom-level. It looks at all pairwise distances then includes additional terms to prioritize distances between nearby atoms and atoms involved in protein-ligand bonds.
- $L_{\text{confidence}}$ which evaluates the model's self-awareness about which structures are likely to be inaccurate

Diffusion Loss

$$L_{\text{diffusion}} = (L_{\text{MSE}} + L_{\text{bond}} \cdot \alpha_{\text{bond}}) \cdot \frac{\hat{t}^2 + \sigma_{\text{data}}^2}{(\hat{t} + \sigma_{\text{data}})^2} + L_{\text{smooth_lddt}}$$

- **LMSE**

- Mean squared error over *all* atom positions.
- Similar to distogram loss but not binned.
- DNA, RNA, ligand atoms upweighted.

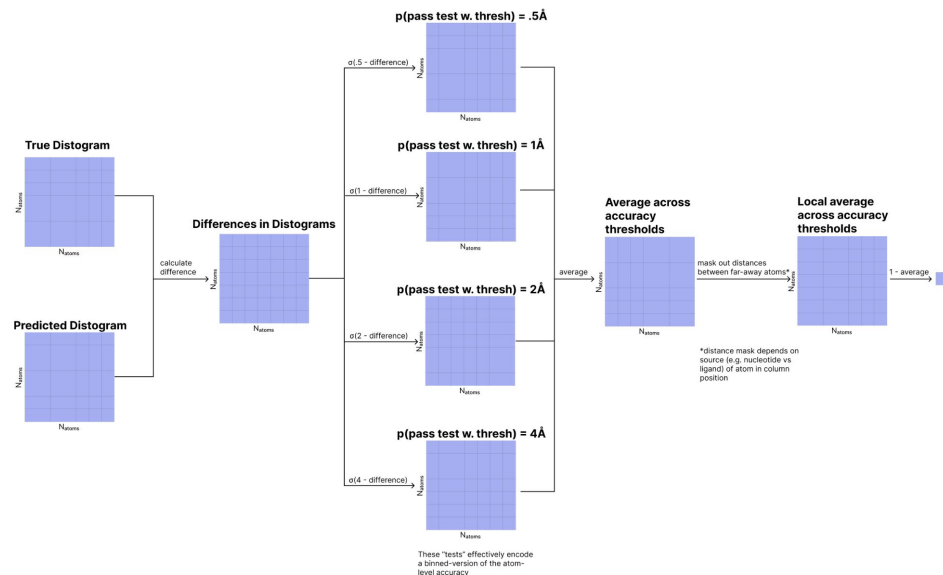
- **Lbond**

- Focuses on protein-ligand bond lengths.
- Extra MSE applied to protein-ligand atom pairs.

- **Lsmooth_LDDT**

- Soft version of LDDT (local distance difference test).
- Smooth probability of passing distance thresholds.
- Uses 4 thresholds: 4Å, 2Å, 1Å, 0.5Å.
- Ignores far atom pairs to focus on local accuracy.

Smooth Local Distance Difference Test (LDDT)

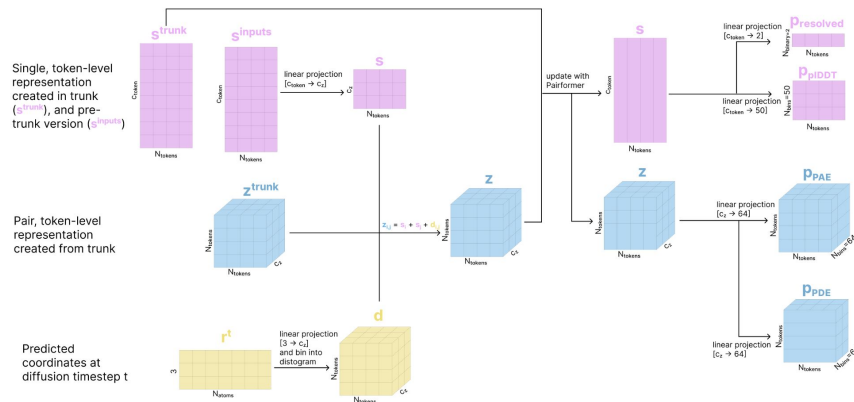


Confidence Loss

$$L_{\text{confidence}} = L_{\text{pIDDT}} + L_{\text{PDE}} + L_{\text{resolved}} + L_{\text{PAE}} \cdot \alpha_{\text{PAE}}$$

- **pIDDT (Predicted LDDT):**
Predicts atom-level local distance accuracy.
- **PAE (Predicted Alignment Error):**
Predicts how far a token is from its true position after aligning with another token.
- **PDE (Predicted Distance Error):**
Predicts errors in pairwise distances between tokens.
- **Experimentally Resolved Prediction:**
Predicts which atoms are experimentally resolved in the structure.

Confidence Head Architecture



Confidence Loss

$$L_{\text{confidence}} = L_{\text{plDDT}} + L_{\text{PDE}} + L_{\text{resolved}} + L_{\text{PAE}} \cdot \alpha_{\text{PAE}}$$

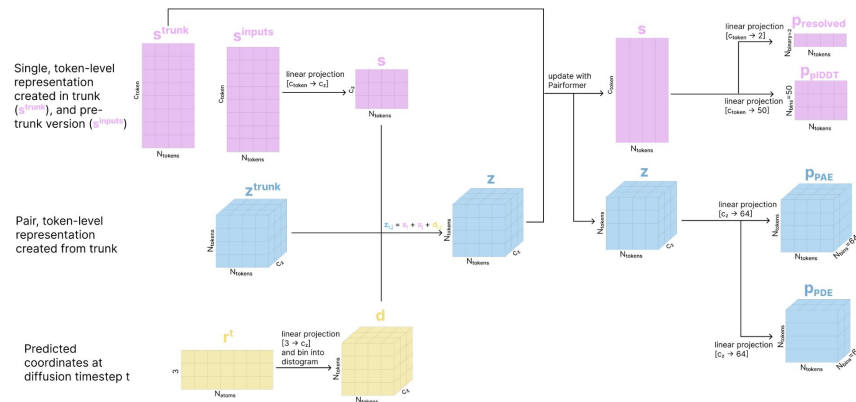
How Confidence Loss Works

- The model predicts error metrics during diffusion.
- True error metrics are calculated on predicted structures.
- Loss measures the difference between predicted vs actual errors.
- Even if structure is wrong, the loss is low if the model correctly predicts its own uncertainty.

Gradients Only Affect Confidence Heads

- Confidence loss updates only the confidence prediction heads.
- Main structure prediction model is unaffected.

Confidence Head Architecture



Other Training Details

Beyond architecture, several training strategies improve AF3 performance:

- Recycling
- Cross-distillation
- Cropping & staged training
- Clashing handling
- Batch size optimization

Recycling

- **Same weights reused multiple times.**
- Inputs pass through modules iteratively → progressively refined representations.
- Diffusion model inherently uses recycling during inference, using timestep information.
- Inspired by AlphaFold 2 (AF2).

Cross-Distillation

- AF3 uses:
 - **Self-distillation**: learning from its own predictions.
 - **Cross-distillation**: learning from AF2 and AF-Multimer predictions.
- Motivation:
 - Diffusion eliminates AF2's characteristic “spaghetti” low-confidence regions.
 - Including AF2 generations helps AF3 learn to output unfolded, low-confidence regions when appropriate.

Cropping & Training Stages

- Long sequences are computationally expensive.
- Cropping strategies:
 - **Contiguous cropping:** random continuous segments.
 - **Spatial cropping:** residues near a reference atom.
 - **Spatial interface cropping:** residues near binding interfaces.
- Progressive fine-tuning on longer sequences to generalize better.

Table 6 | Training stages

	Initial training	Fine tuning 1	Fine tuning 2	Fine tuning 3
Sequence crop size N_{token}	384	640	768	768
Parameters initialized from	Random	Initial training	Fine tuning 1	Fine tuning 2
Sampling weight for disorder PDB distillation	0.02	0.01	0.02	0.02
Train on transcription factor distillation sets	False	False	True	True
Masked diffusion loss for non-protein in disorder PDB distillation	True	False	False	False
Train structure and distogram	True	True	True	False
Train PAE head	False	False	False	True
Diffusion batch size	48	32	32	32
Training samples ($\cdot 10^6$)	≈ 20	≈ 1.5	≈ 1.5	≈ 1.8
Training times (days on 256 A100s)	≈ 10	≈ 3	≈ 5	≈ 2
Polymer-ligand bond loss weight	0	1	1	1
Max number of chains	20	20	20	50

Clashing

- No explicit clash penalty in diffusion loss.
- Overlapping atoms rarely occur after training.
- **Clash penalty applied only during ranking of generated structures.**

Batch Size Optimization

- Diffusion step is lighter than representation trunk.
- Strategy:
 - Pass input once through trunk.
 - Apply **48 parallel data augmentations** on trunk outputs.
 - Efficiently trains many augmented versions simultaneously.

ML Design Insights

Pair-Bias Attention

- Self-attention + additional pair-wise bias.

Self-Supervised Training

- AF3 does not use large-scale self-supervised pretraining on MSA.
- Possible reasons: compute cost, supervised hybrid model works better, mixed token types (AA, DNA, RNA, ligands).

Classification vs Regression

- Uses both MSE and binned classification losses.
- Binning may provide more stable gradients.

Similarities to Recurrent Networks

- Gating mechanisms (like LSTMs/GRUs).
- Iterative weight reuse.
- Adaptive computation reminiscent of recurrent models and ACT.

End of AF3

Thanks for listening!

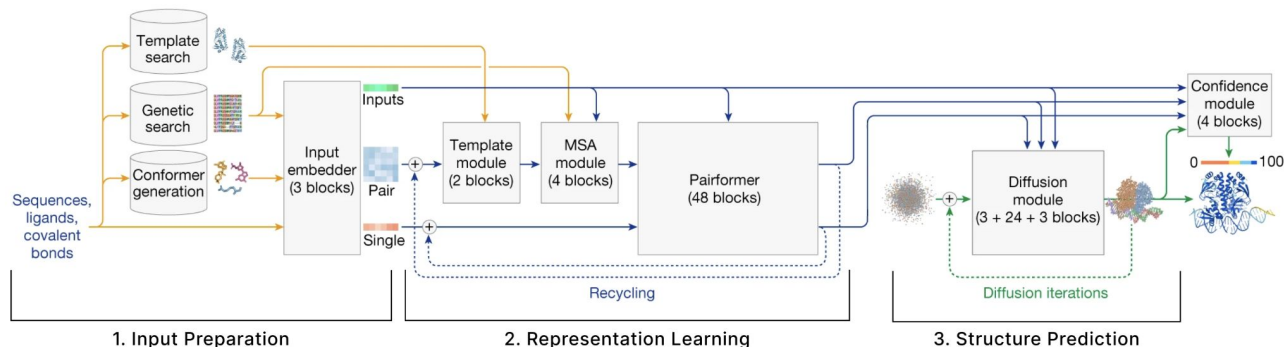
Questions - Comments?

Algorithm 1 Main Inference Loop

```

def MainInferenceLoop( $\{f^*\}$ ,  $N_{\text{cycle}} = 4$ ,  $c_s = 384$ ,  $c_z = 128$ ) :
1:  $\{s_i^{\text{inputs}}\} = \text{InputFeatureEmbedder}(\{f^*\})$ 
2:  $s_i^{\text{init}} = \text{LinearNoBias}(s_i^{\text{inputs}})$   $s_i^{\text{init}} \in \mathbb{R}^{c_s}$ 
3:  $z_{ij}^{\text{init}} = \text{LinearNoBias}(s_i^{\text{inputs}}) + \text{LinearNoBias}(s_j^{\text{inputs}})$   $z_{ij}^{\text{init}} \in \mathbb{R}^{c_z}$ 
4:  $z_{ij}^{\text{init}} += \text{RelativePositionEncoding}(\{f^*\})$ 
5:  $z_{ij}^{\text{init}} += \text{LinearNoBias}(r_{ij}^{\text{token\_bonds}})$ 
6:  $\{\hat{z}_{ij}\}, \{\hat{s}_i\} = \mathbf{0}, \mathbf{0}$ 
7: for all  $c \in [1, \dots, N_{\text{cycle}}]$  do
8:    $z_{ij} = z_{ij}^{\text{init}} + \text{LinearNoBias}(\text{LayerNorm}(\hat{z}_{ij}))$   $z_{ij} \in \mathbb{R}^{c_z}$ 
9:    $\{z_{ij}\} += \text{TemplateEmbedder}(\{f^*\}, \{z_{ij}\})$ 
10:   $\{z_{ij}\} += \text{MsaModule}(\{f_{s_i}^{\text{msa}}\}, \{z_{ij}\}, \{s_i^{\text{inputs}}\})$ 
11:   $s_i = s_i^{\text{init}} + \text{LinearNoBias}(\text{LayerNorm}(\hat{s}_i))$   $s_i \in \mathbb{R}^{c_s}$ 
12:   $\{s_i\}, \{z_{ij}\} = \text{PairformerStack}(\{s_i\}, \{z_{ij}\})$ 
13:   $\{\hat{s}_i\}, \{\hat{z}_{ij}\} \leftarrow \{s_i\}, \{z_{ij}\}$ 
14: end for
15:  $\{\bar{x}_i^{\text{pred}}\} = \text{SampleDiffusion}(\{f^*\}, \{s_i^{\text{inputs}}\}, \{s_i\}, \{z_{ij}\})$ 
16:  $\{p_i^{\text{pddt}}\}, \{p_i^{\text{pae}}\}, \{p_{ij}^{\text{pde}}\}, \{p_i^{\text{resolved}}\} = \text{ConfidenceHead}(\{s_i^{\text{inputs}}\}, \{s_i\}, \{z_{ij}\}, \{\bar{x}_i^{\text{pred}}\})$ 
17:  $p_{ij}^{\text{distogram}} = \text{DistogramHead}(z_{ij})$ 
18: return  $\{\bar{x}_i^{\text{pred}}\}, \{p_i^{\text{pddt}}\}, \{p_i^{\text{pae}}\}, \{p_{ij}^{\text{pde}}\}, \{p_i^{\text{resolved}}\}, \{p_{ij}^{\text{distogram}}\}$ 

```



Resource

-  [The Illustrated AlphaFold](#)
-  [AlphaFold3 and its improvements in comparison to AlphaFold2](#)
-  [AlphaFold2, AlphaFold-Multimer, AlphaFold3](#)
-  ["Sparks of Chemical Intuition"-and Gross Limitations!-in AlphaFold 3](#)
-  [AlphaFold3 Explained](#)
-  [AlphaFold 3 deep dive | Looking Glass Universe](#)
-  [Introduction to diffusion models for machine learning](#)
-  [What are Diffusion Models?](#)
-  [Diffusion Models for AI Image Generation | IBM Technology](#)